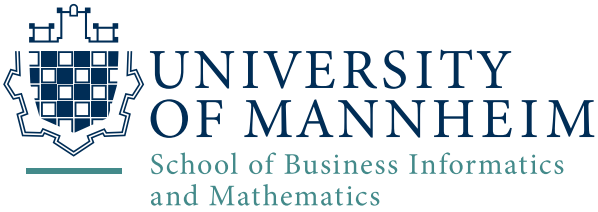


Master's Thesis

Applications of Quasi-Monte Carlo Methods in Neural Network Training and Medical Imaging

Author:
Janik Valentin Hrubant

Supervisors:
Prof. Dr. Andreas Neuenkirch
Prof. Dr. Simone Göttlich

A thesis submitted in fulfillment of the requirements for the degree of Master of Science

September 15, 2025

Declaration of Authorship

English Version. I hereby declare that I have completed this thesis independently and without any unauthorized assistance. I further confirm that neither this thesis nor any part of it has been submitted by me or by others as part of any other academic assessment. Literal or paraphrased quotations from other sources—whether in print or electronic form—are clearly marked as such. All secondary literature and other sources used are properly cited and listed in the bibliography. This also applies to graphical representations, images and all online sources. I furthermore agree that my thesis may be submitted and stored in anonymized electronic form for the purpose of plagiarism detection. I am aware that the thesis may not be evaluated if this declaration is not submitted.

German Version. Hiermit versichere ich, dass diese Arbeit von mir persönlich verfasst wurde und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen. Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn diese Erklärung nicht erteilt wird.

Place, Date

Signature

Abstract

This thesis investigates Quasi-Monte Carlo (QMC) methods from their theoretical foundations to applications in machine learning and medical imaging. QMC methods replace random sampling by low-discrepancy sequences, thereby improving uniformity and under suitable smoothness conditions convergence rates compared to classical Monte Carlo (MC) sampling.

Part I develops the mathematical framework of QMC integration, introducing concepts such as discrepancy, Hardy–Krause variation, and the Koksma–Hlawka inequality, which provides a deterministic error bound and serves as the theoretical basis for the subsequent studies.

Part II examines the use of QMC sampling in the training of Deep Neural Network (DNN). By revisiting and refining recent theoretical results, it is shown that applying the Koksma–Hlawka inequality to the generalization gap requires stricter regularity assumptions than previously stated. Empirical experiments on benchmark problems confirm that QMC-based training sets can reduce generalization error and thus constitute a promising alternative to random sampling in many-query problems.

Part III introduces a QMC-based framework for photon transport simulation in X-ray imaging. By integrating QMC sampling into photon generation and propagation, the framework achieves measurable improvements in accuracy and noise reduction compared to standard MC simulations across different phantom geometries. These results highlight the potential of QMC methods for scatter estimation in computed tomography.

Overall, the thesis demonstrates both the mathematical foundations and the practical versatility of QMC methods, establishing them as valuable tools across distinct domains of computational mathematics.

Acknowledgements

The acknowledgments and the people to thank go here, don't forget to include your project advisor...

Contents

Acronyms

CDF Cumulative Distribution Function

QMC Quasi-Monte Carlo

MC Monte Carlo

HU Hounsfield Unit

CT Computed Tomography

DNN Deep Neural Network

ODE Ordinary Differential Equation

SGD Stochastic Gradient Descent

Adam Adaptive Moment Estimation

CDF Cumulative Distribution Function

Chapter 1

Introduction

The numerical evaluation of high-dimensional integrals is a recurring challenge in applied mathematics, computational physics, and modern data science. In many cases, such integrals cannot be computed in closed form and must be approximated numerically. The Monte Carlo (MC) method has long been the standard approach, valued for its simplicity and dimension-independent convergence rate. However, its stochastic nature leads to slow convergence, with an error rate of order $O(N^{-1/2})$, which quickly becomes a bottleneck in applications that demand high accuracy or involve costly function evaluations.

Quasi-Monte Carlo (QMC) methods offer a deterministic alternative. By replacing random samples with carefully constructed low-discrepancy sequences, QMC aims to distribute sampling points more uniformly across the integration domain. Under suitable smoothness assumptions, this improved equidistribution yields substantially faster convergence rates. The theoretical framework, culminating in the Koksma–Hlawka inequality, provides deterministic error bounds and connects discrepancy of the sampling set with the variation of the integrand. QMC methods have therefore attracted increasing attention both in numerical analysis and in practical fields where high-dimensional integration is unavoidable.

The central objective of this thesis is to investigate the theoretical properties and practical benefits of QMC methods in comparison to classical Monte Carlo sampling. The thesis explores how QMC techniques can provide advantages in high-dimensional problems and illustrates their effectiveness in two distinct application areas: the training of deep neural networks and the simulation of photon transport in X-ray imaging. By combining rigorous mathematical foundations with applied case studies, the work aims to highlight both the potential and the limitations of QMC approaches in contemporary computational mathematics.

This objective is pursued in three parts:

- **Part I** develops the mathematical foundations of QMC methods for integral approximation. Central notions such as uniform distribution modulo one, discrepancy, and Hardy–Krause variation are introduced. The Koksma–Hlawka inequality is then derived, offering a deterministic error bound that motivates the later applications.
- **Part II** applies QMC sampling to the training of deep neural networks (DNNs). Recent work by Mishra and Rusch [21] suggests that the generalization error of DNNs can be analyzed in analogy to integration error bounds, with

low-discrepancy sequences reducing the generalization gap compared to random sampling. A central theorem in this context is revisited and proven under stricter assumptions, namely that the target function must exhibit L^1-continuity of all first-order mixed partial derivatives. Beyond the theoretical contribution, the approach is evaluated empirically on two representative problems: a high-dimensional sum-of-sines function and a surrogate for projectile motion. The results provide evidence that QMC sampling is a promising choice for generating training data in many-query problems.

- **Part III** explores a second, independent application: X-ray photon imaging simulation. Here, QMC methods are employed in the context of Monte Carlo-based photon transport, a key tool for modeling scatter in computed tomography (CT). This part begins with an introduction to the physics of X-ray generation, attenuation, and scattering. Inspired by the promising results of the gQMCFFD algorithm in [17], a simulation framework is then developed, incorporating QMC-based sampling strategies into the photon generation and propagation steps. The framework is evaluated on different phantom geometries, showing measurable improvements in accuracy and noise reduction compared to standard Monte Carlo simulations.

It is important to emphasize that Parts II and III are independent of one another: both address applications of QMC methods, but in distinct problem classes. This dual perspective is intended not as a limitation but as a strength, showcasing the breadth of QMC techniques and their potential across disciplines.

Structure of the Thesis

The remainder of the thesis is structured as follows:

- **Part I (Chapters 1–3)** introduces Monte Carlo and Quasi-Monte Carlo integration. Chapter 2 motivates the problem of high-dimensional integration and explains the principle and convergence of MC methods. Chapter 3 presents discrepancy theory and low-discrepancy sequences, with a focus on the Halton and Sobol' sequences. Chapter 4 introduces notions of function variation and derives the Koksma–Hlawka inequality.
- **Part II (Chapters 4–7)** applies QMC methods to neural network training. Chapter 5 motivates many-query problems in scientific computing and formulates the problem setting. Chapter 6 introduces the theoretical foundations of deep neural networks, including function approximation and optimization. Chapter 7 develops a QMC-based training algorithm and provides theoretical error estimates. Chapter 8 presents empirical evaluations and discusses the results.
- **Part III (Chapters 8–11)** turns to photon transport simulation. Chapter 9 motivates the problem of scatter in X-ray imaging. Chapter 10 introduces the underlying physical models of X-ray generation, attenuation, and scattering. Chapter 11 presents the simulation algorithm, combining these physical laws with QMC-based sampling strategies. Chapter 12 evaluates the framework on different phantoms and discusses results, limitations, and future work.

Part I

Fundamentals of Quasi-Monte Carlo Methods

Chapter 2

Monte Carlo and Quasi-Monte Carlo Integration

2.1 Motivation and Problem Setting

The numerical evaluation of high-dimensional integrals is a fundamental task in modern applied mathematics and scientific computing. Applications range from Bayesian inference and financial mathematics to machine learning and computational physics. In particular, contemporary domains such as deep neural network training and medical imaging often require the estimation of integrals of the form

$$I(f) = \int_{[0,1]^s} f(x)\,dx, \tag{2.1}$$

where s denotes the dimensionality of the problem and f is a function for which the integral is expensive or impractical to evaluate analytically. Throughout this thesis, we will limit our discussion on functions of the type $f : \mathbb{R}^s \to \mathbb{R}$.

One of the most widely used approaches to compute such integrals is the classical MC method, which estimates the integral based on averages over randomly sampled points $\{X_n\}_{n=0}^{N-1}$

$$\frac{1}{N}\sum_{n=0}^{N-1} f(X_n) \approx \int_{[0,1]^s} f(x)\,dx.$$

The primary appeal of MC integration lies in its dimension-independent convergence rate and minimal assumptions on the integrand. However, its asymptotic error rate of $\mathcal{O}(N^{-1/2})$ [16] limits its efficiency — especially when high precision is required or function evaluations are computationally expensive.

QMC methods offer an alternative paradigm: instead of random samples, they employ deterministic sequences — so-called low-discrepancy sequences — that aim to fill the integration domain more uniformly. This structured sampling allows for faster convergence under certain smoothness conditions and forms the basis for many state-of-the-art techniques in high-dimensional numerical integration.

The first part of this thesis is consists of three chapters. Chapter 2 introduces the concepts of MC and QMC methods to estimate the integral from (2.1). Chapter 3

provides a formal introduction to discrepancy theory and some constructions of low-discrepancy sequences. In Chapter 4, the concepts of variation and error bounds such as the *Koksma-Hlawka inequality* are introduced. The subsequent Part II of this thesis focuses on the application of QMC methods for training of neural networks, while Part III applies QMC techniques to photon transport simulation in Computed Tomography.

2.2 Monte Carlo Integration: Principle and Convergence

The classical MC method is a probabilistic approach for numerically evaluating the integral $I(f)$ as defined in (2.1). The goal is to approximate this integral by averaging function evaluations at randomly sampled points in the integration domain $[0,1]^s$.

Definition 2.2.1 (Monte Carlo Estimator)**.**
Let $f \in L^2([0,1]^s)$ and let $X_0, \dots, X_{N-1} \overset{\text{i.i.d.}}{\sim} \mathcal{U}([0,1]^s)$ be independent and identically distributed drawn random samples. Then the Monte Carlo estimator of the integral I from (2.1) is given by

$$I_N^{\text{MC}}(f) := \frac{1}{N} \sum_{n=0}^{N-1} f(X_n). \tag{2.2}$$

Remark 2.2.2.
Let $f \in L^1([0,1]^s)$. Then by the Strong Law of Large Numbers, the Monte Carlo Estimator from (2.2) converges almost surely to the integral $I(f)$ from (2.1) as the number of samples N tends to infinity:

$$\mathbb{P}\left(\lim_{N\to\infty} I_N^{\text{MC}}(f) = \int_{[0,1]^s} f(\mathbf{x})\, \mathrm{d}\mathbf{x} \right) = 1, \tag{2.3}$$

i.e. $I_N^{\text{MC}}(f) \xrightarrow{\text{a.s.}} \int_{[0,1]^s} f(\mathbf{x})\, \mathrm{d}\mathbf{x}$ for $N \to \infty$.

Beyond the almost sure convergence by the Strong Law of Large Numbers, the MC estimator has a well-known convergence rate as stated in [16, Section 1.2].

Theorem 2.2.3 (Monte Carlo Convergence Rate)**.**
Let $f \in L^2([0,1]^s)$. Then

$$\mathbb{E}\left[\left| I_N^{\text{MC}}(f) - I(f) \right| \right] \leq \frac{\sigma[f]}{\sqrt{N}}, \tag{2.4}$$

where $\sigma[f] := \sqrt{\text{Var}[f]}$.

Sketch of Proof.
By independence,

$$\text{Var}\left(I_N^{\text{MC}}(f) \right) = \frac{\text{Var}[f]}{N}.$$

With $X = I_N^{\mathrm{MC}}(f) - I(f)$, Jensen's inequality for the concave $\phi(t) = \sqrt{t}$ gives

$$\mathbb{E}[|X|] = \mathbb{E}[\phi(X^2)] \leq \phi(\mathbb{E}[X^2]) = \sqrt{\mathrm{Var}(X)}.$$

This yields the stated $O(N^{-1/2})$ bound. □

Despite its robustness and simplicity, MC integration has several well-known limitations:

- The convergence rate is relatively slow, especially for smooth integrands.
- Error bounds are probabilistic rather than deterministic.
- The method does not exploit structural properties of the integrand, such as smoothness or sparsity.

These limitations arise from the imperfect uniformity of random sample points in the hypercube $[0,1]^s$, a phenomenon that motivates the introduction of the concepts of uniformity and discrepancy in the next section.

Remark 2.2.4.
In this thesis, all random samples used in Monte Carlo procedures are produced by deterministic pseudorandom number generators. As a result, the simulated draws only approximate independent and identically distributed uniform variables on the interval $[0,1]$. Unless stated otherwise, the statistical quality of the generator is assumed to be sufficient for the results reported here. A systematic study of the choice of generator and its influence on accuracy or variance lies outside the scope of this work.

2.3 Uniformity Concepts

The efficiency of QMC methods hinges on how well the employed point sets "fill" the integration domain $[0,1]^s$. This motivates the need for a precise mathematical understanding of *uniformity*. The present section introduces two key concepts in this regard — *uniform distribution modulo one* and *equidistribution* — and clarifies their relevance to numerical integration.

2.3.1 Uniform Distribution Modulo One

In QMC methods, the notion of *uniform distribution modulo one* provides a rigorous criterion for how evenly a sequence covers the unit cube. This concept forms the foundation for analyzing the convergence behavior of QMC estimators.

Definition 2.3.1 (Uniform Distribution Modulo One)**.**
Let $(\boldsymbol{x}_n)_{n\in\mathbb{N}_0} \subset [0,1]^s$ be a sequence of sample points. The sequence is said to be *uniformly distributed modulo one* (u.d. mod 1) if for every axis-aligned box $[\boldsymbol{a},\boldsymbol{b}) \subset [0,1]^s$, the proportion of points falling into this box converges to its Lebesgue measure as $N \to \infty$:

$$\lim_{N\to\infty} \frac{1}{N} \sum_{n=0}^{N-1} \chi_{[\boldsymbol{a},\boldsymbol{b})}(\boldsymbol{x}_n) = \lambda_s([\boldsymbol{a},\boldsymbol{b})),$$

where $\chi_{[\boldsymbol{a},\boldsymbol{b})}$ is the indicator function of the set $[\boldsymbol{a},\boldsymbol{b})$ and λ_s denotes the s-dimensional Lebesgue measure.

Here $[\boldsymbol{a},\boldsymbol{b})$ denotes the half-open hypercube in $\mathbb{R}^s$ defined by

$$[\boldsymbol{a},\boldsymbol{b}) = \{\boldsymbol{x} \in \mathbb{R}^s \mid a_i \le x_i < b_i \text{ for } i = 1,\dots,s\}.$$

This definition emphasizes asymptotic spatial coverage: in the limit, every subregion of the domain is sampled in proportion to its volume. Importantly, this property is purely deterministic and does not rely on any probabilistic assumptions —- in contrast to Monte Carlo methods, which only guarantee uniformity in expectation.

In contrast, when speaking of a random sample $\{X_1,\dots,X_N\} \subset [0,1]^s$ drawn independent and identically distributed (i.i.d.) from the uniform distribution, we refer to a probabilistic concept of uniformity: each point is independently drawn according to the uniform measure, but the empirical distribution may not be uniformly spread in finite samples. Thus, equidistribution refers to the ideal uniform coverage that we seek deterministically, while uniform random sampling only ensures this behavior in expectation or with high probability. In Figure 2.1, a randomly sampled set of 300 points in $[0,1]^2$ is compared to the first 300 sequence elements from a QMC sequence. The latter sequence is u.d. mod 1.

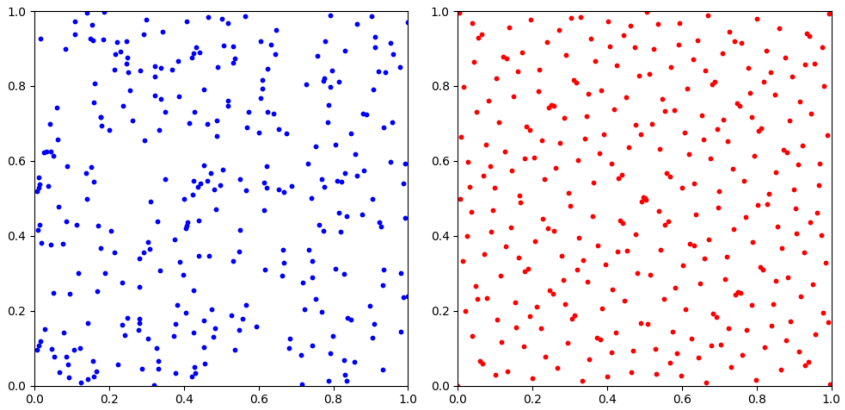


Figure 2.1: Comparison of 300 randomly drawn sample points in $[0,1]^2$ using standard MC (left) and the first 300 elements of the Sobol' sequence in 2 dimensions.

Remark 2.3.2.
Uniform distribution modulo one is a necessary condition for the convergence of

QMC estimators. If a point sequence is not u.d. mod 1, then there exists at least one Riemann-integrable function for which the sample average

$$\frac{1}{N}\sum_{n=1}^{N} f(x_n)$$

fails to converge to the integral. [16, Section 2.1]

The following result, known as Weyl's criterion, provides a necessary and sufficient condition for uniform distribution in terms of exponential sums.

Theorem 2.3.3 (Weyl's Criterion)**.**
A sequence $(\boldsymbol{x}_n)_{n\geq 0} \subset [0,1)^s$ is uniformly distributed modulo one if and only if, for all nonzero $\boldsymbol{h} \in \mathbb{Z}^s \setminus \{\boldsymbol{0}\}$, we have

$$\lim_{N\to\infty} \frac{1}{N}\sum_{n=0}^{N-1} \exp(2\pi i\, \boldsymbol{h}\cdot\boldsymbol{x}_n) = 0.$$

A proof can be found in [16, Section 2.1].

Example 2.3.4.
Let $\alpha \in \mathbb{R}$ be irrational. Then the Kronecker sequence $(n\alpha \bmod 1)_{n\geq 0}$ is uniformly distributed modulo one in $[0,1]$. This is a consequence of Weyl's criterion and illustrates the existence of simple, deterministic sequences with excellent uniformity properties.

Chapter 3

Discrepancy Theory and Low-Discrepancy Sequences

3.1 Measuring Uniformity: Discrepancy

One of the cornerstones of QMC theory is the concept of *discrepancy*, which quantifies how uniformly a finite point set samples the s-dimensional unit cube $[0,1]^s$. While uniform distribution modulo one describes the asymptotic behavior of infinite sequences, discrepancy provides a finite-sample measure of deviation from perfect uniformity. Low-discrepancy sequences are the foundation of QMC methods because they minimize this deviation, thus yielding more accurate numerical integration.

This section introduces formal definitions, geometric intuition and empirical insights into discrepancy measures. It lays the theoretical groundwork for understanding how the structure of point sets affects QMC integration performance.

3.1.1 Definition of Discrepancy and Star Discrepancy

First, we define the more general *extreme discrepancy*:

Definition 3.1.1 (Extreme Discrepancy)**.**
Let $\mathcal{P} \subset [0,1)^s$ with $|\mathcal{P}| = N$ be a finite point set. Then the extreme discrepancy $D_N(\mathcal{P})$ is defined as

$$D_N(\mathcal{P}) = \sup_{\substack{\boldsymbol{a},\boldsymbol{b}\in[0,1]^s \\ \boldsymbol{a}\leq\boldsymbol{b}}} \left| \frac{A([\boldsymbol{a},\boldsymbol{b}),\mathcal{P})}{N} - \lambda_s([\boldsymbol{a},\boldsymbol{b})) \right|.$$

Here $A([\boldsymbol{a},\boldsymbol{b}),\mathcal{P})$ denotes the number of points in $\mathcal{P}$ that fall into the box $[\boldsymbol{a},\boldsymbol{b})$, and $\lambda_s([\boldsymbol{a},\boldsymbol{b}))$ is the Lebesgue measure of that box, given by $\prod_{i=1}^{s}(b_i - a_i)$.

In practice, a common and slightly more tractable variant is the *star discrepancy*, which restricts the test boxes to be anchored at the origin.

Definition 3.1.2 (Star Discrepancy)**.**
Let $\mathcal{P} \subset [0,1)^s$ with $|\mathcal{P}| = N$ be a finite point set. Then the star discrepancy D_N^* is defined as

$$D_N^*(\mathcal{P}) = \sup_{\boldsymbol{t} \in [0,1]^s} \left| \frac{A([\mathbf{0}, \boldsymbol{t}), \mathcal{P})}{N} - \lambda_s([\mathbf{0}, \boldsymbol{t})) \right|.$$

This form is used in the Koksma–Hlawka inequality (Theorem 4.2.1) and serves as a central measure in QMC analysis. Note that both discrepancy definitions are deterministic and depend solely on the point configuration.

Remark 3.1.3.
A low star discrepancy implies that the empirical distribution of the points approximates the uniform distribution well over all axis-aligned subrectangles anchored at the origin.

3.1.2 Geometric Interpretation and Examples

To build geometric intuition, consider the one-dimensional case. Given N sample points $x_0, \ldots, x_{N-1} \in [0,1)$, we can visualize discrepancy as the maximum vertical deviation between the empirical distribution function

$$F_N(t) := \frac{1}{N} \sum_{n=0}^{N-1} \chi_{[0,t)}(x_n)$$

and the uniform cumulative distribution function $F(t) = t$. The discrepancy corresponds to the largest gap between $F_N(t)$ and $F(t)$.

In higher dimensions, the idea generalizes: the star discrepancy measures the maximal difference in the number of points falling into an axis-aligned box $[\mathbf{0}, \boldsymbol{t})$ versus the volume of that box. Intuitively, it quantifies whether the point set "overpopulates" or "underpopulates" certain regions.

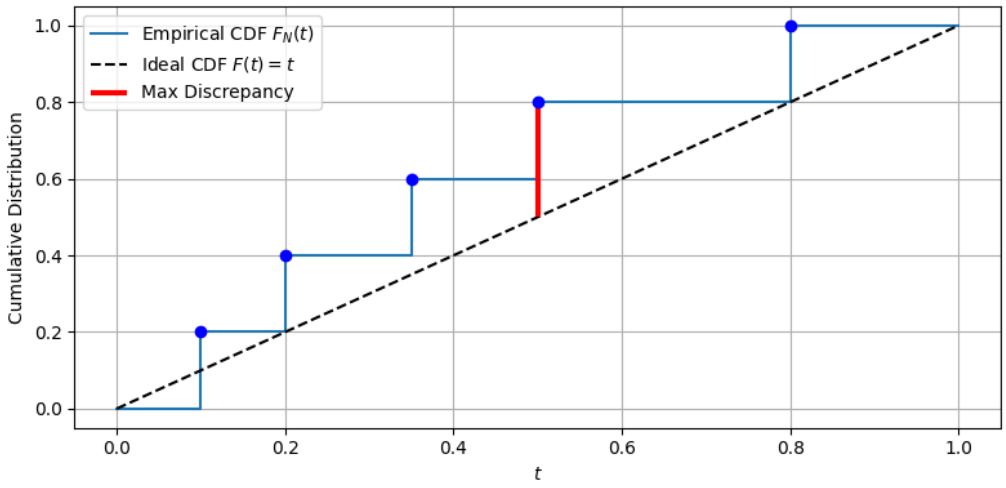


FIGURE 3.1: Visualization of 1D discrepancy: the maximum vertical distance between the empirical CDF $F_N(t)$ and the uniform CDF $F(t) = t$.

Example 3.1.4.
Let $x_n = \frac{n}{N}$ for $n = 0,\dots,N-1$. Then $F_N(t)$ is a piecewise constant staircase function. With Theorem 2.4 from [35], one gets that the discrepancy is $\frac{1}{N}$. This is an optimal rate in one dimension.

Remark 3.1.5.
Discrepancy provides a worst-case error metric over all subintervals. Thus, even a point set that appears visually well-distributed may have large discrepancy due to subtle gaps or clustering in certain regions.

3.1.3 Empirical Observation of Discrepancy in QMC

Discrepancy is not just a theoretical construct — its practical relevance becomes evident when comparing the behavior of QMC sequences with MC samples. In particular, structured point sets like Halton and Sobol' sequences as introduced in the next section, achieve much lower discrepancy than random samples of the same size.

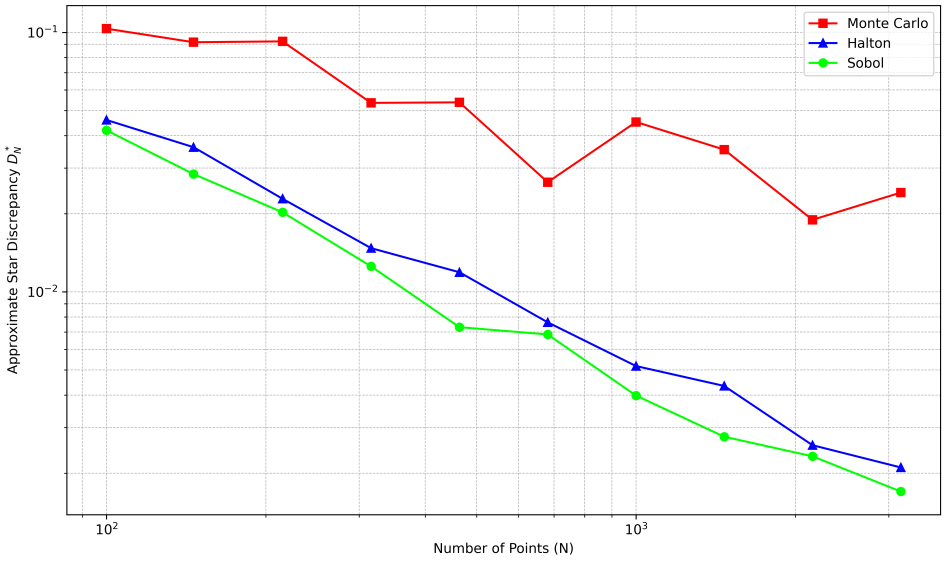


FIGURE 3.2: Comparison of discrepancy growth for Sobol', Halton and random (MC) point sets in two dimensions. Low-discrepancy sequences exhibit significantly slower discrepancy growth.

Remark 3.1.6.
The expected star discrepancy of i.i.d. Monte Carlo samples decreases at the rate $\mathcal{O}(N^{-1/2})$, whereas for low-discrepancy sequences provable upper bounds of order $\mathcal{O}((\log N)^s/N)$ exist. [16, Section 2.2]

3.2 Low-Discrepancy Sequences

Now we will introduce the concept of low-discrepancy sequences and analyze their dimensional performance properties. We start by defining the term of low-discrepancy sequences, which are designed to fill the unit cube $[0,1)^s$ as uniformly as possible.

Definition 3.2.1 (Low-Discrepancy Sequence)**.**
A point set $\mathcal{P} = \{\boldsymbol{x}_0, \dots, \boldsymbol{x}_{N-1}\} \subset [0,1)^s$ with $|\mathcal{P}| = N$ is called a low-discrepancy point set if its star discrepancy satisfies

$$D_N^*(\mathcal{P}) = \mathcal{O}\left(\frac{(\log N)^s}{N}\right). \tag{3.1}$$

A sequence $\mathcal{S} = (x_n)_{n\in\mathbb{N}}$ in $[0,1)^s$ is called a low-discrepancy sequence if its star discrepancy satisfies

$$D_N^*(\mathcal{S}) = \mathcal{O}\left(\frac{(\log N)^s}{N}\right) \tag{3.2}$$

for all $N \in \mathbb{N}$. Such sequences are also referred to as *Quasi-Monte Carlo sequences* or QMC sequences.

Remark 3.2.2.
Low-discrepancy sequences are constructed to minimize the star discrepancy. For dimension $s = 1$ and $s = 2$, it is known that (3.1) is an optimal bound. K. F. Roth proved in 1954 that for every dimension $s \in \mathbb{N}$ there exists a constant $c_s > 0$ such that for every point set $\mathcal{P} \subset [0,1)^s$ with $|\mathcal{P}| = N$ the lower bound

$$D_N(\mathcal{P}) \geq D_N^*(\mathcal{P}) \geq c_s \frac{(\log N)^{\frac{s-1}{2}}}{N}$$

holds [16, Theorem 2.24]. It is still an open problem whether the bound also holds with exponent $s-1$ instead of $\frac{s-1}{2}$ for $s \geq 3$. Nevertheless, since this is conjectured to be true, it has become common in literature to refer to sequences satisfying (3.2) as low-discrepancy sequences. The bound on the discrepancy is crucial for the convergence of QMC methods according to the Koksma–Hlawka inequality introduced in Chapter 4.

With the definition of a low-discrepancy sequence in place, we now get back to the MC estimator from 2.2.1 and define its equivalent QMC estimator:

Definition 3.2.3 (Quasi-Monte Carlo Estimator)**.**
Let $f\colon [0,1]^s \to \mathbb{R}$ be a measurable function and let $\{\boldsymbol{x}_n\}_{n=1}^N \subset [0,1]^s$ be a QMC point set. Then the *Quasi-Monte Carlo estimator* for the integral

$$I(f) = \int_{[0,1]^s} f(\boldsymbol{x})\, d\boldsymbol{x}$$

is defined as

$$I_N^{\mathrm{QMC}}(f) := \frac{1}{N}\sum_{n=1}^{N} f(\boldsymbol{x}_n). \tag{3.3}$$

Remark 3.2.4.
Even though grid points as in Example 3.1.4 can achieve optimal discrepancy in 1D, they are not considered low-discrepancy sequences in the sense of the above definition, since they do not satisfy the asymptotic bound for all N. Furthermore, grids are not easily extensible — adding new points requires global recomputation

and destroys nesting.

Now that we have established the definitions of low-discrepancy sequences, we will introduce two examples of low-discrepancy sequences that are widely used in practice: the Halton sequence and the Sobol' sequence.

3.2.1 The Halton Sequence

The Halton sequence is one of the earliest and most widely used constructions of low-discrepancy sequences in arbitrary dimensions.

The construction of the Halton sequence is based on the one-dimensional *Van der Corput sequence*, introduced by Johannes van der Corput in 1935. Therefore, we utilize the radical-inverse function $\phi_b(n)$, which maps an integer n to a real number in $[0,1)$ by reflecting its base-b representation across the decimal point. This function is defined as follows:

$$\phi_b(n) = \sum_{k=0}^{\infty} d_k b^{-k-1}, \quad \text{where } n = \sum_{k=0}^{\infty} d_k b^k \quad \text{and } d_k \in \{0,1,\dots,b-1\}.$$

Definition 3.2.5 (Van der Corput Sequence)**.**
The one-dimensional Van der Corput sequence in base b is defined as

$$\mathcal{S}_b = (\phi_b(n))_{n\in\mathbb{N}}.$$

In 1960 John Halton generalized this idea to higher dimensions by combining multiple Van der Corput sequences in different bases [10].

Definition 3.2.6 (Halton Sequence)**.**
Let $b_1,\dots,b_s$ be pairwise coprime integers greater than 1, typically chosen as the first s prime numbers. The n-th point $\boldsymbol{x}_n \in [0,1)^s$ of the s-dimensional Halton sequence is defined componentwise by

$$\boldsymbol{x}_n = \Big(\phi_{b_1}(n),\dots,\phi_{b_s}(n)\Big),$$

where $\phi_b(n)$ denotes the Van der Corput radical-inverse function in base b. The Halton sequence is then given by $\mathcal{S}_{b_1,\dots,b_s} = (x_n)_{n\in\mathbb{N}}$.

Example 3.2.7 (First Elements of the Halton Sequence in 2D).
Consider the two-dimensional Halton sequence based on the first two prime bases $b_1 = 2$ and $b_2 = 3$. The first three elements are obtained by computing the radical-inverse values $\phi_{b_1}(n)$ and $\phi_{b_2}(n)$ for $n = 1,2,3$:

- $n = 1$: $\boldsymbol{x}_1 = (\phi_2(1),\phi_3(1)) = \left(\frac{1}{2},\frac{1}{3}\right)$
- $n = 2$: $\boldsymbol{x}_2 = (\phi_2(2),\phi_3(2)) = \left(\frac{1}{4},\frac{2}{3}\right)$
- $n = 3$: $\boldsymbol{x}_3 = (\phi_2(3),\phi_3(3)) = \left(\frac{3}{4},\frac{1}{9}\right)$

This illustrates how the Halton sequence fills the unit square in a low-discrepancy manner even for small n.

Intuitively, this construction ensures that each component of the sequence explores the unit interval in a structured, non-redundant way. By combining several one-dimensional Van der Corput sequences in different coprime bases, the resulting multi-dimensional point set avoids regular grid-like patterns and achieves asymptotic uniformity.

As stated in [16, Section 2.4], the Halton sequence has star discrepancy of order

$$D_N^*(\{\boldsymbol{x}_0,\dots,\boldsymbol{x}_{N-1}\}) = \mathcal{O}\left(\frac{(\log N)^s}{N}\right),$$

making it a prototypical example of a low-discrepancy sequence. However, for larger dimensions, correlation effects between the different base components can lead to degraded uniformity and higher discrepancy. To mitigate this, variants are used in practice: leaping (taking only every k-th element of the sequence) and scrambling (applying randomized permutations to the digits in the radical-inverse function). Both reduce correlation effects and improve uniformity in higher dimensions.

Remark 3.2.8.
The Halton sequence is extensible in N and s, making it suitable for applications that require growing or adaptive point sets. However, its distribution properties in higher dimensions tend to be inferior compared to constructions such as the Sobol' sequence.

3.2.2 The Sobol' Sequence

Sobol' sequences are another class of low-discrepancy sequences that are widely used in QMC methods, particularly for high-dimensional problems. Constructed to fill the s-dimensional unit cube $[0,1)^s$ as uniformly as possible, Sobol' sequences are based on a recursive structure that allows for efficient generation and high-dimensional performance.

Therefore, functions from the *ring of polynomials*

$$\mathbb{F}_2[x] = \{a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n \mid n \in \mathbb{N},\ a_i \in \mathbb{F}_2\}$$

with coefficients in the finite field $\mathbb{F}_2$ are necessary. $\mathbb{F}_2 = \{0,1\}$ denotes the finite field with two elements, equipped with addition and multiplication modulo 2. A primitive polynomial of degree m in $\mathbb{F}_2[x]$ is an irreducible polynomial whose roots generate all non-zero elements of the field extension $\mathbb{F}_{2^m}$.

Definition 3.2.9 (Sobol' Sequence Construction)**.**
Let $p_1,\dots,p_s \in \mathbb{F}_2[x]$ be primitive polynomials ordered by non-decreasing degree. Each polynomial p_j for dimension j is of the form

$$p_j(x) = x^{e_j} + a_{1,j}x^{e_j-1} + \cdots + a_{e_j-1,j}x + 1,$$

where $e_j \in \mathbb{N}$ and the coefficients $a_{k,j} \in \{0,1\}$.
Choose initial values $m_{1,j},\dots,m_{e_j,j}$ such that each $m_{k,j}$ is an odd integer and $1 \le m_{k,j} < 2^k$ for $1 \le k \le e_j$. For $k > e_j$, the values $m_{k,j}$ are computed recursively as:

$$\begin{aligned} m_{k,j} =& 2a_{1,j} \cdot m_{k-1,j} \oplus 2^2 a_{2,j} \cdot m_{k-2,j} \oplus \dots \\ & \oplus 2^{e_j-1} a_{e_j-1,j} \cdot m_{k-(e_j-1),j} \oplus 2^{e_j} m_{k-e_j,j} \oplus m_{k-e_j,j}, \end{aligned}$$

where $\oplus$ denotes bitwise exclusive-or (XOR).
The corresponding direction numbers are defined by

$$v_{k,j} = \frac{m_{k,j}}{2^k}.$$

Let $n \in \mathbb{N}_0$ with binary expansion

$$n = n_0 + 2n_1 + \cdots + 2^{r-1}n_{r-1}, \quad n_i \in \{0,1\}.$$

Then the j-th coordinate of the n-th Sobol' point is given by

$$x_{n,j} = n_0 v_{1,j} \oplus n_1 v_{2,j} \oplus \cdots \oplus n_{r-1} v_{r,j}.$$

The s-dimensional Sobol' point is

$$\boldsymbol{x}_n = (x_{n,1},\dots,x_{n,s}).$$

As stated in [16, Chapter 5], the Sobol' sequence is a low-discrepancy sequence.

Example 3.2.10 (First Elements of the Sobol' Sequence in 2D).
We fix $s = 2$ with primitive polynomials ordered by degree

$$p_1(x) = x + 1 \ (e_1 = 1,\ a_{1,1} = 1), \qquad p_2(x) = x^2 + x + 1 \ (e_2 = 2,\ a_{1,2} = a_{2,2} = 1),$$

and choose admissible initials

$$m_{1,1} = 1, \qquad m_{1,2} = 1, \quad m_{2,2} = 1.$$

Thus the first direction numbers are

$$v_{1,1} = \tfrac{1}{2}, \qquad v_{1,2} = \tfrac{1}{2}, \qquad v_{2,1} = \tfrac{3}{4}, \qquad v_{2,2} = \tfrac{1}{4}.$$

With $n = n_0 + 2n_1 + \cdots$ and $x_{n,j} = n_0 v_{1,j} \oplus n_1 v_{2,j} \oplus \cdots$, the first three points are

$$n = 0 : (n_0, n_1) = (0,0) \Rightarrow \boldsymbol{x}_0 = (0,0).$$
$$n = 1 : (1,0) \Rightarrow \boldsymbol{x}_1 = \left(\tfrac{1}{2}, \tfrac{1}{2}\right).$$
$$n = 2 : (0,1) \Rightarrow \boldsymbol{x}_2 = \left(\tfrac{3}{4}, \tfrac{1}{4}\right).$$

Remark 3.2.11.
Different valid choices of p_j and $m_{k,j}$ produce different — but equivalent — Sobol' sequences.

The bitwise structure of the Sobol' sequence makes it particularly efficient to generate and it enables streaming or online sampling in high dimensions.

3.2.3 Dimensional Performance and Sequence Comparison

The practical performance of low-discrepancy sequences is strongly influenced by the dimension s of the integration domain. Although both Halton and Sobol' sequences achieve the asymptotic star discrepancy bound of order $\mathcal{O}((\log N)^s / N)$, their behavior in higher dimensions differs significantly in practice due to the hidden constants.

Halton Sequence. While the Halton sequence performs well in low-dimensional settings, its structure leads to correlation artifacts in higher dimensions due to the use of increasing prime bases. These artifacts manifest as clustering or gaps in certain coordinate directions, which can degrade uniformity. As a result, the empirical discrepancy of the Halton sequence often grows faster than the theoretical bound suggests in moderate to high dimensions, as the hidden constant C depends on the dimension s [16, Section 2.4].

Sobol' Sequence. In contrast, the Sobol' sequence is explicitly constructed for high-dimensional performance. The use of carefully selected primitive polynomials and direction numbers minimizes inter-dimensional correlations. Furthermore, the bitwise construction allows for better numerical stability and efficient implementation. Sobol' sequences generally exhibit lower discrepancy than Halton sequences in dimensions $s > 10$, making them a standard choice in modern QMC applications.

Figure 3.3 below illustrates the difference in a two-dimensional setting between the two low-discrepancy sequences. The Sobol' sequence fills the unit square more uniformly, while the Halton sequence shows visible clustering.

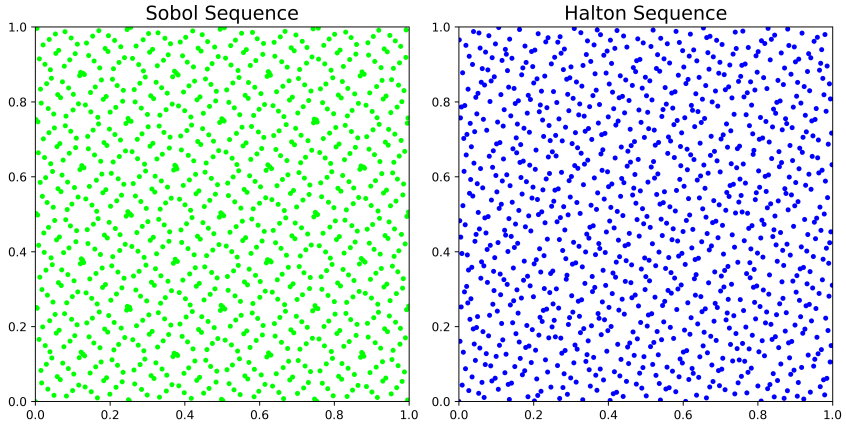


FIGURE 3.3: Comparison of Halton and Sobol' sequences in $s = 2$ dimensions for $N = 1024$ points. The Sobol' sequence fills the unit square more uniformly, demonstrating lower star discrepancy than the Halton sequence.

Remark 3.2.12.
In the experiments of Part II and Part III, the Sobol' and Halton sequences are not implemented manually. Instead, the implementation available in the Python package *SciPy* (version 1.16.0) through the module `scipy.stats.qmc` [34, 31] was used. The Sobol' and Halton sequences from this package, however, follow Definition 3.2.9 and Definition 3.2.6.

Chapter 4

Function Variation and Error Bounds in QMC

4.1 Function Variation

In Quasi-Monte Carlo (QMC) theory, the variation of a function plays a key role in bounding the integration error. Intuitively, variation quantifies how much a function oscillates or changes over its domain. This can be observed particularly well in the univariate total variation:

Definition 4.1.1 (Total Variation)**.**
Let $f : [a,b] \to \mathbb{R}$ be a real-valued univariate function. The *total variation* of f on the interval $[a,b] \subset \mathbb{R}$ is defined as

$$V_a^b(f) := \sup_{\mathcal{P}} \sum_{i=1}^{n} \left| f(x_i) - f(x_{i-1}) \right|,$$

where the supremum is taken over all partitions

$$\mathcal{P} = \{a = x_0 < x_1 < \cdots < x_n = b\},$$

of $[a,b]$.

While the total variation is sufficient for univariate functions, higher dimensions necessitate more refined notions. In this section, we follow the definitions of [25] and introduce two such notions: the Vitali variation and the variation in the sense of Hardy–Krause. These concepts are crucial for understanding the convergence guarantees provided by the Koksma–Hlawka inequality.

4.1.1 Variation in the Sense of Vitali

To express multidimensional variation concisely, we begin with the following notational conventions.

Definition 4.1.2 (Mixed Component Selection)**.**
Let $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^s$ and let $u \subseteq \{1, \ldots, s\}$ be an index set. We define

$$\boldsymbol{c} := \boldsymbol{a}^u : \boldsymbol{b}^{-u} \quad \text{with} \quad c_j := \begin{cases} a_j, & \text{if } j \in u, \\ b_j, & \text{if } j \notin u. \end{cases}$$

Here, the term $-u$ denotes the complement of u in $\{1, \ldots, s\}$. Using this notation, the *alternating sum* is defined as follows:

Definition 4.1.3 (Alternating Sum)**.**
The *alternating sum* on s dimensions with respect to a function $f : [\boldsymbol{a}, \boldsymbol{b}] \to \mathbb{R}$ on the hyperrectangle $[\boldsymbol{a}, \boldsymbol{b}]$ is defined as

$$\Delta(f; \boldsymbol{a}, \boldsymbol{b}) = \sum_{v \subseteq \{1, \ldots, s\}} (-1)^{|v|} f(\boldsymbol{a}^v : \boldsymbol{b}^{-v}).$$

A classical approach to quantify function variation on hyperrectangles is based on the concept of *ladders*:

Definition 4.1.4 (Ladder)**.**
A *ladder* on the hyperrectangle $[\boldsymbol{a}, \boldsymbol{b}]$ is of the form $\mathcal{Y} = \bigtimes_{j=1}^{s} \mathcal{Y}^j$, where $\bigtimes$ denotes the Cartesian product of sets. Each $\mathcal{Y}^j$ is a finite subset of $[a_j, b_j]$ for $j = 1, \ldots, s$ where $\mathcal{Y}^j = \{y_1^j, \ldots, y_k^j\}$ is ordered such that $a_j = y_1^j < y_2^j < \cdots < y_k^j$. The ladder may consist of a different number of points in each dimension.
The *successor* of a point $\boldsymbol{y} \in \mathcal{Y}$ in the ladder is defined componentwise as

$$y_+^j = \min\Big\{(y^j, \infty) \cap (\mathcal{Y}^j \cup \{b_j\})\Big\}.$$

with $\boldsymbol{y}_+ = (y_+^1, \ldots, y_+^s)$.

$\mathbb{Y}$ denotes the set of all such ladders on $[\boldsymbol{a}, \boldsymbol{b}]$.

Example 4.1.5. Consider the two-dimensional hyperrectangle $[\boldsymbol{a}, \boldsymbol{b}] = [0, 1]^2$ and the ladder $\mathcal{Y} = \{0, 0.5\} \times \{0, 0.25\}$. The ladder points are

$$\mathcal{Y} = \{(0, 0), (0, 0.25), (0.5, 0), (0.5, 0.25)\}.$$

The successors of these points are

$$\begin{aligned} (\ \ 0, \quad 0)_+ &= (0.5, 0.25), \\ (\ \ 0, 0.25)_+ &= (0.5, 1\ \ \ ), \\ (0.5, \quad 0)_+ &= (1\ \ \ , 0.25), \\ (0.5, 0.25)_+ &= (1\ \ \ , 1\ \ \ ). \end{aligned}$$

The visualization can be found in Figure 4.1.

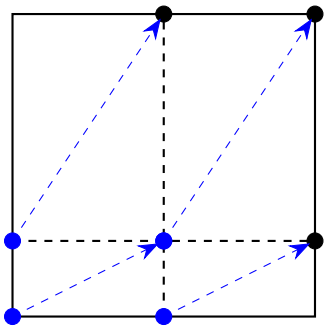


FIGURE 4.1: Visualization of the ladder $\mathcal{Y} = \{0, 0.5\} \times \{0, 0.25\}$ on the unit square $[0,1]^2$ and its successors.

Remark 4.1.6.
For a ladder $\mathcal{Y}$ on the hypercube $[\boldsymbol{a}, \boldsymbol{b}]$ the following equation holds:

$$\bigcup_{\boldsymbol{y} \in \mathcal{Y}} [\boldsymbol{y}, \boldsymbol{y}_+] = [\boldsymbol{a}, \boldsymbol{b}]$$

This is due to the ladder splitting the hypercube $[\boldsymbol{a}, \boldsymbol{b}]$ into axis-aligned rectangular subregions by the points $[\boldsymbol{y}, \boldsymbol{y}_+]$ for each $\boldsymbol{y} \in \mathcal{Y}$. Thus, the union of all such subregions is a disjoint cover of the hypercube.

The *variation with respect to a ladder* is then defined as:

Definition 4.1.7 (Variation with respect to a ladder)**.**
Let $\mathcal{Y} = \prod_{j=1}^{s} \mathcal{Y}^j$ be a ladder on $[\boldsymbol{a}, \boldsymbol{b}]$. Then the *variation* of a function f with respect to $\mathcal{Y}$ is

$$V_{\mathcal{Y}}(f) = \sum_{\boldsymbol{y} \in \mathcal{Y}} \left| \Delta(f; \boldsymbol{y}, \boldsymbol{y}_+) \right|.$$

The *Vitali variation* of a function is then the supremum of these ladder-based variations:

Definition 4.1.8 (Vitali Variation)**.**
The *Vitali variation* of a function f on the hyperrectangle $[\boldsymbol{a}, \boldsymbol{b}]$ is defined as

$$V_{[\boldsymbol{a}, \boldsymbol{b}]}(f) = \sup_{\mathcal{Y} \in \mathbb{Y}} V_{\mathcal{Y}}(f).$$

This variation captures the total fluctuation of f over axis-aligned rectangular subregions. It generalizes finite differences and serves as a foundation for defining the Hardy–Krause variation.

4.1.2 Hardy–Krause Variation

The *Hardy–Krause variation* refines the Vitali variation by summing variations over all lower-dimensional axis-aligned faces of the unit hypercube. It is defined as:

Definition 4.1.9 (Hardy–Krause Variation)**.**
The *Hardy–Krause variation* of a function f on the hyperrectangle $[\boldsymbol{a},\boldsymbol{b}]$ is defined as

$$V_{\mathrm{HK}}(f) = \sum_{u\subseteq\{1,\dots,s\}u\neq\varnothing} V_{[\boldsymbol{a}^u,\boldsymbol{b}^u]}\Big(f(x^u;\boldsymbol{b}^{-u})\Big). \tag{4.1}$$

Note that the Hardy–Krause variation reduces to the total variation in the univariate case. Importantly, $V_{\mathrm{HK}}(f) < \infty$ implies that f has bounded variation in all axis-aligned directions and projections.

Example 4.1.10.
Let $f(x) = \prod_{i=1}^{s} x_i$, the s-dimensional monomial on the hypercube $[0,1]^s$. Then it is clear that $f \in C^s$. According to equation (7.2) and Remark 7.1.3 the calculation follows as:

$$\begin{aligned} V_{\mathrm{HK}}(f) &= \widehat{V}_{\mathrm{HK}}(f) \\ &= \sum_{\substack{u\subseteq\{1,\dots,s\}\\ u\neq\varnothing}} \int_{[0,1]^{|u|}} \left|\frac{\partial^{|u|}}{\partial x^u} f(x^u;1^{-u})\right| dx^u \\ &= \sum_{\substack{u\subseteq\{1,\dots,s\}\\ u\neq\varnothing}} \int_{[0,1]^{|u|}} 1\, dx^u \\ &= \sum_{\substack{u\subseteq\{1,\dots,s\}\\ u\neq\varnothing}} 1 \\ &= 2^s - 1\,. \end{aligned}$$

These notations will also be introduced in Chapter 7.

Example 4.1.11.
Consider $f_{d,r}(x) = \left(\max\left\{\sum_{i=1}^{d} x_i - \frac{1}{2}, 0\right\}\right)^r$. For $d > r+2$, this function has infinite Vitali and hence infinite Hardy–Krause variation. Such examples underline the limitations of QMC for functions with low smoothness or sharp nonlinearities [25, Prop. 16].

In the next section, the Koksma–Hlawka inequality will be presented, which highly depends on the concept of function variation introduced in this section. These examples illustrate the importance of regularity assumptions. In practice, many physically motivated maps — such as solutions to elliptic or parabolic PDEs — do exhibit sufficient smoothness to ensure bounded Hardy–Krause variation, justifying the use of QMC error bounds.

4.2 The Koksma–Hlawka Inequality

The Koksma–Hlawka inequality is a central result in Quasi-Monte Carlo theory. It provides a deterministic error bound for numerical integration that combines two distinct concepts: the discrepancy of the point set and the variation of the integrand. This inequality formalizes the intuition that more uniformly distributed sample points lead to smaller integration errors — provided the integrand is regular enough.

The inequality relates the QMC integration error to the star discrepancy of the point set and the Hardy–Krause variation of the integrand. A proof can be found in [5].

Theorem 4.2.1 (Koksma–Hlawka Inequality)**.**
Let $f\colon [0,1]^s \to \mathbb{R}$ be a function of bounded variation in the sense of Hardy–Krause and let $P = \{x_1, \dots, x_N\} \subset [0,1]^s$ be a finite point set. Then the integration error satisfies

$$\left| \frac{1}{N} \sum_{n=1}^{N} f(x_n) - \int_{[0,1]^s} f(x)\, dx \right| \leq D_N^*(P) \cdot V_{\mathrm{HK}}(f), \tag{4.2}$$

where $D_N^*(P)$ denotes the star discrepancy of the point set P and $V_{\mathrm{HK}}(f)$ is the Hardy–Krause variation of f.

Remark 4.2.2.
This bound is deterministic and non-asymptotic. It applies to any dimension s and makes no probabilistic assumptions on the sampling procedure. However, it requires f to have bounded variation in the Hardy–Krause sense, which excludes some classes of highly irregular or discontinuous functions.

The two factors on the right-hand side of the inequality — discrepancy and variation — capture complementary aspects of integration error.

- The **star discrepancy** $D_N^*(P)$ measures how uniformly the point set P samples the unit cube. It is determined entirely by the geometry of the sample points and vanishes if the empirical distribution matches the uniform distribution.
- The **Hardy–Krause variation** $V_{\mathrm{HK}}(f)$ reflects the total directional variation of the integrand f along all coordinate axes and lower-dimensional projections. It penalizes irregularity and ensures that the integrand behaves well under uniform sampling.

Both quantities are required: even a low-discrepancy point set cannot guarantee a small error if the function has unbounded variation. Conversely, even a smooth function may incur a large error if the points cluster poorly.

Remark 4.2.3.
The Koksma–Hlawka inequality can be seen as a product-type estimate: each component controls a different source of error and their product bounds the total integration error.

The Koksma–Hlawka inequality provides valuable insight into the behavior of Quasi-Monte Carlo estimators:

- It motivates the use of low-discrepancy sequences such as Halton or Sobol', which minimize $D_N^*(P)$.
- It emphasizes the importance of function regularity — only functions with bounded Hardy–Krause variation are eligible for the bound.
- It establishes a worst-case, deterministic error guarantee, unlike the probabilistic bounds of Monte Carlo integration.

In practical terms, the Koksma–Hlawka inequality implies that for sufficiently smooth integrands with bounded Hardy–Krause variation and point sets obtained by low-discrepancy sequences, with star discrepancy of order (3.2) by definition, one can achieve integration errors fulfilling

$$\left| \frac{1}{N} \sum_{n=1}^{N} f(x_n) - \int_{[0,1]^s} f(x)\, dx \right| \leq \mathcal{O}\left(\frac{(\log N)^s}{N} \right),$$

which significantly improves over the $\mathcal{O}(N^{-1/2})$ rate of standard Monte Carlo methods. This improvement, however, comes at the cost of stronger assumptions and increased sensitivity to dimensionality.

Building upon this theoretical foundation, the next parts of this thesis turn from abstract principles to concrete applications. Part II investigates how QMC methods can enhance the training of neural networks, while Part III applies these techniques to the simulation of X-ray images in medical imaging.

Part II

Quasi-Monte Carlo Sampling for Neural Network Training

Chapter 5

Motivation and Problem Formulation

The second part of this thesis investigates the application of Quasi-Monte Carlo (QMC) sampling techniques to enhance the training of Deep Neural Networks (DNNs) in *many-query problems* — computational tasks in which a parameterized map must be evaluated for a large number of inputs, each requiring costly simulations or numerical solutions of PDEs. This situation arises frequently in scientific computing, e.g. in uncertainty quantification, Bayesian inversion, optimal control and model calibration. This part of the thesis thus addresses a current topic, specifically the article by Mishra and Rusch [21].

Such problems motivate the use of surrogate models, typically based on deep neural networks, which are trained offline on a set of inputs and then used for fast evaluation online. However, using Monte Carlo (MC) sampling for generating training data may lead to slow convergence as introduced in Part I.

To overcome this limitation, this thesis explores the use of low-discrepancy sequences to generate training data which fill the input domain more uniformly. Under mild regularity assumptions, they can significantly reduce the generalization error due to improved equidistribution properties.

5.1 Many-Query Problems in Scientific Computing

Many-query problems refer to scenarios in which a computationally expensive model $f(x)$ must be evaluated for many different parameter values $x \in \mathcal{X} \subset \mathbb{R}^s$. Typical applications include:

- Uncertainty quantification (UQ)
- Optimal design/control
- Inverse problems

The sheer cost of evaluating $f(x)$ (e.g., via numerical PDE solvers) motivates the use of surrogate models. Once trained, a surrogate $\hat{f}$ can provide rapid predictions for unseen inputs at a fraction of the computational cost.

5.2 The Role of Function Approximation

We formalize the problem as learning a map $f\colon [0,1]^s \to \mathbb{R}^m$ from data. The assumption of a unit hypercube domain reflects common practice in QMC theory and is justified via homeomorphic mappings from more general compact and simply connected input spaces.

In this thesis, the surrogate model $\hat{f}$ will be chosen from the class of deep neural networks and is trained on data $\{(x_i, f(x_i))\}_{i=1}^N$. The aim is to approximate f with high accuracy while minimizing the number of required evaluations.

Deep Neural Networks are known to be universal approximators. Under mild conditions on the *activation functions* introduced later, they have been proven to approximate any continuous function on compact sets arbitrarily well [7].

5.3 Applicability of the Koksma–Hlawka Inequality to DNN Training

Generally, training data is generated using Monte Carlo sampling, where points x_i are drawn independently and identically distributed (i.i.d.) according to a probability measure μ on $[0,1]^s$. Under this approach, statistical learning theory provides an estimate for the expected generalization error $\mathbb{E}[\mathcal{E}_G]$ as follows:

$$\mathcal{E}_G \sim \mathcal{E}_T + \mathcal{O}\left(\frac{1}{\sqrt{N}}\right),$$

where $\mathcal{E}_T$ is the empirical training error. Both the generalization and training error will be formally introduced in Section 6.2. However, as argued in [21], this rate is unsatisfactory in many-query problems because:

- A small generalization error requires a large N, which is costly due to expensive evaluations of $f(x)$.
- The constant in the bound depends on the variance and correlations in the training process, which are hard to control.

A promising alternative to random sampling is to use Quasi-Monte Carlo (QMC) sampling. Low-discrepancy sequences, such as the introduced Sobol' or Halton sequence, provide a way to sample the domain $[0,1]^s$ more uniformly than random sampling. In Quasi-Monte Carlo theory, the Koksma–Hlawka inequality (Theorem 4.2.1) applied to the difference $f - \hat{f}$ gives an error bound of the form

$$|\mathcal{E}_G - \mathcal{E}_T| \leq V_{\mathrm{HK}}(|f - \hat{f}|) \cdot D_N^* \leq V_{\mathrm{HK}}(|f - \hat{f}|) \cdot C_s \cdot \frac{(\log N)^s}{N}.$$

The left part of the inequality will be formally derived in Theorem 7.2.1. The right part follows from the known discrepancy for low-discrepancy sequences. It is not immediately clear whether the Koksma–Hlawka inequality can be applied to the generalization gap $|\mathcal{E}_G - \mathcal{E}_T|$ in the context of training DNNs. This is because the Koksma–Hlawka inequality requires the integrand to have bounded variation in the sense of Hardy–Krause to ensure a finite bound.

In the following chapters, it will be shown that under mild assumptions on the surrogate model $\hat{f}$ and that the ground truth function f has bounded variation in the

sense of Hardy–Krause, the Koksma–Hlawka inequality is applicable to the generalization gap. In particular, it will be demonstrated that using low-discrepancy sequences for generating training data can significantly reduce the generalization gap.

Since the direct application of the Koksma–Hlawka inequality, in particular the determination of the Hardy–Krause variation, is not practical for general formulations of the approximation function $\hat{f}$ depending on the training data, the usefulness and applicability of the approach will be verified in Chapter 8 through selected examples.

Chapter 6

Theoretical Foundations of Deep Neural Networks

Deep Neural Networks (DNNs) have become a powerful class of surrogate models in scientific computing, particularly for high-dimensional and expensive-to-evaluate functions. This chapter introduces the mathematical structure and training procedure of DNNs, with a focus on activation functions, generalization theory and gradient-based optimization algorithms. It follows the framework established in [21].

6.1 Theoretical Foundations of Deep Neural Networks

The key feature enabling neural networks to approximate nonlinear functions is the application of scalar activation functions. We begin by introducing these components before defining the class of Deep Neural Network functions.

Definition 6.1.1 (Activation Function)**.**
An *activation function* is a measurable function $\sigma : \mathbb{R} \to \mathbb{R}$, typically nonlinear, that is applied componentwise to the output of each layer in a neural network.

Remark 6.1.2.
In general, the activation function σ is 1-dimensional, i.e. $\sigma : \mathbb{R} \to \mathbb{R}$. However, it is common to apply it componentwise to vectors and extend the definition to $\sigma : \mathbb{R}^k \to \mathbb{R}^k$ with $\sigma(z) = (\sigma(z_1), \ldots, \sigma(z_k))^\top$.

Example 6.1.3 (Common Activation Functions).
Two widely used activation functions are given below and illustrated in Figure 6.1. These are also the activation functions we employ in the experimental evaluations presented in Chapter 8.

- **Tanh**:
$$\sigma(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$
The hyperbolic tangent function maps real numbers to $(-1, 1)$ and is differentiable everywhere. It is often used in hidden layers of neural networks.

- **Sigmoid**:
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid function maps real numbers to $(0,1)$ and is differentiable everywhere. It is often used in theoretical analysis and binary classification tasks.

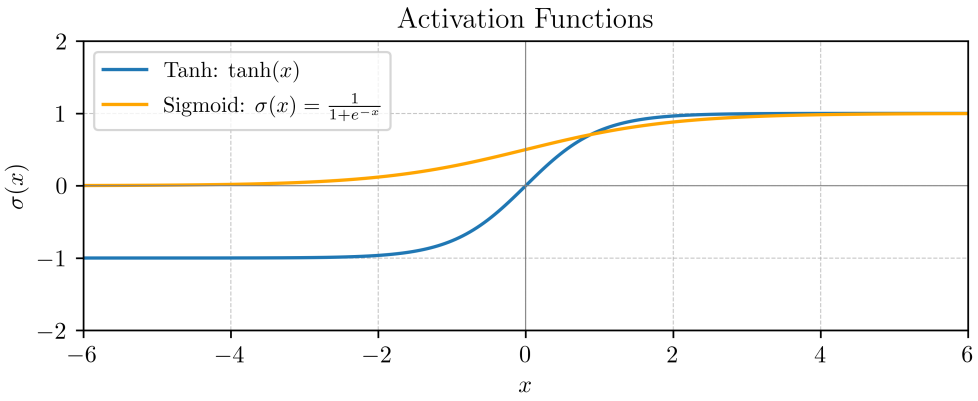


FIGURE 6.1: Comparison of ReLU and Sigmoid activation functions.

With an activation function in place, we now define the term Deep Neural Network by introducing the representing function that a fully connected feedforward neural network expresses [21].

Definition 6.1.4 (Fully Connected Deep Neural Network)**.**
Let $L \in \mathbb{N}$ and $\mathbf{d} = (d_0, d_1, \ldots, d_L)$ be a sequence of positive integers, where $d_0 = s$ is the input dimension and $d_L = 1$ the output dimension. A Deep Neural Network (DNN) of *depth* L and *layer widths* $d_1, \ldots, d_{L-1}$ is a function $\hat{f}_\theta : \mathbb{R}^s \to \mathbb{R}$ of the form

$$\hat{f}_\theta(y) = A_L \circ \sigma \circ A_{L-1} \circ \cdots \circ \sigma \circ A_1(y), \tag{6.1}$$

where each *layer map* $A_l : \mathbb{R}^{d_{l-1}} \to \mathbb{R}^{d_l}$ is an affine transformation

$$A_l(z) = W_l z + b_l,$$

with weights $W_l \in \mathbb{R}^{d_l \times d_{l-1}}$ and biases $b_l \in \mathbb{R}^{d_l}$. The activation function $\sigma : \mathbb{R} \to \mathbb{R}$ is applied componentwise.
The collection of all trainable parameters is denoted by

$$\theta = \{(W_l, b_l)\}_{l=1}^{L},$$

and fully determines the function $\hat{f}_\theta$ for a fixed activation function σ. The set of all such realizable functions for a given architecture $\mathbf{d}$ and activation function σ is denoted by $\mathcal{F}_{\mathbf{d},\sigma}$.

The architectural parameters are interpreted as follows:

- The *depth* L is the number of affine transformations.
- The number of *hidden layers* is $L-1$.
- The *width* d_l specifies the number of neurons in layer l.
- The *layer structure* is encoded in the tuple $\mathbf{d}$.
- The *activation function* σ usually introduces nonlinearity.

As is common in literature, the *number of layers* is referenced by the number of affine transformations L and coincides with the term *depth*. The input layer is not counted as a layer in this context. In Figure 6.2, a fully connected DNN with constant width is illustrated.

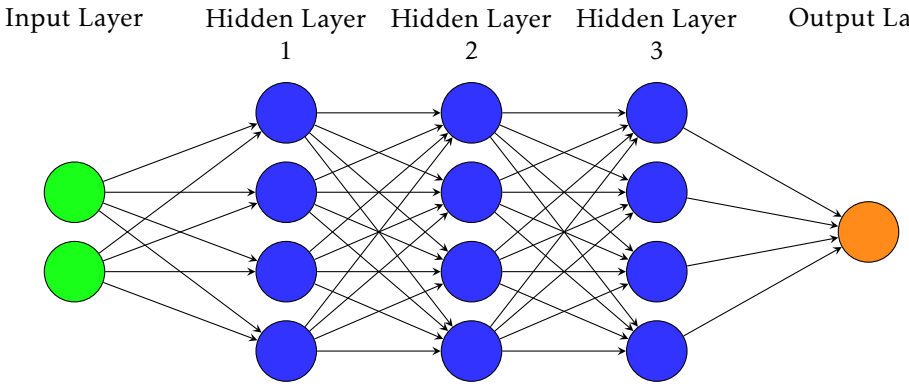


FIGURE 6.2: Schematic of a fully connected Deep Neural Network with constant width $m = 4$ and depth $L = 4$ layers. Each circular node represents an artificial neuron. The arrows represent the connections between neurons in adjacent layers.

Remark 6.1.5.
For common DNN architectures with L layers, a constant number of neurons m in each layer, input dimension $d_0 = s$ and output dimension $d_L = 1$ the layer structure follows:

$$d = (d_0, d_1, ..., d_{L-1}, d_L) = (s, m, \dots, m, 1)$$

Thus, for each affine transformation $l \in \{1, \dots, L\}$, one counts $d_{l-1} \cdot d_l$ weights and d_l biases. Hence, the total number of trainable parameters (weights and biases) θ in the network is given by

$$\begin{aligned} N_\theta &= \sum_{l=1}^{L} (d_{l-1} \cdot d_l + d_l) \\ &= (s \cdot m + m) + \sum_{l=2}^{L-1} (m \cdot m + m) + (m \cdot 1 + 1) \\ &= (L-2) \cdot m^2 + (L+s) \cdot m + 1. \end{aligned}$$

6.2 DNN Evaluation

The training of a DNN involves supervised learning, where the goal is to minimize the difference between the surrogate model $\hat{f}$ and the ground truth function f. Supervised training of a DNN therefore requires minimizing the difference between the network output $\hat{f}_\theta(y)$ and a ground truth function $f(y)$ on a given training set $\{y_i\}_{i=1}^N$.

We start by defining the loss function which is used to measure this difference during the training:

Definition 6.2.1 (Loss Function)**.**
Let $f : [0,1]^s \to \mathbb{R}$ be a ground truth function defined on the hypercube and $\hat{f}_\theta$ a respective surrogate with network architecture $\mathbf{d}$, according parameters θ and activation function σ. Then, given the surrogate and a respective point set $\mathcal{P}$, the *loss function* is defined as

$$\mathcal{L}_\mathcal{P}(\theta) := \frac{1}{|\mathcal{P}|} \sum_{y \in \mathcal{P}} |f(y) - \hat{f}_\theta(y)|^p \,, \tag{6.2}$$

for some $1 \leq p < \infty$.

The point set $\mathcal{P}$ in the loss function is usually given by the training set. By defining the loss on the training data, the *training error* $\mathcal{E}_T$ is defined:

Definition 6.2.2 (Training Error)**.**
Let $f : [0,1]^s \to \mathbb{R}$ be a ground truth function defined on the hypercube and $\hat{f}_\theta$ a respective surrogate model with architecture $\mathbf{d}$, according parameters θ and activation function σ. Then, given the matching training set $\{(y_i, f(y_i))\}_{i=1}^N$ with $y_i \in [0,1]^s$ of the surrogate model, the *training error* is defined as

$$\mathcal{E}_\mathrm{T}(\theta) := \frac{1}{N} \sum_{i=1}^{N} |f(y_i) - \hat{f}_\theta(y_i)|.$$

The *generalization error* $\mathcal{E}_\mathrm{G}$ is evaluating the difference between the surrogate model $\hat{f}_\theta$ and the ground truth function f on the entire domain $[0,1]^s$.

Definition 6.2.3 (Generalization Error)**.**
Let $f : [0,1]^s \to \mathbb{R}$ be a ground truth function defined on the hypercube and $\hat{f}_\theta$ a respective surrogate with network architecture $\mathbf{d}$, according parameters θ and activation function σ. Then, given the surrogate, the *generalization error* is defined as:

$$\mathcal{E}_\mathrm{G}(\theta) := \int_{[0,1]^s} |f(y) - \hat{f}_\theta(y)| \, \mathrm{d}y.$$

Remark 6.2.4.
Whereas the training error $\mathcal{E}_\mathrm{T}$ is numerically straightforward to compute, the generalization error $\mathcal{E}_\mathrm{G}$ is often not directly computable, as the ground truth function f could not be known besides the training data or is costly to compute.

Definition 6.2.5 (Generalization Gap)**.**
The *generalization gap* for a given DNN architecture $\mathbf{d}$, according parameters θ and activation function σ is the difference between the generalization and training error:

$$|\mathcal{E}_\mathrm{G}(\theta) - \mathcal{E}_\mathrm{T}(\theta)|.$$

6.3 Optimization and Training Procedure

In practice, an initial surrogate DNN $\hat{f}_\theta$ is generated with initial parameters θ. In supervised learning, the goal is to minimize the *regularized loss function* with respect to the parameters θ by adapting the weights of the network during the training process. Regularization is introduced to counteract *overfitting*, i.e. the tendency of a highly flexible network to reproduce noise of the training data instead of the underlying function. The regularization term penalizes certain properties of the parameters, thereby enforcing additional structure on the solution. For instance, an L^2 penalty discourages large weight magnitudes and effectively promotes smoother functions.

Thus, one aims to find a solution to the following minimization problem:

$$\theta^* = \arg\min_\theta \mathcal{J}_\mathcal{P}(\theta), \quad \text{with } \mathcal{J}(\theta) = \mathcal{L}_\mathcal{P}(\theta) + \lambda \cdot R(\theta),$$

where $\mathcal{P}$ is the training set, $\mathcal{L}_\mathcal{P}$ is the empirical loss (e.g. mean squared error), $R(\theta)$ is a regularization functional and $\lambda > 0$ is the regularization parameter balancing fidelity to the training data and the strength of the imposed constraint.

The training is carried out on a given *training dataset* consisting of N input–output pairs $(x_n, f(x_n))_{n=1}^N$, where N is called the *training set size*. Since the dataset is usually too large to be processed at once, it is divided into smaller subsets of fixed cardinality B, called *mini-batches*. The number B is referred to as the *batch size* and determines how many samples are processed simultaneously in each training step.

The training procedure is organized into multiple passes over the dataset, called *epochs*. At the beginning of each epoch $e \in 1, \dots, E$, the dataset is typically shuffled and then partitioned into mini-batches

$$(\mathcal{B}_k^e)_{k=1}^K, \quad \text{with } K = \left\lceil \tfrac{N}{B} \right\rceil,$$

such that the union of all mini-batches corresponds to the full training set. This ensures that in each epoch, the entire dataset is used for training.

For each mini-batch $\mathcal{B}_k^e$, the algorithm executes the following steps: the current surrogate network $\hat{f}_\theta$ is applied to all samples in $\mathcal{B}_k^e$ in a forward pass and the corresponding *regularized loss function* $\mathcal{J}_{\mathcal{B}_k^e}$ is evaluated with respect to these samples. The gradient of the loss with respect to the parameters, $\nabla_\theta \mathcal{J}_{\mathcal{B}_k^e}$, is then computed by backpropagation. Finally, the parameters θ are updated according to the chosen optimization algorithm Opt, e.g. SGD, Adam or Lion as introduced in the followup subsections.

This procedure is repeated over all mini-batches of all epochs or until a stopping criterion is met, resulting in the final trained network $\hat{f}_{\theta^*} = \hat{f}_{\theta_E}$.

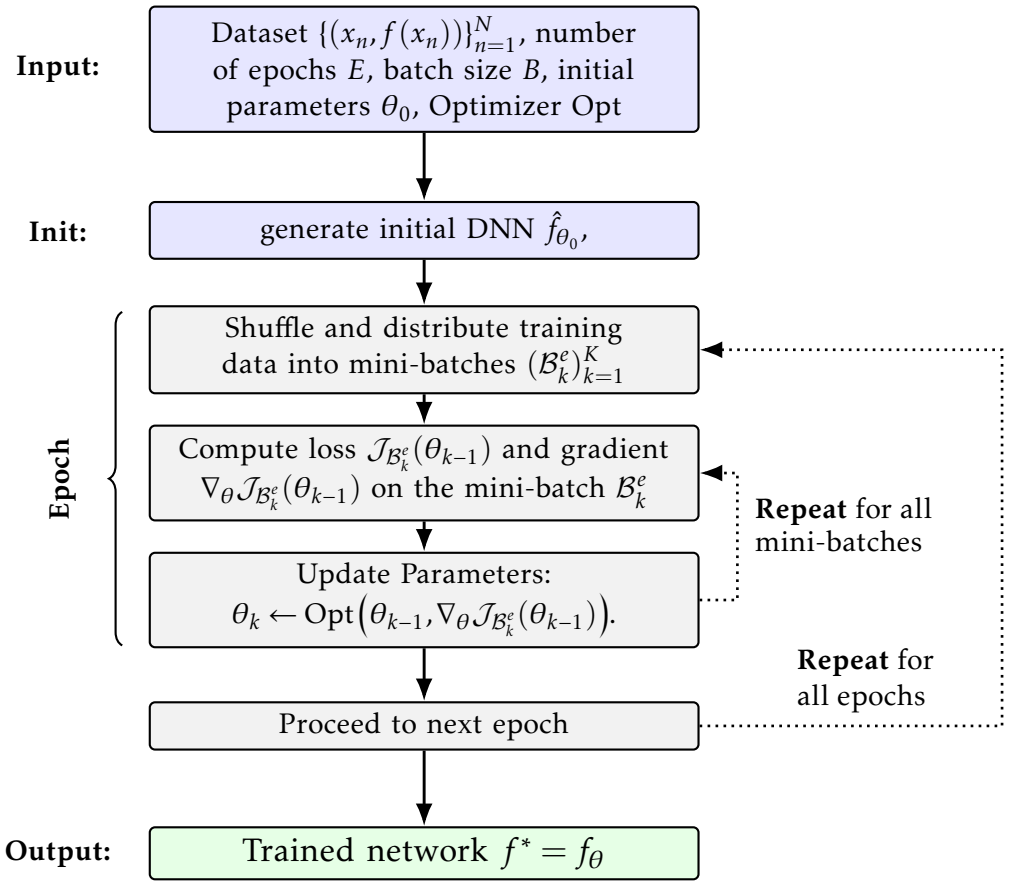


FIGURE 6.3: Generic DNN training: for each of E epochs, iterate over mini-batches of size B, compute loss/gradients and update parameters using optimizer Opt.

Training deep neural networks requires the minimization of a loss function $\mathcal{J}(\theta)$ with respect to the network parameters θ. As computing the exact minimum is intractable in practice, iterative first-order methods are typically employed. In this section, two widely used optimization algorithms are briefly introduced: Stochastic Gradient Descent and Adam. Since the primary focus of this thesis lies on sampling strategies rather than on optimization methods, the discussion remains concise; for a more detailed treatment, see [9, Chapter 8].

6.3.1 Stochastic Gradient Descent (SGD)

The most basic algorithm for training neural networks is Gradient Descent. As defined above, given the entire dataset $\mathcal{P}$, the Gradient Descent update is given by

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta \mathcal{J}_{\mathcal{P}}(\theta_k),$$

where $\eta > 0$ is the learning rate.

Computing this full gradient at every iteration is costly for large datasets. The idea of *Stochastic Gradient Descent* is to approximate it by using a randomly chosen mini-batch $\mathcal{B}_k \subset \mathcal{P}$ at each iteration k:

$$g_k = \nabla_\theta \mathcal{J}_{\mathcal{B}_k}(\theta_{k-1}).$$

The parameter update is then given by

$$\theta_k = \theta_{k-1} - \eta g_k,$$

The crucial point is that g_k is an *exact gradient* on the mini-batch, but only an *unbiased stochastic estimator* of the full gradient. This randomness in the update direction explains why the method is called *stochastic gradient descent*. The gradient itself is computed efficiently by backpropagation. Further details on backpropagation can be found in [9, Section 6.5].

A widely used extension of Stochastic Gradient Descent (SGD) is *momentum*. Instead of following the raw stochastic gradient at each step, the algorithm maintains a moving average of past gradients, called the second momentum

$$v_k = \beta_2 v_{k-1} + (1-\beta_2) g_k^2,$$

with $\beta_2 \in [0,1)$. The updated gradient step is then scaled by this second momentum:

$$\theta_k = \theta_{k-1} - \frac{\eta}{v_k} g_k \,.$$

Intuitively, momentum accelerates progress along consistent descent directions and damps oscillations in directions of high curvature, much like inertia in physical systems.

6.3.2 Adam Optimizer

The Adam optimizer [9, Section 8.5] extends the use of momentum. At each iteration, Adam maintains exponential moving averages of both the gradient (first moment) and its elementwise square (second moment):

$$m_k = \beta_1 m_{k-1} + (1-\beta_1) g_k,$$
$$v_k = \beta_2 v_{k-1} + (1-\beta_2) g_k^2,$$

where $\beta_1, \beta_2 \in [0,1)$ are decay parameters.

The algorithm also includes bias-correction terms to account for the initialization, as given in Algorithm 1.

Algorithm 1 Adam Optimizer

Require: initial parameters θ_0; learning rate $\eta > 0$; decay rates $\beta_1, \beta_2 \in [0,1)$; small $\varepsilon > 0$

$m_0 \leftarrow 0$
$v_0 \leftarrow 0$
for $k = 1, 2, \ldots$ **do**
  sample mini-batch $\mathcal{B}_k \subset \mathcal{P}$
  $g_k \leftarrow \nabla_\theta \mathcal{J}_{\mathcal{B}_k}(\theta_{k-1})$
  $m_k \leftarrow \beta_1 m_{k-1} + (1-\beta_1) g_k$ // first momentum
  $v_k \leftarrow \beta_2 v_{k-1} + (1-\beta_2) g_k^2$ // second momentum
  $\hat{m}_k \leftarrow \dfrac{m_k}{1-\beta_1^k}$ // bias correction
  $\hat{v}_k \leftarrow \dfrac{v_k}{1-\beta_2^k}$ // bias correction
  $\theta_k \leftarrow \theta_{k-1} - \eta \, \dfrac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon}$
end for

Overall, Adam combines the stabilizing effect of momentum with per-parameter adaptive learning rates, which makes it one of the most robust and widely used optimizers in modern deep learning applications.

Chapter 7

QMC-Based Deep Learning Algorithm

The goal of this chapter is to mathematically analyze the function approximation problem and find bounds on the generalization gap of DNN surrogate models obtained by low-discrepancy training data. The analysis is based on the Hardy–Krause variation and the Koksma-Hlawka inequality introduced in Chapter 4.

7.1 Preliminaries

For the sake of completeness, we start with some results on approximations of variations defined in Section 4.1. First, we recall an important identity for the alternating sum.

The following results come from [25, Section 9]. For completeness, the conceptual proofs are given here in full.

Proposition 7.1.1.
Let f be a function defined on the hypercube $[a,b] \subset \mathbb{R}^s$ with existing and integrable derivative $\frac{\partial^s}{\partial x_1 \cdots \partial x_s} f$. Then, the following holds for the alternating sum:

$$\int_{[a,b]} \frac{\partial^s}{\partial x_1 \cdots \partial x_s} f(x)\, dx = \Delta(f;a,b) \tag{7.1}$$

where $\Delta(f;a,b)$ is the alternating sum from Definition 4.1.3.

Proof.
The proof follows by induction on the dimension s.

Base case ($s = 1$).

$$\int_a^b \frac{\partial f(x)}{\partial x}\, dx = f(b) - f(a) = \Delta(f;a,b).$$

Thus, the statement holds for $s = 1$.

Induction hypothesis.
Assume the statement holds for dimension $s-1$.

Induction step ($s-1 \to s$).
Write $x = (x', x_s)$ with $x' \in [a', b'] := \prod_{j=1}^{s-1} [a_j, b_j]$. First, apply the one-dimensional fundamental theorem of calculus in the last coordinate:

$$\int_{a_s}^{b_s} \frac{\partial^s f(x', t)}{\partial x_1 \cdots \partial x_s} \, dt = \frac{\partial^{s-1} f(x', b_s)}{\partial x_1 \cdots \partial x_{s-1}} - \frac{\partial^{s-1} f(x', a_s)}{\partial x_1 \cdots \partial x_{s-1}}.$$

Now integrate over x' and use Fubini's theorem:

$$\int_{[a,b]} \frac{\partial^s f(x)}{\partial x_1 \cdots \partial x_s} \, dx = \int_{[a',b']} \frac{\partial^{s-1} f(x', b_s)}{\partial x_1 \cdots \partial x_{s-1}} \, dx' - \int_{[a',b']} \frac{\partial^{s-1} f(x', a_s)}{\partial x_1 \cdots \partial x_{s-1}} \, dx'.$$

We now apply the induction hypothesis to both integrals (for the functions $g_1(x') := f(x', b_s)$ and $g_2(x') := f(x', a_s)$):

$$\int_{[a',b']} \frac{\partial^{s-1} f(x', b_s)}{\partial x_1 \cdots \partial x_{s-1}} \, dx' = \Delta\Big(f(\cdot, b_s); a', b'\Big),$$

$$\int_{[a',b']} \frac{\partial^{s-1} f(x', a_s)}{\partial x_1 \cdots \partial x_{s-1}} \, dx' = \Delta\Big(f(\cdot, a_s); a', b'\Big).$$

Thus,

$$\int_{[a,b]} \frac{\partial^s f(x)}{\partial x_1 \cdots \partial x_s} \, dx = \Delta\Big(f(\cdot, b_s); a', b'\Big) - \Delta\Big(f(\cdot, a_s); a', b'\Big).$$

This is exactly the recursive definition of the corner difference in s dimensions. Hence, the statement holds for s. □

This yields another result for the variation in the sense of Vitali on functions with existing and integrable derivative.

Corollary 7.1.2.
Let f be a function defined on the hypercube $[a, b] \subset \mathbb{R}^s$ with existing and integrable derivative $\frac{\partial^s}{\partial x_1 \cdots \partial x_s} f$. Then, the following holds for the variation in the sense of Vitali:

$$V_{[a,b]}(f) \leq \int_{[a,b]} \left| \frac{\partial^s}{\partial x_1 \cdots \partial x_s} f(x) \right| dx.$$

Proof.
Recalling the definition of variation with respect to a ladder from Definition 4.1.7 and applying Proposition 7.1.1 yields

$$V_{\mathcal{Y}}(f) = \sum_{y \in \mathcal{Y}} \big|\Delta(f; y, y_+)\big| = \sum_{y \in \mathcal{Y}} \left| \int_{[y, y_+]} \frac{\partial^s}{\partial x_1 \cdots \partial x_s} f(x) \, dx \right|.$$

Applying the triangle inequality then leads to

$$\sum_{y\in\mathcal{Y}}\left|\int_{[y,y_+]}\frac{\partial^s}{\partial x_1\cdots\partial x_s}f(x)\,dx\right|\leq\sum_{y\in\mathcal{Y}}\int_{[y,y_+]}\left|\frac{\partial^s}{\partial x_1\cdots\partial x_s}f(x)\right|dx$$

Since for a ladder $\mathcal{Y}$ of a hyperrectangle $[a,b]$ the equality $\bigcup_{y\in\mathcal{Y}}[y,y_+]=[a,b]$ holds per definition, the sum builds the integral over the whole hyperrectangle $[a,b]$:

$$\sum_{y\in\mathcal{Y}}\int_{[y,y_+]}\left|\frac{\partial^s}{\partial x_1\cdots\partial x_s}f(x)\right|dx=\int_{[a,b]}\left|\frac{\partial^s}{\partial x_1\cdots\partial x_s}f(x)\right|dx.$$

Therefore

$$V_{[a,b]}(f)=\sup_{\mathcal{Y}\in\mathbb{Y}}V_{\mathcal{Y}}(f)\leq\int_{[a,b]}\left|\frac{\partial^s}{\partial x_1\cdots\partial x_s}f(x)\right|dx.$$

□

Remark 7.1.3.
As computing the Hardy–Krause variation is not tractable, the following upper bound follows immediately from the preceding results by Corollary 7.1.2 and Definition 4.1.9 of the Hardy–Krause variation:

$$V_{\mathrm{HK}}(f)\leq\widehat{V}_{\mathrm{HK}}(f):=\sum_{\substack{u\subseteq\{1,\dots,s\}\\u\neq\emptyset}}\int_{[a^u,b^u]}\left|\frac{\partial^{|u|}}{\prod_{j\in u}\partial x_j}f(x^u;b^{-u})\right|dx^u\,. \tag{7.2}$$

Assuming $\widehat{V}_{\mathrm{HK}}(f)<\infty$ implies that all derivatives in the sum $\frac{\partial^{|u|}}{\partial x^u}f(x^u;b^{-u})$ for all $\emptyset\neq u\subseteq\{1,\dots,s\}$ exist.

If all derivatives in the sum exist and are continuous, then by [25, Section 9] it follows that:

$$V_{\mathrm{HK}}(f)=\widehat{V}_{\mathrm{HK}}(f)\,.$$

7.2 Error estimation of DNN surrogates obtained by low-discrepancy training data

Given the computable training error $\mathcal{E}_T$ depending on the training data, we now seek to estimate the generalization error $\mathcal{E}_G$. To get there, we will use an estimate of the generalization gap and follow [21].

Theorem 7.2.1.
Assume that $\hat{f}$ is a surrogate model obtained by DNN training of a mapping f on the hypercube $[0,1]^s$ and $\mathcal{S} = \{x_n\}_{n=1}^N \subset [0,1]^s$ is a low-discrepancy sequence. Further, let $V_{\mathrm{HK}}(|f - \hat{f}|) < \infty$. Then, the following estimate on the generalization gap holds:

$$|\mathcal{E}_G - \mathcal{E}_T| \leq C_s \cdot \frac{V_{\mathrm{HK}}(|f - \hat{f}|)(\log N)^s}{N},$$

where C_s is a constant and $V_{\mathrm{HK}}(|f - \hat{f}|)$ is the Hardy–Krause variation from Definition 4.1.9.

Proof.
This simply follows from the Koksma-Hlawka inequality (Theorem 4.2.1) applied to the difference $f - \hat{f}$:

$$\begin{aligned}
|\mathcal{E}_G - \mathcal{E}_T| &= \left| \int_{[0,1]^s} f(x) - \hat{f}(x)\, dx - \frac{1}{N} \sum_{n=1}^{N} \left[f(x_n) - \hat{f}(x_n) \right] \right| \\
&\leq V_{\mathrm{HK}}(|f - \hat{f}|) \cdot D_N^* \\
&\leq C_s \cdot \frac{V_{\mathrm{HK}}(|f - \hat{f}|)(\log N)^s}{N}
\end{aligned}$$

Here, D_N^* is the star-discrepancy of the low-discrepancy sequence $\mathcal{S}$ and the constant C_s, associated with the discrepancy of the sequence $\mathcal{S}$, is dependent on the dimension s. □

Theorem 7.2.1 states that the generalization gap can be bounded by the Hardy–Krause variation of the difference $|f - \hat{f}|$ and the discrepancy of the training data. However, it is still not clear under what conditions the Hardy–Krause variation of the difference $|f - \hat{f}|$ is bounded. The following theorem states sufficient conditions for the boundedness of the Hardy–Krause variation of the difference $|f - \hat{f}|$.

For this, we need the following definition of L^q spaces.

Definition 7.2.2.
For a measurable set $\Omega \subseteq \mathbb{R}^n$ and $1 \leq q < \infty$, the space

$$L^q(\Omega) = \left\{ f : \Omega \to \mathbb{R} \,\middle|\, \int_\Omega |f(x)|^q\, dx < \infty \right\}$$

is the set of measurable functions whose q-th power is integrable. For $q = \infty$,

$$L^\infty(\Omega) = \left\{ f : \Omega \to \mathbb{R} \,\middle|\, \sup_{x \in \Omega} |f(x)| < \infty \right\}.$$

Now, we can state the main theorem of this chapter.

Theorem 7.2.3.
Let $f : [0,1]^s \to \mathbb{R}$ be a map with $\widehat{V}_{HK}(f) < \infty$ and $\frac{\partial^{|v|}}{\prod_{j\in v}\partial x_j} f(x^v; 1^{-v}) \in L^1([0,1]^{|v|})$ for all $\emptyset \neq v \subseteq \{1,\dots,s\}$. Further, let $\hat{f}$ be the DNN surrogate with an activation function $\sigma \in C^s$ trained on a low-discrepancy sequence $\mathcal{S} = \{x_n\}_{n=1}^N \subset [0,1]^s$. Then for a given tolerance $\delta > 0$ the following holds:

$$\mathcal{E}_G \leq \mathcal{E}_T + C \cdot \frac{(\log N)^s}{N} + 2\delta.$$

The constant C depends on the training algorithm and the tolerance δ associated with the mollifier introduced in the proof.

To prove this theorem, we will use the following proposition stating a multivariate version of the Faà di Bruno formula from [11].

Proposition 7.2.4 (Multivariate Faà di Bruno formula)**.**
Let $h : \mathbb{R}^s \to \mathbb{R}$ with existing mixed first order derivatives $\frac{\partial^{|B|}}{\prod_{j\in B}\partial x_j} h(x)$ for all $\emptyset \neq B \subseteq \{1,\dots,s\}$ and $g \in C^s(\mathbb{R})$. Then the following derivative of the composition $g \circ h$ follows:

$$\frac{\partial^s}{\partial x_1 \cdots \partial x_s}(g \circ h)(x) = \sum_{\pi} \left(g^{(|\pi|)} \circ h\right)(x) \cdot \prod_{B\in\pi} \frac{\partial^{|B|}}{\prod_{j\in B}\partial x_j} h(x)$$

Here, the sum is taken over all partitions π of the set $\{1,\dots,s\}$ and B are the sets within the partition π.

A proof of the multivariate Faà di Bruno formula can be found in [11, Proposition 1].

The multivariate Faà di Bruno formula will be used to prove the following result on the conditions for boundedness of the Hardy–Krause variation of compositions of functions.

Corollary 7.2.5.
Let $h : [a,b] \to \mathcal{X}$ for a hyperrectangle $[a,b] \in \mathbb{R}^s$ and $\mathcal{X} \subset \mathbb{R}$. Further, assume that $\widehat{V}_{\mathrm{HK}}(h) < \infty$ and $\frac{\partial^{|v|}}{\prod_{j\in v}\partial x_j} h(x^v; b^{-v}) \in L^1([a,b]^{|v|})$ for all $\emptyset \neq v \subseteq \{1,\dots,s\}$ is continuous. Then, the Hardy–Krause variation of the composition $g \circ h$ is bounded

$$V_{\mathrm{HK}}(g \circ h) < \infty$$

for all $g \in C^s(\mathbb{R})$.

Proof.
To check the boundedness of the Hardy–Krause variation of a composition of two

functions, first Corollary 7.1.2 is applied on the Hardy–Krause variation of the composition $g \circ h$:

$$\begin{aligned} V_{\mathrm{HK}}(g \circ h) &= \sum_{\substack{v \subseteq \{1,\dots,s\} \\ v \neq \emptyset}} V_{[a^v, b^v]}(g \circ h)(x^v; b^{-v}) \\ &\leq \sum_{\substack{v \subseteq \{1,\dots,s\} \\ v \neq \emptyset}} \int_{[a^v, b^v]} \left| \frac{\partial^{|v|}}{\prod_{j \in v} \partial x_j} (g \circ h)(x^v; b^{-v}) \right| dx^v . \end{aligned}$$

As the dimension s is finite, the sum in the definition of the Hardy–Krause variation has finitely many summands and therefore it suffices to show for each summand that it is bounded.

Applying the multivariate Faà di Bruno formula (Proposition 7.2.4) to each summand yields

$$\begin{aligned} &\int_{[a^v, b^v]} \left| \frac{\partial^{|v|}}{\prod_{j \in v} \partial x_j} (g \circ h)(x^v; b^{-v}) \right| dx^v \\ &= \int_{[a^v, b^v]} \left| \sum_{\pi} \left(g^{(|\pi|)} \circ h \right)(x^v; b^{-v}) \cdot \prod_{B \in \pi} \frac{\partial^{|B|}}{\prod_{j \in B} \partial x_j} h(x^v; b^{-v}) \right| dx^v \end{aligned}$$

with the sum taken over all partitions π of the subset v. Now applying the triangle inequality gives

$$\leq \sum_{\pi} \int_{[a^v, b^v]} \left| \left(g^{(|\pi|)} \circ h \right)(x^v; b^{-v}) \cdot \prod_{B \in \pi} \frac{\partial^{|B|}}{\prod_{j \in B} \partial x_j} h(x^v; b^{-v}) \right| dx^v .$$

Again, the sum has finitely many summands as there are finitely many partitions π of the finite set $v \subseteq \{1, \dots, s\}$. Thus, to show that the Hardy–Krause variation of the composition $g \circ h$ is bounded, it remains to show that each summand

$$\int_{[a^v, b^v]} \left| \underbrace{\left(g^{(|\pi|)} \circ h \right)(x^v; b^{-v})}_{(*)} \cdot \prod_{B \in \pi} \frac{\partial^{|B|}}{\prod_{j \in B} \partial x_j} h(x^v; b^{-v}) \right| dx^v$$

is bounded.

From the boundedness of h given by $\widehat{V}_{\mathrm{HK}}(h) < \infty$ and $g \in C^s$, it follows that $g \circ h$ and thus the term $(*)$ is bounded on the whole integration domain $[a, b]$. Therefore, it suffices to show that

$$\int_{[a^v, b^v]} \prod_{B \in \pi} \left| \frac{\partial^{|B|}}{\prod_{j \in B} \partial x_j} h(x^v; b^{-v}) \right| dx^v < \infty. \tag{7.3}$$

Following the assumption it holds for all factors with $B \in \pi$ that $\frac{\partial^{|B|}}{\prod_{j\in B}\partial x_j} h(x^v; b^{-v}) \in L^1([a,b])$. Thus, by the generalized Hölder inequality, see for example [3, Corollary 2.11.5] applied to equation (7.3) and using the known relation $L^p([a^v,b^v]) \subseteq L^1(a^v,b^v)$ for $p \geq 1$, it suffices to show that

$$\int\limits_{[a^v,b^v]} \left| \frac{\partial^{|B|}}{\prod_{j\in B}\partial x_j} h(x^v; b^{-v}) \right| dx^v < \infty. \tag{7.4}$$

for all $B \in \pi$.

Recalling the definition of $\widehat{V}_{\mathrm{HK}}(h)$ from Remark 7.1.3 and with the assumption that $\widehat{V}_{\mathrm{HK}}(h) < \infty$ one gets:

$$\widehat{V}_{\mathrm{HK}}(h) = \sum_{\substack{v\subseteq\{1,\dots,s\}\\ v\neq\emptyset}} \int\limits_{[a^v,b^v]} \left| \frac{\partial^{|v|}}{\prod_{j\in u}\partial x_j} h(x^v; b^{-v}) \right| dx^v < \infty. \tag{7.5}$$

Consequently, all summands in (7.5) are bounded. Thus, equation (7.4) holds for all $B \in \pi$ and consequently, the Hardy–Krause variation of the composition $g \circ h$ is bounded. □

Now we can prove the theorem.

Proof (Theorem 7.2.3).
First, a mollifier function $\eta_\delta \in C^s(\mathbb{R}, \mathbb{R}_+)$ fulfilling

$$\max\left\{ \left\| |a| - \eta_\delta(a) \right\|_\infty \right\} \leq \delta \quad \text{for all } a \in \mathbb{R},$$

is chosen. Mollifying the absolute value function in a neighborhood of the origin guarantees the existence of such a function.

The following auxiliary functions are defined:

$$\mathcal{E}_G^\delta := \int_X \eta_\delta(f(x) - \hat{f}(x))$$
$$\mathcal{E}_T^\delta := \frac{1}{N}\sum_{n=1}^{N} \eta_\delta(f(x_n) - \hat{f}(x_n))$$

Next, it is shown that the mollified error $\eta_\delta \circ (f - \hat{f})$ has bounded Hardy–Krause variation.

Since $\hat{f}$ is a composition of affine functions and the activation function $\sigma \in C^s$ by definition (6.1), $\hat{f} \in C^s([0,1]^s, \mathbb{R})$ and $\hat{f} \in L^1([0,1]^s)$ holds. Consequently, it follows that

$$V_{\mathrm{HK}}(\hat{f}) \equiv \widehat{V}_{\mathrm{HK}}(\hat{f}) < \infty.$$

By the equation (7.2) and from the given assumption $\widehat{V}_{\mathrm{HK}}(f) < \infty$, it follows that

$$\widehat{V}_{\mathrm{HK}}(f-\hat{f}) \leq \infty \tag{7.6}$$

Using the assumptions on f and equation (7.6), Corollary 7.2.5 can be applied to $\eta_\delta \circ (f-\hat{f})$ which yields

$$V_{\mathrm{HK}}\big(\eta_\delta \circ (f-\hat{f})\big) \leq \widehat{V}_{\mathrm{HK}}\big(\eta_\delta \circ (f-\hat{f})\big) \leq C_1 < \infty.$$

for a constant C_1. Therefore, one can utilize the Koksma-Hlawka inequality (Theorem 4.2.1) to $f-\hat{f}$ to get

$$|\mathcal{E}_G^\delta - \mathcal{E}_T^\delta| \leq C_2 \cdot \frac{V_{\mathrm{HK}}(\eta_\delta(f-\hat{f}))(\log N)^s}{N}. \tag{7.7}$$

With $C = C_1 \cdot C_2$, where the constant C_1 depends on δ and the dimension s, it follows that

$$|\mathcal{E}_G^\delta - \mathcal{E}_T^\delta| \leq C \cdot \frac{(\log N)^s}{N}. \tag{7.8}$$

By construction of η_δ, it holds that

$$\big|\mathcal{E}_G - \mathcal{E}_G^\delta\big| \leq \delta \qquad \text{and} \qquad \big|\mathcal{E}_T - \mathcal{E}_T^\delta\big| \leq \delta. \tag{7.9}$$

By applying the triangle inequality to the initial generalization gap, it follows that

$$\begin{aligned} |\mathcal{E}_G - \mathcal{E}_T| &\leq \underbrace{|\mathcal{E}_G - \mathcal{E}_G^\delta|}_{\leq \delta} + \underbrace{|\mathcal{E}_T^\delta - \mathcal{E}_T|}_{\leq \delta} + |\mathcal{E}_G^\delta - \mathcal{E}_T^\delta| \\ &\leq 2\delta + |\mathcal{E}_G^\delta - \mathcal{E}_T^\delta| \\ &\leq 2\delta + C \cdot \frac{(\log N)^s}{N}. \end{aligned}$$

The bound on the generalization error then follows as stated in the theorem:

$$\mathcal{E}_G \leq \mathcal{E}_T + C \cdot \frac{(\log N)^s}{N} + 2\delta. \tag{7.10}$$

□

Remark 7.2.6.
Theorem 7.2.3 gives an upper bound on the generalization error. If $f \in C^s([0,1]^s, \mathbb{R})$, the assumptions of the theorem regarding f are necessarily fulfilled. However, this is a stronger assumption on the ground truth function than actually needed. By the demonstrated theorem, it suffices, that $\widehat{V}_{\mathrm{HK}}(h) < \infty$ and $\frac{\partial^{|v|}}{\prod_{j \in v} \partial x_j} h(x^v; b^{-v}) \in L^1([a,b])$ for all $\varnothing \neq v \subseteq \{1, \ldots, s\}$.

Remark 7.2.7.
To apply Theorem 7.2.3 the activation function σ of the neural network has to be s-times continuously differentiable. This is, for example, the case for the commonly

used sigmoid or hyperbolic tangent activation functions. However, popular activation functions such as ReLU are not covered by this theorem.

Chapter 8

Empirical Evaluation and Discussion

In the last chapter, the Koksma–Hlawka inequality was applied to the function approximation problem via Deep Neural Networks (DNNs). These theoretical results yielded a significant improvement in the approximation of the generalization error for training sets sampled from low-discrepancy sequences compared to pseudo-random sampling.

In this chapter, these theoretical results are empirically evaluated for different examples.

8.1 Implementation details of the DNN surrogate training

Before applying the Deep Learning algorithm to optimize the weights θ of the Deep Neural Networks for the approximation of different ground truth functions f later in this chapter, some implementation details are provided in the following. The ground truth functions studied in this chapter are the smooth sum of sines function on multiple dimensions and the approximate Ordinary Differential Equation (ODE) solution to the distance achieved by projectile motion. The implementation is done in Python using the *PyTorch* library [27] and accessible via GitHub[1].

As the main goal of this chapter is to evaluate the impact of different sampling strategies, the Deep Learning algorithm was applied on training sets sampled from three different strategies: pseudo-random sampling and two low-discrepancy sequences: Sobol' and Halton as introduced in Chapter 4. Each training set is obtained by evaluating the ground truth function f at the sampled points. In the experiments, the number of training samples N is varied from 64 up to 8192. According to the training set, a test set $\mathcal{T}$ is chosen from the remaining sampling strategies. To ensure comparability and uniform coverage of the domain, the test sets are always chosen to be the full low-discrepancy sequence with $|\mathcal{T}| = 8192$ points. The setup can be summarized as in Table 8.1.

[1] https://github.com/janikvhrubant/photon-imaging-sim

Training Set	Test Set
Pseudo-random	Sobol'
Sobol'	Halton
Halton	Sobol'

Table 8.1: Test sets used for the evaluation of the DNNs trained on different training sets.

For the evaluation of the approximation quality, the loss function (6.2) for $p = 1$ is evaluated on the test set $\mathcal{T}$, yielding:

$$\mathcal{L}_{\mathcal{T}}(\theta) := \frac{1}{|\mathcal{T}|} \sum_{y \in \mathcal{T}} |f(y) - \hat{f}_\theta(y)|,$$

Further, the Deep Learning algorithm is evaluated for a varying number of epochs between 100 and 2000 together with the Adaptive Moment Estimation (Adam) optimizer [9] introduced in Chapter 7.

For the other training hyperparameters, a systematic study was conducted using *Optuna* [1]. Hereby the architecture of the DNNs was varied in terms of the number of hidden layers, the number of neurons per layer and the activation function. Further different batch sizes were examined. For the two optimizers, different parameters were studied. The parameter ranges are summarized in Table 8.2. The specific configurations used in the experiments will be detailed in the corresponding sections.

DNN Architecture	
Number of hidden layers	2 – 20
Number of neurons per layer	2 – 12
Activation function	Sigmoid, Tanh
Training Setup	
Batch size	64, 128, 256, 512, 1024
Adam Hyperparameters	
Learning rate	$1.0e{-}5$ – $1.0e{-}2$
Weight decay	$1.0e{-}8$ – $1.0e{-}3$
β_1	0.85 – 0.99
β_2	0.95 – 0.999
ϵ	$1e-8$

Table 8.2: Configurations used for Hyperparameter optimization.

8.2 Multi-dimensional sum of sines

As a first example, the multidimensional and nonlinear sum of sines function is studied for $s = 6$ dimensions. The function was introduced in [19] as a benchmark example. Further the function will be analyzed for $s = 8$ dimensions to study the

impact of the curse of dimensionality.

$$f(\mathbf{x}) = \sum_{i=1}^{s} \sin(4\pi x_i), \quad \mathbf{x} \in [0,1]^s. \tag{8.1}$$

To add oscillation to the function, the input is scaled by a factor of 4π. It is clear, that $f \in C^\infty([0,1]^s)$ and thus $V_{\mathrm{HK}}(f) \sim \hat{V}_{\mathrm{HK}}(f) < \infty$ on the compact domain $[0,1]^s$.

Evaluating different hyperparameters (as in Table 8.2) for the DNNs training resulted in the following configurations yielding consistently good and reliable results specified in Table 8.3.

DNN Architecture	
Number of hidden layers	12
Number of neurons per layer	4
Activation function	Sigmoid
Adam Parameters	
Learning Rate	1.0×10^{-1}
Weight Decay	1.0×10^{-6}
β_1	0.9
β_2	0.999
ε	1×10^{-8}

Table 8.3: DNN and Adam configurations for the 6D sum of sines.

The Deep Learning algorithm was applied for various number of training set sizes with $N \in \{32, 64, 128, 256, 512, 1024, 2048, 4096, 8192\}$ samples drawn from MC randoms, Sobol' and Halton sequences. The results were stored after each epoch for up to 2000 epochs.

The training evaluation of the test error on the corresponding test sets for the different epochs is illustrated in Figure 8.1.

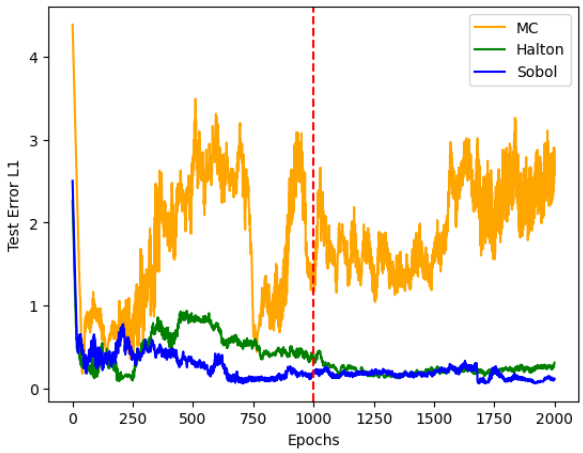


Figure 8.1: Test error for training set size $N = 1024$ for the 6D sum of sines function.

Beginning from ~ 1000 epochs, the test error shows a significant improvement for training data sampled according to Sobol' and Halton sequences. Therefore, an evaluation of the test error for different training set sizes N is conducted for 1000 epochs. The results are illustrated in Figure 8.2 in a $\log_2 - \log_{10}$ plot.

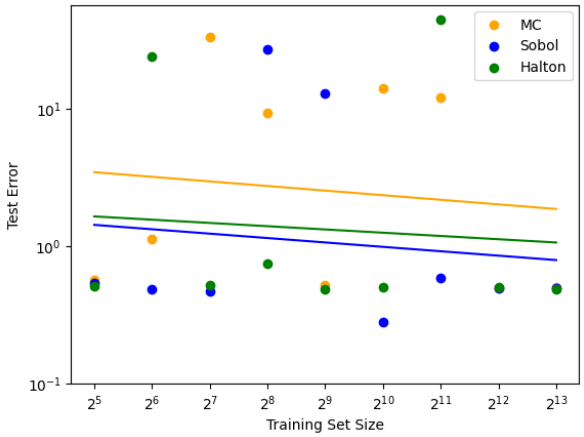


FIGURE 8.2: Test error for the 6D sum of sines function approximation by training with different sizes N.

To analyze the convergence behavior, linear regression lines are fitted to the data points. The results show similar rates of convergence for the MC and Sobol' sampling, although the QMC sampling strategies show better fit to the ground truth function. The Sobol' sequence shows a slightly better performance compared to the Halton sequence. Further, the Halton sequence seems to stagnate for larger training set sizes.

It is important to mention that the results are highly dependent on the chosen hyperparameters. For different configurations, the results can vary significantly.

8.3 Projectile Motion

As a second illustrative example, this section investigates the motion of a projectile. The experiment is introduced by formulating the underlying ODEs system that describes the physical dynamics of the process. Based on approximate solutions of these ODEs at the prescribed training points, DNNs are trained and subsequently evaluated.

8.3.1 ODE formulation of projectile motion

Projectile motion describes the trajectory of an object launched into the air and evolving under gravity and aerodynamic drag. We consider a planar motion with state

$$x(t) \in \mathbb{R} \quad \text{(horizontal position)}, \qquad h(t) \in \mathbb{R} \quad \text{(vertical position)},$$

and velocities $u(t) = \dot{x}(t)$, $w(t) = \dot{h}(t)$, according to the system parameters θ (defined below), including the launch angle α. The motion is terminated at the first impact time

$$t_\star = \inf\{t > 0 : h(t) = 0\},$$

and the traveled range is

$$R(\theta) = x(t_\star). \tag{8.2}$$

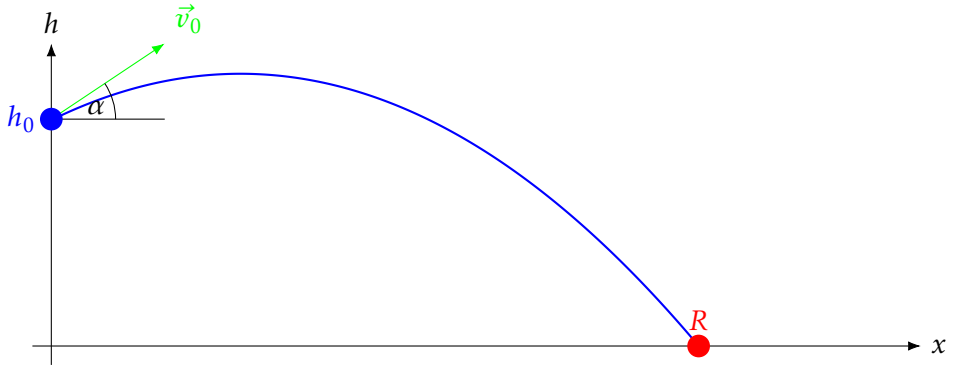


FIGURE 8.3: Projectile motion setup with projectile launched from height h_0 and initial velocity $\vec{v}_0$ at angle α to the horizontal. The final range R is achieved at impact time $t_\star$.

Parameters and modeling assumptions

1. The moving object is modeled as a spherical projectile with mass m in the range $0.3 - 0.78\,\mathrm{kg}$ and radius r in range $0.0213 - 0.124\,\mathrm{m}$. The extreme values correspond to a golf ball of gold and a basketball, respectively.
2. Gravitational acceleration is constant, $g = 9.81\,\mathrm{m/s^2}$.
3. Air density ρ is varying in the range of $0.269 - 1.225\,\frac{\mathrm{kg}}{\mathrm{m}^3}$ which corresponds to the air density at the Mount Everest summit and at sea level, respectively.
4. The drag coefficient c_d of the projectile is constant in the range $0.25 - 0.7$.
5. The initial hight h_0 of the projectile is in the range $1 - 100\,\mathrm{m}$.
6. The initial speed v_0 of the projectile is in the range $1 - 50\mathrm{m/s}$.
7. The launch angle α of the projectile is in the range $0 - 90°$.

Quadratic-drag model

Let $\mathbf{z}(t) = (x(t), h(t), u(t), w(t))^\top$ with horizontal/vertical positions (x,h) and velocities $(u,w) = (\dot{x},\dot{h})$. Denote $\mathbf{v} = (u,w)$ and $\|\mathbf{v}\| = \sqrt{u^2+w^2}$. For a spherical body of radius r and mass m in air of density ρ with constant drag coefficient c_d, the cross-sectional area is $A = \pi r^2$ and the drag-specific constant is

$$k = \frac{\rho\, c_d\, A}{2m} = \frac{\rho\, c_d\, \pi r^2}{2m}.$$

The physically consistent planar model with quadratic aerodynamic drag reads

$$\dot{\mathbf{z}}(t) = \begin{pmatrix} u \\ w \\ -k\,\|\mathbf{v}\|\,u \\ -g - k\,\|\mathbf{v}\|\,w \end{pmatrix}. \tag{8.3}$$

Equivalently, in component form,

$$\dot{x} = u, \quad \dot{h} = w, \quad \dot{u} = -k\|\mathbf{v}\|u, \quad \dot{w} = -g - k\|\mathbf{v}\|w. \tag{8.4}$$

Initial conditions
Given launch speed $v_0 > 0$, angle $\alpha \in [0, \frac{\pi}{2}]$ and height $h_0 \geq 0$,

$$x(0) = 0, \qquad h(0) = h_0, \qquad u(0) = v_0 \cos\alpha, \qquad w(0) = v_0 \sin\alpha. \tag{8.5}$$

Parameterization for learning
We collect the physical parameters in

$$\theta \;=\; (\rho, r, c_d, m, h_0, \alpha, v_0) \in \Theta$$

with the given application-driven bounds:

$$\begin{aligned}
&\rho \in [\rho_{\min}, \rho_{\max}] = [0.269, 1.225]\ \mathrm{kg/m^3}, \quad r \in [r_{\min}, r_{\max}] = [0.0213, 0.124]\ \mathrm{m},\\
&c_d \in [c_{d,\min}, c_{d,\max}] = [0.25, 0.70], \quad m \in [m_{\min}, m_{\max}] = [0.30, 0.78]\ \mathrm{kg},\\
&h_0 \in [h_{0,\min}, h_{0,\max}] = [1, 100]\ \mathrm{m}, \quad \alpha \in [0, \frac{\pi}{2}], \quad v_0 \in [v_{0,\min}, v_{0,\max}] = [1, 50]\ \mathrm{m/s}.
\end{aligned}$$

Here, Θ denotes the Cartesian product of all admissible intervals from above. In the experiments, $\mathbf{y} \in [0,1]^7$ is sampled according to the chosen sampling method. Then, $\theta(\mathbf{y})$ is mapped affinely to the physical parameter ranges via

$$p(\mathbf{y}) = p_{\min} + (p_{\max} - p_{\min})\, y, \quad p \in \{\rho, r, c_d, m, h_0, \alpha, v_0\},$$

and define the parameter-to-output map

$$\mathcal{F} : [0,1]^7 \to \mathbb{R}, \qquad \mathcal{F}(\mathbf{y}) \;=\; R\Big(\theta(\mathbf{y})\Big), \tag{8.6}$$

where $R(\cdot)$ is given by (8.2) for the ODE (8.3)-(8.5). The DNNs are trained to approximate $\mathcal{F}$.

Well-posedness
The right-hand side in (8.3) is continuous and locally Lipschitz in $\mathbf{z}$ (the map $\mathbf{v} \mapsto \|\mathbf{v}\|\mathbf{v}$ is locally Lipschitz), hence for any fixed $\theta \in \Theta$ there exists a unique maximal solution up to $t_\star$. Moreover $h(t)$ is strictly decreasing after the apex, ensuring a unique impact time.

Numerical solution
We integrate (8.3) with a fixed-step explicit scheme. By a small step size $\Delta t > 0$, updates are computed with the forward Euler method

$$\begin{pmatrix} x_{n+1} \\ h_{n+1} \\ u_{n+1} \\ w_{n+1} \end{pmatrix} = \begin{pmatrix} x_n \\ h_n \\ u_n \\ w_n \end{pmatrix} + \Delta t \begin{pmatrix} u_n \\ w_n \\ -k\|\mathbf{v}_n\| u_n \\ -g - k\|\mathbf{v}_n\| w_n \end{pmatrix}.$$

and terminate at the first n with $h_{n+1} \leq 0$. This approximates the traveled range $R \approx x_{n+1}$.

8.3.2 DNN training and evaluation

Since $R \circ \theta \in C^1([0,1]^7)$, we have $\widehat{V}_{\mathrm{HK}}(R \circ \theta) < \infty$. For small enough step size Δt, the numerical approximation of $R \circ \theta$

Again, the deep learning algorithm was applied to various trainin hyperparameters and showed the best performance for the following configurations summarized in Table 8.4.

DNN Architecture	
Number of hidden layers	6
Number of neurons per layer	16
Activation function	Tanh
Adam Parameters	
Learning Rate	1.0×10^{-2}
Weight Decay	1.0×10^{-4}
β_1	0.9
β_2	0.999
ε	1×10^{-8}

Table 8.4: DNN and Adam configurations for the projectile motion example.

The Deep Learning algorithm was applied for various number of training set sizes with $N \in \{32, 64, 128, 256, 512, 1024, 2048, 4096, 8192\}$ samples drawn from MC randoms, Sobol' and Halton sequences. The results were again stored after each epoch for up to 2000 epochs.

The training evaluation of the test error on the corresponding test sets for the different epochs is illustrated in Figure 8.4.

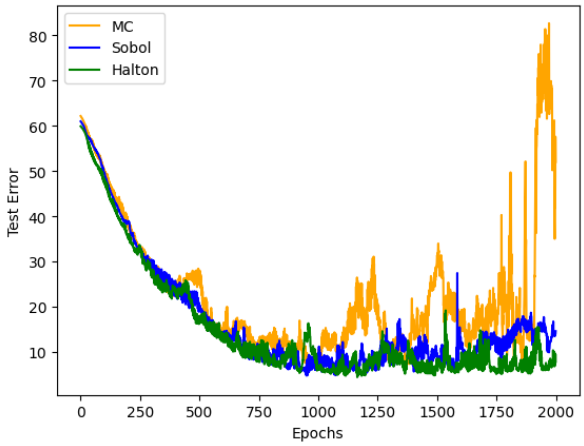


FIGURE 8.4: Test error for training set size $N = 1024$ for the projectile motion function.

TODO: Evaluation mit Plots

8.4 Conclusion

QMC war zwar auch häufig unter den besten trainings zu finden, allerdings lange nicht ausschliesslich. monte carlo war auch häufig dabei.

Part III

X-ray Simulation using QMC Methods

Chapter 9

Motivation and Problem Statement

9.1 Computed Tomography Imaging

Computed Tomography (CT) imaging is a widely used diagnostic technique that generates detailed cross-sectional images by combining multiple X-ray projections acquired from different angles. An X-ray source rotates around the patient while a detector array records the transmitted radiation. Each projection reflects the cumulative attenuation of X-rays as they traverse tissues of varying density and composition.

Reconstruction algorithms, such as filtered backprojection or iterative methods, convert these projections into volumetric images, which can be further processed to enhance contrast and reduce noise. A key challenge is scattered radiation (Compton and Rayleigh scattering), which introduces artifacts and degrades image quality.

The goal of *scatter reduction in CT* is therefore to suppress scatter contributions in the acquired two-dimensional projection data, thereby improving the accuracy of the reconstructed three-dimensional volume.

9.2 X-ray Imaging and the Challenge of Scatter

X-ray CT relies on measuring the attenuation of X-rays along straight-line paths through a phantom. The photon trajectories typically extend from an X-ray source $\mathcal{S}$ to a detector element $\mathcal{D}$, forming the path:

$$l = \overrightarrow{\mathcal{SD}}$$

Under idealized conditions, the attenuation process is governed by the *Lambert-Beer law*, which describes an exponential decay in intensity I as the beam interacts with the material. As stated in [20, Chapter 7], this relationship is given by:

$$I = I_0(E) \cdot \int_0^{E_{\max}} \exp\left(- \int_{\mathcal{S}}^{\mathcal{D}} \mu(x,E)\, dx \right) dE \tag{9.1}$$

Here, $I_0(E)$ denotes the incident intensity of X-rays with initial energy E at the source $\mathcal{S}$. $\mu(x,E)$ represents the linear attenuation coefficient at spatial location x along the path l, for energy E. The attenuation coefficient represents the attenuation per unit length and usually has the unit $\frac{1}{cm}$. The resulting intensity I is measured at the detector element $\mathcal{D}$.

Although exponential attenuation along straight-line paths such as $\overrightarrow{SD}$ is the idealized model for X-ray imaging, this assumption is systematically violated in practice. As X-rays traverse the scanned phantom, they undergo scattering interactions — such as Compton or Rayleigh scattering — that alter their trajectories. Despite deviating from the primary path, these scattered X-rays may still reach the detector, adding unintended signals to the detector. As a result, the measured intensities no longer represent pure line integrals of the attenuation map. This discrepancy introduces nonlinear errors and visible artifacts in the reconstructed image, ultimately degrading both visual quality and quantitative accuracy.

Scattered radiation is a major source of image artifacts — such as cupping and streaks — and reduces both spatial and contrast resolution. These effects not only degrade visual image quality but also compromise the accuracy of quantitative measurements. Precise CT-images are important for various clinical applications, such as treatment planning in radiation therapy. Generating accurate three-dimensional models of the phantom is crucial for precise dose calculations — strong enough to be effective but sufficiently weak to minimize unintended tissue damage.

In clinical practice, such three-dimensional patient models are represented in Hounsfield Units (HUs), which quantify tissue-specific attenuation properties and form the standard basis for accurate dose calculations in radiation therapy [20, Chapter 8].

In CT, the reconstruction algorithm produces a volumetric map of effective attenuation coefficients, from which each voxel value is normalized according to the Hounsfield definition. The resulting HU values form the basis for the displayed grayscale intensities in the CT image.

HU provide a dimensionless measure of X-ray attenuation. They are defined by normalizing the reconstructed attenuation coefficient μ with respect to water and air:

$$\mathrm{HU} = 1000 \cdot \frac{\mu - \mu_{\mathrm{water}}}{\mu_{\mathrm{water}} - \mu_{\mathrm{air}}} \approx 1000 \cdot \frac{\mu - \mu_{\mathrm{water}}}{\mu_{\mathrm{water}}}, \tag{9.2}$$

where μ_{water} and μ_{air} denote the attenuation coefficients of water and air, respectively. By convention, water is assigned 0 HU and air -1000 HU, while dense bone can exceed $+1000$ HU. Although HU reduce scanner- and protocol-dependent variability, they still depend on factors such as tube voltage and reconstruction settings. They are therefore best understood as a standardized but not absolute measure of tissue attenuation in clinical CT imaging.

The impact of scatter becomes especially pronounced in modern CT systems using high-energy X-rays or large-area flat-panel detectors, where scatter may dominate the measured signal. Accurate modeling and correction of scatter are thus essential for achieving high-fidelity CT images, particularly in clinical applications where precision and reliability are paramount [20].

For precise estimation of the models and **HU!** (**HU!**) values respectively, it is therefore important to correct for scatter effects in the acquired projection data.

9.3 Motivation for X-Ray Simulation

In X-ray imaging, scattered photons are a major source of image artifacts and quantitative inaccuracies. To mitigate these effects, a range of computational scatter correction methods has been developed. These approaches can be broadly classified

into the following categories:

- **Empirical and Analytical Methods:**
  These include techniques such as primary modulation, convolution-based correction, and energy windowing. Scatter is typically estimated using simplified models or empirical kernels, often assuming a smooth background distribution, and then subtracted from the measured signal. While computationally efficient, these methods rely on assumptions regarding the spatial and energy distribution of scattered photons. As a result, they may fail to accurately model complex scatter phenomena in heterogeneous anatomical structures.
- **Physics-Based Models:**
  These methods aim to provide a more accurate representation of the underlying photon transport physics, including scattering phenomena. Physics-based models typically rely on simulating the interaction of X-rays with matter by simulating the X-rays on photon level. Among them, MC simulation is regarded as the most rigorous and comprehensive technique due to its ability to statistically model complex photon interactions without relying on simplifying assumptions. In such simulations, primary and scattered photon contributions are detected separately, allowing the estimated scatter signal to be subtracted from measured data in order to restore image fidelity.

In 2011, Sisniega et al. already demonstrated promising results[32] using computationally expensive MC simulations of CT scans of a cylindrical phantom with two bone inserts. By using scatter correction methods, the image quality leads to significant improvements. Motivated by the work of Sisniega et al., who applied scatter correction to CT images via MC photon simulation, this part of the thesis aims to develop more efficient simulations using QMC methods.

In this thesis, a QMC simulation method is presented that further enhances the efficiency of MC photon simulations. In Figure 9.1, the QMC simulation, which will be explained in detail in the following chapters, is used to simulate an X-ray image of a similar phantom composition as by Sisniega et al. The Sobol' sampling method was used and 5×10^6 photons were simulated from a spectrum with a maximum energy of 35 keV which corresponds to a 35 kVp X-ray tube voltage.

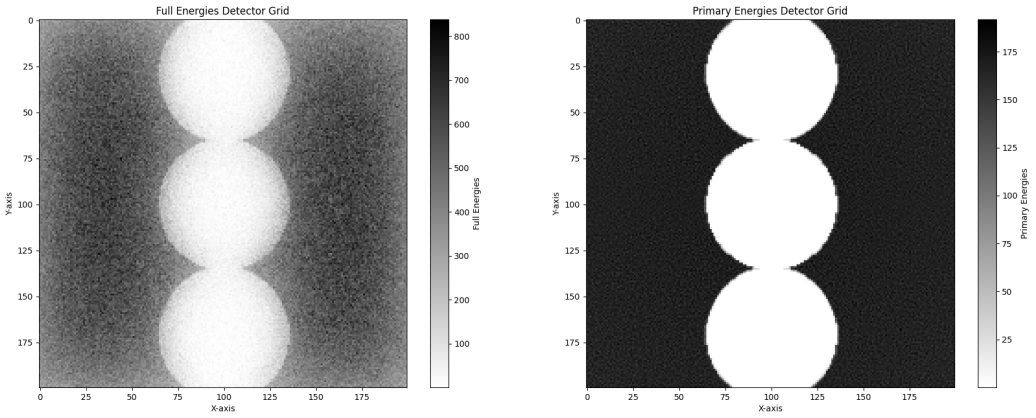


FIGURE 9.1: QMC generated Cone-Beam X-ray image with $5e{+}6$ photons of a 70 mm soft-tissue cylinder with three bone inserts. Left: Simulation of a X-ray image with Compton scattering. Right: Image after removing Compton scatter.

9.4 Monte Carlo Photon Simulation

For the implementation of scatter correction methods, the rays need to be simulated at the photon level. Therefore, each photon must be simulated by following its physical interactions, thereby fulfilling the mathematical distributions of attenuation. Scatter correction using MC methods proceeds by first estimating a three-dimensional phantom from the X-ray projections. An MC photon transport simulation is then performed to estimate the scatter signal $I_{\text{scattered}}$ for each detector pixel. The corrected signal is obtained by subtracting the scatter contribution from the measured intensity:

$$I_{\text{corrected}} = I_{\text{measured}} - I_{\text{scattered}}\,. \tag{9.3}$$

MC simulation is widely regarded as the gold standard for scatter correction in CT and related imaging modalities as they capture the stochastic nature of photon transport, including Compton and Rayleigh scattering, photoelectric absorption and multiple scattering events, based directly on fundamental cross-sections and material properties. They can incorporate complex phantom geometries, heterogeneous tissue composition and realistic X-ray spectra, producing highly accurate scatter estimates that serve as reference standards in validation studies.

The general procedure of MC-based scatter estimation follows a sequence of steps [32]:

1. **Photon Emission:** Photons are sampled from a virtual X-ray source according to a predefined energy spectrum.
2. **Photon Tracking:** Each photon is propagated through the object. At each step, the interaction probability is determined by the local material properties and cross-sections.

3. **Interaction Sampling:** For each interaction, the event type (Compton, Rayleigh, or photoelectric) is sampled, and the photon's energy and direction are updated accordingly.
4. **Detection:** Photons reaching the detector—whether primary or scattered—are recorded, allowing estimation of both primary and scatter contributions.
5. **Statistical Averaging:** A sufficiently large photon population is simulated to obtain a stable, statistically robust estimate of the scatter distribution.

A schematic pseudocode representation of this workflow is summarized in Figure 9.2.

Monte Carlo Photon Transport Algorithm

- **Initialization:** Define geometry, material composition and source spectrum.
- **Loop over photons:**
  - Sample the step size to the next interaction point.
  - Move the photon; check for boundary crossings.
  - Sample the interaction type; update direction and energy.
  - If the photon reaches the detector, record the event.
  - If the photon is absorbed or exits the system, terminate its trajectory.
- **Aggregation:** Compute both scatter and primary intensity distributions.

FIGURE 9.2: Pseudocode representation of the Monte Carlo algorithm for photon transport in scatter modeling.

Although MC methods yield highly accurate and physically correct scatter estimates, they are computationally demanding since a very large number of photons must be simulated. To obtain a statistically stable estimate of the scatter distribution, photons with a wide range of energies, directions, and interaction histories have to be tracked. Moreover, only a fraction of these photons eventually reach the detector and contribute to the image, while the majority are absorbed or scattered in other directions. Consequently, the required computation times can become very high even for moderately complex phantoms and increase further with higher accuracy demands.

9.5 Prospects of QMC Simulations

While traditional MC methods are widely used for the development of new CT scanners and are commonly referred to as benchmarks for X-ray image simulation, they lag behind in terms of computational efficiency and speed. Therefore, MC-based scatter correction is almost never applied in clinical practice, where time and computational resources are limited.

One broadly adopted simulation framework is the *Geant4* toolkit [6]. Initially released in 1998 for high-energy physics applications, Geant4 has since been extended to support medical physics and imaging. However, as the architecture of Geant4 is built around MC methods, so adopting QMC would require significant structural

changes [13, 8]. This limitation might be one explanation why QMC methods have so far not found widespread application in X-ray simulation.

Recently, QMC-based photon transport algorithms have been proposed for scatter simulation. Lin, Deng, and Wang introduced the GPU-accelerated gQMCFFD algorithm, which combines QMC sequences with forced fixed detection [17]. In a series of numerical experiments on different phantoms (aluminium, bone–tissue cylinder, Shepp–Logan and head phantom), the method produced results in close agreement with reference MC simulations. Reported relative differences were below 1.5%, while the efficiency improvement factor varied between 4 and 46, depending on the geometry. Subsequent work is supporting this approach [18, 8]. As this resource group is not publishing their code, it was not possible to compare the results directly.

Therefore, the goal of this thesis is to develop a proof of concept for QMC photon transport simulation which is more generally applicable, to demonstrate the feasibility of, and argue for the benefits of QMC methods in X-ray simulation. The developed method will be compared to the same MC reference simulation, to evaluate the accuracy and efficiency of the QMC approach.

Chapter 10

Physical Laws of Photon Simulation

This chapter introduces the physical and mathematical principles underlying the simulation of X-ray imaging. Central to this is the modeling of individual photon interactions with matter, including attenuation and scattering processes. These interactions are inherently stochastic due to the quantum nature of radiation-matter interactions. The stochastic nature of photons and their interactions with matter is modeled using monte Carlo methods of uniformly distributed values between 0 and 1, which are then transformed to follow the physical probability distributions relevant to the specific interactions being simulated. This approach allows for a realistic representation of photon transport and interaction within the imaging system.

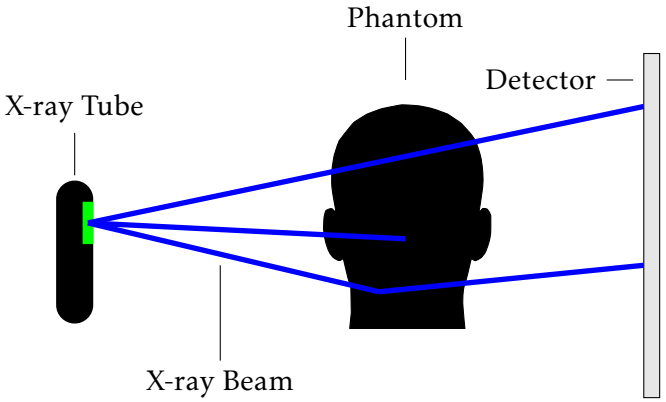


Figure 10.1: Schematic illustration of photon transport in X-ray imaging.

In a typical simulation workflow, individual photons are emitted from the X-ray tube and propagate through air before reaching the phantom. Upon entering the phantom, they may interact with the material through scattering or absorption, depending on the local composition of the tissue and photon energy. Only a fraction of the photons will exit the phantom, continue their path through air, and ultimately reach the detector, contributing to the final image formation.

The simulation framework described here forms the basis for all subsequent analyses presented in this thesis.

10.1 X-Ray Tube Spectrum

X-ray beams are generated within evacuated glass tubes containing several critical components that convert electrical energy into X-ray photons and heat. The X-ray tube acts as an energy converter, where the vast majority of the input energy is transformed into heat and only a small fraction becomes useful radiation. The following Figure 10.2 illustrates the basic construction of an X-ray tube as in [29]

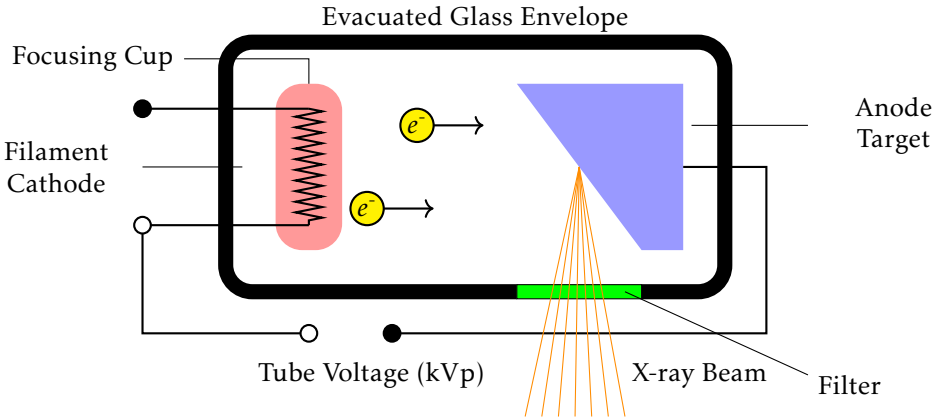


FIGURE 10.2: Schematic of an X-ray tube with filter.

To capture this randomness, Monte Carlo methods are employed. In such simulations, random numbers—typically sampled uniformly from the interval $[0,1]$ are transformed into physically meaningful quantities according to the relevant probability distributions. This probabilistic modeling enables realistic simulation of photon transport and forms the basis for the analyses presented in this thesis.

The key components are:

- **Filament (Cathode):** A tungsten wire filament serves as the electron source through thermionic emission. When a current of approximately 3–6 A passes through it, the filament reaches incandescence, releasing electrons from its surface. These free electrons form a cloud near the cathode until they are accelerated toward the anode by the applied high voltage.
- **Focusing Cup:** The filament is embedded in a negatively charged, nickel-made focusing cup. Its function is to electrostatically shape and direct the electron stream toward the anode's focal spot, thereby influencing the resolution and size of the resulting X-ray beam.
- **Anode Target:** The anode consists of a tungsten target, often embedded in a copper support. Tungsten is used for its high atomic number and melting point, enhancing X-ray production and durability. The copper base improves heat dissipation. Typically, less than 1% of the electron energy is converted into X-rays, with the remainder generating heat that must be effectively managed.
- **Evacuated Glass Envelope:** All components are sealed within a borosilicate glass or metal-ceramic housing evacuated to a low pressure (typically 10^{-5} to 10^{-7} hPa). The vacuum allows unimpeded electron flow and prevents arcing.

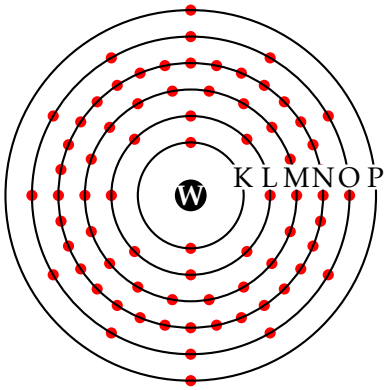
The housing is usually immersed in insulating oil to provide thermal and electrical isolation.

- **Filter (Filtration):** A filter is positioned at the tube exit, between the anode target and the patient, to selectively absorb low-energy photons. These photons contribute little to image formation but significantly increase patient dose. Their removal (beam hardening) improves image quality by reducing scatter and enhancing contrast. Common filter materials are aluminum, copper and molybdenum; in this thesis, a clinical setup with 1 mm aluminum (Al) and 0.2 mm copper (Cu) is used [29, 33].

When high-energy electrons strike the tungsten target, X-ray photons are produced through two primary mechanisms:

- **Bremsstrahlung (Braking Radiation):** This accounts for approximately 80% of X-ray production. When electrons pass close to tungsten nuclei, they are decelerated by the electrostatic attraction, causing them to lose kinetic energy that is emitted as X-ray photons. This process produces a continuous spectrum of X-ray energies from near zero up to the maximum electron energy.
- **Characteristic Radiation:** When high-energy electrons eject inner-shell electrons from tungsten atoms, vacancies are created in the K or L shells. These vacancies are subsequently filled by electrons from outer shells and the resulting energy difference is emitted as an X-ray photon with a discrete, element-specific energy. The corresponding peaks in the spectrum therefore occur at the differences of the binding energies of the involved shells. For reference, the atomic model of tungsten is shown in Figure 10.3a and the approximate binding energies of its shells and subshells are listed in Table 10.3b.

  From these binding energies, the main characteristic line energies can be derived. For example, the $K\alpha_1$ line corresponds to an electron transition from the L_3 subshell into the K-shell (69.5–10.2 keV $\approx$ 59.3 keV), the $K\alpha_2$ line results from the $L_2 \rightarrow$ K transition (69.5–11.5 keV $\approx$ 58.0 keV) and the $K\beta_1$ line originates from the $M_2/M_3 \rightarrow$ K transition (69.5–2.3 keV $\approx$ 67.2 keV). These discrete transitions explain the multiple sharp peaks superimposed on the continuous bremsstrahlung spectrum. In practice, only the K, L and M shells are relevant for X-ray production, while binding energies of the N shell and higher are too small to contribute significantly.



(A) Bohr model of tungsten (W, Z=74) with electron shells.

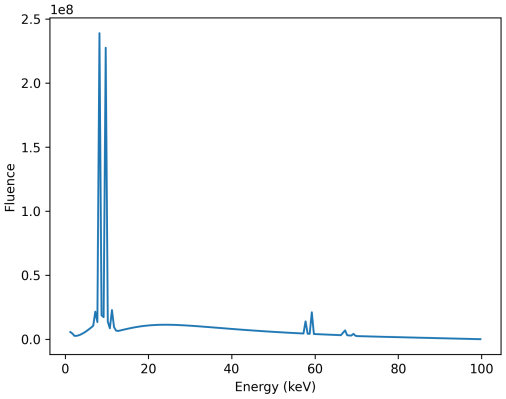
Shell / Subshell	Binding Energy (keV)
K	69.5
L_1	12.1
L_2	11.5
L_3	10.2
M_1	2.82
M_2	2.30
M_3	2.15
N_1	0.43
N_2	0.32
N_3	0.22

(B) Approximate binding energies of tungsten shells/subshells.

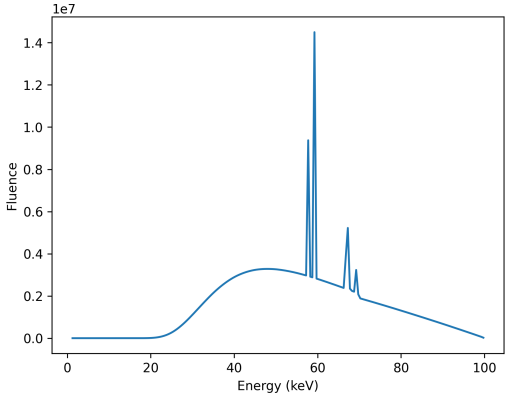
FIGURE 10.3: (a) Tungsten atom schematic and (b) corresponding binding energies.

Although it is not necessary to understand every technical detail of the X-ray apparatus for the purposes of simulation, this background information allows one to assess how key simulation parameters influence the resulting X-ray spectrum and photon behavior.

Figure 10.4a below shows a resulting energy spectrum for an X-ray tube with a tungsten cathode operated at 100 kVp. It illustrates the resulting distribution of photon energies, which is shaped by both the material and the applied voltage. The intensity of the different photon energies is measured as *spectral fluence* in $\mathrm{cm}^{-2}\,\mathrm{keV}^{-1}$, which describes the number of X-ray photons per unit area per unit energy interval. It clearly shows the two components of the X-ray spectrum: the continuous bremsstrahlung spectrum and the characteristic radiation peaks:



(A) Without additional filter.
Features: continuous bremsstrahlung up to 100 kVp; pronounced L-lines (~8-12 keV) and K-lines (~59-67 keV).



(B) With filter (1 mm Al + 0.2 mm Cu).
Effect: removal of low-energy photons (beam hardening); suppressed L-lines; continuum shifted toward higher mean energy.

FIGURE 10.4: X-ray spectra for a tungsten anode at 100 kVp: (a) without and (b) with added filtration. The filter selectively attenuates low-energy photons, reducing patient dose while preserving higher-energy components. Both spectra were generated using *SpekPy* [28].

The applied filter in Figure 10.4b effectively removes low-energy photons below approximately 30 keV, which would otherwise contribute to patient dose without enhancing image quality, as low energy photons are likely to be absorbed superficially. The filter thus "hardens" the beam by increasing its mean energy, which improves penetration and reduces scatter. The characteristic K-lines remain prominent, even though their intensity is slightly reduced due to the filter's attenuation.

10.2 Photon Generation

In the context of Monte Carlo simulations, photons are generated with initial positions, directions and energies that reflect the physical properties of the X-ray source and the imaging setup.

10.2.1 Initial Photon Energy

The initial photon energy is sampled according to the X-ray tube spectrum as described in Section 10.1. As in Figure 10.4, the spectrum maps the photon energy to the according frequency. In the simulation applied in this thesis, the spectra are generated utilizing *SpekPy* [28, 30].

The actual energies of the emitted photons in the simulation are then determined by *inverse transform sampling* [23, Chapter 4]. To determine a matching random variable with the desired distribution, the following steps need to be made:

(i) Construct the appropriate probability density function from the given spectrum.

(ii) Compute the Cumulative Distribution Function (CDF) F from the probability density function.

(iii) Determine the generalized inverse of the CDF F^{-1} as defined below.

(iv) Sample a uniformly distributed random variable U on the interval $[0,1]$ and compute the photon energy as $E = F^{-1}(U)$.

To get the probability density function from the given spectrum, the fluences w_i of the energies e_i in the spectrum are normalized such that

$$p_i = \frac{w_i}{\sum_j w_j}.$$

Now, the CDF $F : \mathbb{R} \to [0,1]$ can be computed as

$$F(e_i) = \mathbb{P}[E \leq e_i] = \sum_{j \leq i} p_j .$$

It is clear that $F(e_i)$ is monotonically increasing with $F(-\infty) = 0$ and $F(\infty) = 1$. The generalized inverse is defined as follows:

Definition 10.2.1 (Generalized Inverse)**.**
Let $f : \mathbb{R} \to \mathbb{R}$ be a monotonically increasing function. The *generalized inverse* $f^{-1} : \mathbb{R} \to \mathbb{R}$ is defined as

$$f^{-1}(y) = \inf\{x \in \mathbb{R} : f(x) \geq y\}.$$

Remark 10.2.2. For discrete distributions with support $\{x_1, \ldots, x_M\}$ with $x_1 < \ldots < x_M$, the generalized inverse can be explicitly expressed as:

$$f^{-1}(y) = \begin{cases} -\infty, & y = 0 \\ x_i, & f(x_{i-1}) < y \leq f(x_i) \\ x_M, & y = 1 \end{cases}$$

As F is monotonically increasing, the generalized inverse F^{-1} exists and is also monotonically increasing. The photon energies are then sampled via

$$E = F^{-1}(U),$$

where U is a uniformly distributed random variable on the interval $[0,1]$.

The following theorem shows that the evaluation of the inverse CDF of the uniformly distributed random variable $U \sim \mathcal{U}(0,1)$ inherits the desired distribution of the spectrum.

Theorem 10.2.3.
Let $F : \mathbb{R} \to [0,1]$ be a cumulative distribution function and F^{-1} its generalized inverse. Then for a random variable X generated by

$$X := F^{-1}(U)$$

for $U \sim \mathcal{U}(0,1)$, it holds that X has cumulative distribution function F, i.e.,

$$\mathbb{P}[X \leq x] = F(x), \quad \text{for all } x \in \mathbb{R}.$$

Proof.
It needs to be shown that

$$\mathbb{P}[F^{-1}(U) \leq x] = F(x).$$

It is clear that $\mathbb{P}[U \leq F(x)] = F(x)$ for all $x \in \mathbb{R}$, since $U \sim \mathcal{U}(0,1)$.

Now let $u \in (0,1)$ and $x \in \mathbb{R}$ and $F(x) \geq u$. Then $F^{-1}(u) \leq x$ as $x \in \{z \in \mathbb{R} : F(z) \geq u\}$. Now let $x > F^{-1}(u) = \inf\{z \in \mathbb{R} : F(z) \geq u\}$. Since F is non-decreasing, it holds that $F(x) \geq u$. Lastly, let $x = F^{-1}(u)$. Since F is additionally right-continuous, it holds that $F(x) \geq u$. Thus, it was shown that

$$\mathbb{P}[F^{-1}(U) \leq x] = \mathbb{P}[X \leq x] = \mathbb{P}[U \leq F(x)] = F(x).$$

□

This proof is adapted from [24]

Remark 10.2.4.
In the continuous setting with a distribution with strictly positive probability density function and thus, strictly increasing CDF, the generalized inverse coincides with the usual inverse.

10.2.2 Photon Position and Direction

In addition to the photon energy at emission, its initial position and direction must be set. Within the simulation framework, two different source models are considered:

1. **Point Source:** Photons are emitted from a single point in space, typically representing the focal spot of the X-ray tube. The emission direction is sampled uniformly within a conical emission region defined by the half-opening angle $\Theta \in [0°, 90°]$.
2. **Extended Source:** Photons are emitted from a finite rectangular area on a plane, with each photon initially traveling perpendicular to this plane.

Point Source
This model reflects the global imaging geometry of an X-ray tube with a point-like focal spot and divergent beam. In line with common practice (e.g. Geant4's angular settings [6]), photon directions are sampled uniformly on a spherical cap, i.e. uniformly in the solid angle.

All photons are emitted from a fixed source position $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2, \mathcal{S}_3) \in \mathbb{R}^3$. To describe the corresponding probability measure, recall that the unit sphere S^2 in spherical coordinates is parameterized by

$$x = \sin\theta\cos\phi, \quad y = \sin\theta\sin\phi, \quad z = \cos\theta,$$

with $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi)$. The induced surface element on S^2 is given by

$$d\Omega = \sin\theta \, d\theta \, d\phi.$$

A uniform distribution on the spherical surface thus corresponds to a constant density with respect to this measure.

For sampling, one draws $u_1, u_2 \sim \mathcal{U}(0,1)$ and sets

$$\phi = 2\pi u_2, \qquad \cos\theta = 1 - u_1\,(1 - \cos\Theta), \qquad \theta = \arccos\Big(1 - u_1(1 - \cos\Theta)\Big),$$

leading to the direction vector

$$\vec{d} = \Big(\sin\theta\cos\phi,\ \sin\theta\sin\phi,\ \cos\theta\Big).$$

This procedure yields a conical beam with half-opening angle Θ whose directions are uniformly distributed over the spherical cap surface. Photon energies are sampled independently from the spectrum as described above. Figure 10.5 illustrates the resulting geometry.

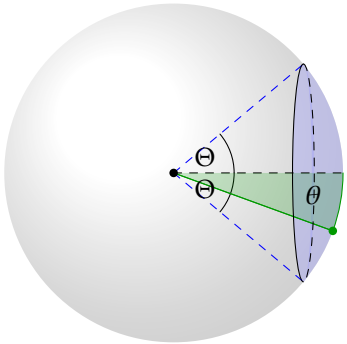


FIGURE 10.5: Sampling of photon directions uniformly within a cone defined by the half-opening angle Θ.

In Geant4's `General Particle Source`, this corresponds to *isotropic* angular sampling restricted to a cone via min/max θ and ϕ (e.g. `/gps/ang/type iso`, `/gps/ang/mintheta 0`, `/gps/ang/maxtheta` Θ, `/gps/ang/minphi 0`, `/gps/ang/maxphi 360 deg`).

Extended Source

In contrast, the finite planar source serves primarily as an analytical model. It describes a local view of the photon–phantom interaction and is employed to study the influence of source extension on scattering and attenuation effects.

The initial position of a photon is sampled uniformly on the rectangular surface of the source. For a rectangular source with vertices $P_1, P_2, P_3, P_4 \in \mathbb{R}^3$ with $\overrightarrow{P_1P_2} \parallel \overrightarrow{P_3P_4}$ and $\overrightarrow{P_1P_4} \parallel \overrightarrow{P_2P_3}$ being parallel, the initial position of a photon is determined by two random variables $u_1, u_2 \sim \mathcal{U}(0,1)$:

$$A_0 = P_1 + u_1 \cdot (P_2 - P_1) + u_2 \cdot (P_4 - P_1).$$

This yields a uniform distribution of starting points over the rectangular surface. The emission direction of each photon is then set to the unit normal vector of the source plane, which is aligned with the detector plane, thus representing a parallel beam geometry.

Such a rectangular planar source model is also supported in Geant4's `General Particle Source`, where planar shapes (square or rectangle) can be chosen for the position distribution and combined with a *planar* angular type to enforce emission perpendicular to the plane (see `/gps/pos/type Plane`, `/gps/pos/shape Rectangle`, `/gps/ang/type planar`). This configuration is commonly employed in benchmark simulations to model collimated or extended X-ray beams.

Within this thesis, the experiments were conducted to simulate an X-ray tube with the parameters listed in Table 10.1.

Parameter	Value
Tube voltage	30–120 kVp
Anode material	Tungsten
Filtration	1 mm Aluminum (Al), 0.2 mm Copper (Cu)
Target angle	10–60°

TABLE 10.1: Parameters used to generate the X-ray spectrum with *SpekPy*.

10.3 Scattering and Attenuation

Following their emission from the X-ray source, photons traverse space until they encounter matter, where various interaction processes may occur. The subsequent section outlines the fundamental mechanisms of photon interactions with matter. The interaction relevant for medical imaging results in a reduction of radiation intensity, which corresponds to a decreased number of photons reaching the detector. Here, X-ray photons may be fully absorbed by *photoelectric absorption* or undergo either *elastic scattering* (Rayleigh) or *inelastic scattering* (Compton) as they interact with matter. In the subsequent sections, these effects will be explained. These processes depend on the photon energy and the atomic molecules in the material respecting the density and molecular distribution of the material. A visualization of these interaction mechanisms is shown in Figure 10.6.

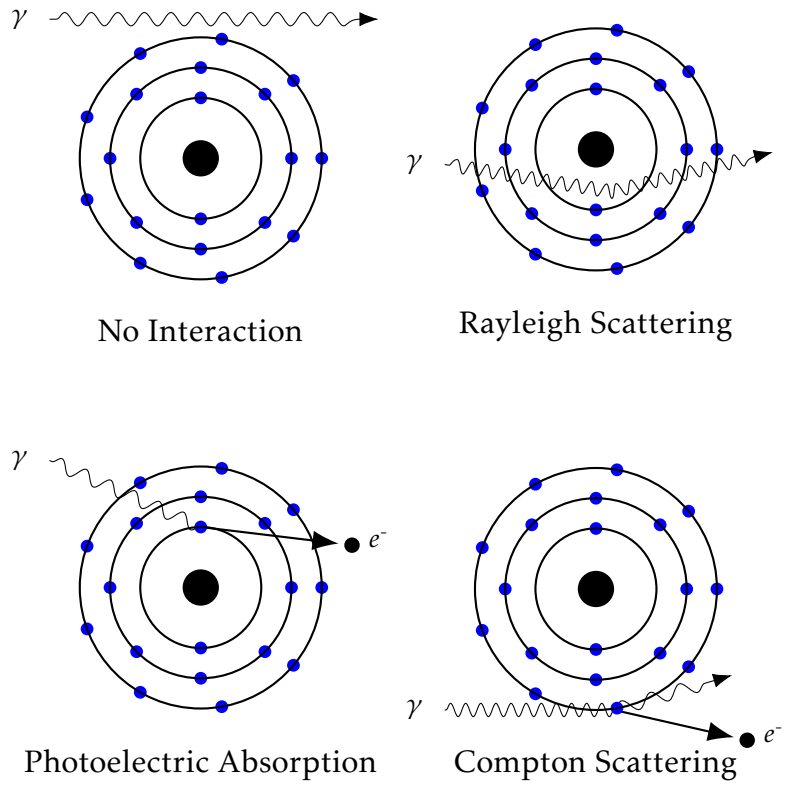


FIGURE 10.6: Principles from photon interaction with matter similar to [20, Chapter 7]

The attenuation of X-ray photons arises from physical processes that alter their number, direction or energy as they interact with matter. These interactions occur at the level of individual photons and are highly dependent on the photon energy. This section presents an overview of the primary interaction mechanisms relevant to attenuation as in [20].

For correctness, in the simulations it is further assumed that the X-ray photons are propagating through air before entering and after exiting the phantom, where the possible interactions with molecules in the air are neglected.

10.3.1 Free Path Length

All mentioned interaction processes - the photoelectric effect, Compton scattering and Rayleigh scattering - are probabilistic in nature. Their attenuation is stochastically characterized by the *mass attenuation coefficient* $\tilde{\mu}$ in $\mathrm{cm}^2\,\mathrm{g}^{-1}$. The attenuation coefficient reflects the composition of the material and the energy of the photon. Even though, the attenuation is depending on the density of the material, the *mass attenuation coefficients* do not depend on the density.

Therefore, the *linear attenuation coefficient* μ in cm^{-1}, which is then applied within the Beer–Lambert law, is obtained by multiplying the mass attenuation coefficient $\tilde{\mu}$ with the density ρ of the material:

$$\mu(x,E) = \rho(x) \cdot \tilde{\mu}(x,E)$$

For comprehensibility, the dependence on the material and density is implicitly expressed by the Cartesian coordinates x of the photon position. This is justified, as the material and density also depend on the spatial location within the phantom. Therefore, a simple and not further mentioned helper function is returning the *linear attenuation coefficient* μ for a given position x and energy E. The coefficients are summarized in Table 10.2.

Interaction Type	**Attenuation Coefficient**
Photoelectric Effect	$\mu_{\mathrm{PE}}(x,E)$
Rayleigh Scattering	$\mu_{\mathrm{RS}}(x,E)$
Compton Scattering	$\mu_{\mathrm{CS}}(x,E)$

Table 10.2: Attenuation coefficients for different photon interaction mechanisms.

The linear attenuation coefficient $\mu(x,E)$ is the sum of the individual linear attenuation coefficients for interactions:

$$\mu(x,E) = \mu_{\mathrm{PE}}(x,E) + \mu_{\mathrm{RS}}(x,E) + \mu_{\mathrm{CS}}(x,E)$$

For the simulations within this thesis, the attenuation coefficients for the respective phantoms are retrieved from the *XCOM Photon Cross Sections Database* [2]. XCOM is standard database provided by the National Institute of Standards and Technology (NIST) that comprehends attenuation coefficients for elements, compounds, and mixtures at energies ranging from 1 keV to 100 GeV.

For the behavior of X-ray photons two main effects are considered:

- **Attenuation:** The photon may be absorbed or scattered.
- **No Interaction:** The photon may pass through the phantom without any interaction.

An interaction may occur at some point along the ray. The probability that a phantom survives from a certain point A to another point B without any interaction, is described as follows [4]:

$$\mathcal{P}(A, \overrightarrow{AB}, E) = \exp\left[- \int_A^B A + \mu\left(\frac{s}{\|\overrightarrow{AB}\|}(B-A), E \right) ds \right]$$

The free path length of a photon is the distance it travels before interacting with matter. To sample the free path length of a photon, one can project the photon within a given direction $\vec{\omega}$ from its current position A to the detector position $\hat{C}$, as shown in Figure 10.7. Then a random variable $U \sim \mathcal{U}(0,1)$ is sampled.

- If $U < \mathcal{P}(A, \vec{\omega}, E)$, the photon reaches the detector without any interaction.
- If $U \geq \mathcal{P}(A, \vec{\omega}, E)$, the photon interacts with matter after traveling the free path length t. The free path length t is then determined by solving the equation

$$\int_0^t \mu(A + s \cdot \vec{\omega}, E)\, ds = -\log(1-U)$$

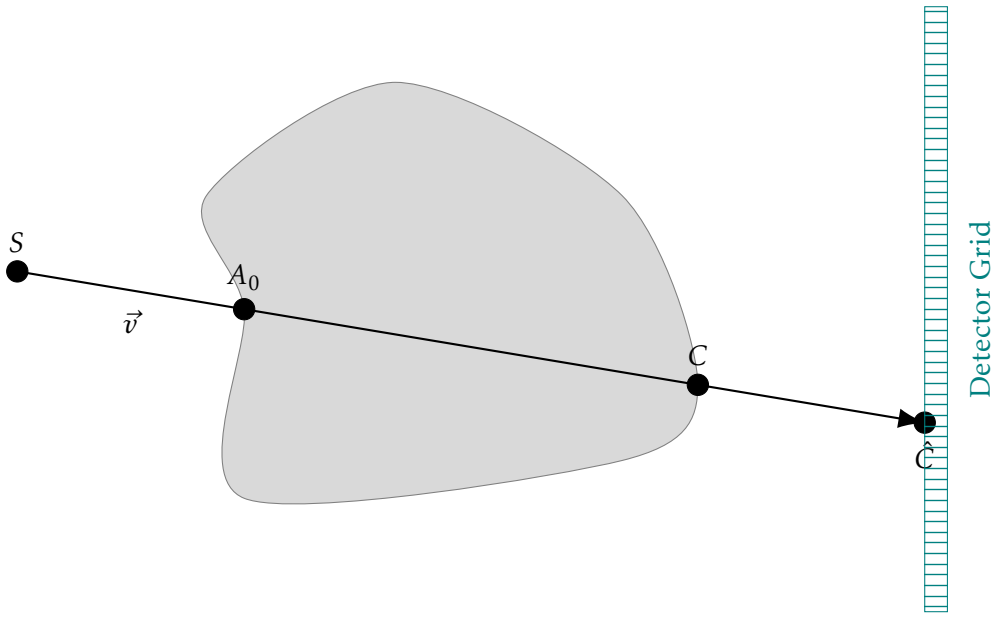


FIGURE 10.7: Illustration of the entry and exit point of the ray of a photon without interaction.

10.3.2 Compton Scattering

Compton scattering is the most dominant interaction mechanism for X-ray photons in tissue [20, Chapter 7]. It occurs when a X-ray photon with considerably higher

energy than the binding energy of an outer shell electron collides with this electron. This interaction results in a transfer of energy and momentum. The electron is ejected from the atom, while the incoming photon is scattered at an angle and its energy is reduced.

A portion of the incident photon energy is transferred to the electron, which is referred to as the "recoil electron" or Compton electron. The interaction produces a positive ion, the "recoil electron" and a scattered photon. If the deflection angle of the scattered photon is small, most of the energy is retained by the scattered photon. The deflection angle can vary from 0 to 180 degrees, depending on the energy transfer during the interaction. Thus, the photon can be scattered in any direction, with the probability of scattering at smaller angles being higher than at larger angles.

For the distribution of the scattering angle θ of the scattered photons in the differential cross section, we apply the formula derived by *Klein and Nishina* in 1928. This equation describes the angular distribution of X-ray photons in Compton scattering, assuming the electron is initially free and stationary. The relation between energy loss and photon deflection is determined from conservation of momentum and energy between the photon and the recoiling electron [12, 15].

The Klein–Nishina formula provides the differential cross section for Compton scattering with scattering angle θ, initial photon energy E and scattered photon energy E'. Thus, θ is the angle between the direction of the incident photon and the direction of the scattered photon.

$$\frac{d\sigma}{d\Omega} = \frac{r_e^2}{2}\left(\frac{E'}{E}\right)^2\left(\frac{E}{E'} + \frac{E'}{E} - \sin^2(\theta)\right)$$

Further, the relation between the energies of the incident photon E and the scattered photon E' is given by the *Compton formula*:

$$E' = \frac{E}{1 + \frac{E}{m_e c^2}(1 - \cos\theta)} = \frac{E}{1 + k(1 - \cos\theta)} \tag{10.1}$$

where $k = \frac{E}{m_e c^2}$ is used for readability. Here, $m_e c^2$ denotes the electron rest energy, which is approximately $0.511 MeV$ and r_e denotes the classical electron radius. Together, this yields the following form of the Klein-Nishina formula:

$$\frac{d\sigma}{d\Omega}(\theta, k) = \frac{1}{2} r_e^2 \frac{1}{(1 + k(1 - \cos\theta))^2}\left(1 + \cos^2\theta + \frac{k^2(1 - \cos\theta)^2}{1 + k(1 - \cos\theta)}\right), \tag{10.2}$$

The Klein–Nishina formula describes the angular distribution of the scattering probability. Here, σ denotes the scattering cross section, which quantifies the probability of an interaction by the effective area for scattering, and Ω denotes the solid angle, describing the three-dimensional angular space into which the photon may be scattered. The expression $\frac{d\sigma}{d\Omega}$ is thus the *differential cross section*, which represents the probability density of photons being scattered into a particular direction.

For sampling from the Klein-Nishina distribution, we cannot apply the Inverse Transform Sampling method as described in Section 10.2.1, since the structure of the

Klein-Nishina formula and the dependency of θ is more complex and does not allow for an analytical inversion. Instead the method of *acceptance–rejection sampling* is used.

Acceptance–rejection sampling is a method for generating random samples from a probability distribution $f(x)$ when direct sampling is difficult. The idea is to employ a simpler envelope distribution $h(x)$, from which samples can be drawn efficiently, and which dominates $f(x)$ up to a constant factor. More precisely, one requires a constant $0 < c \leq 1$ such that $c\,f(x) \leq h(x)$ for all x. A candidate value x is drawn from $h(x)$, and an auxiliary uniform random number $u \sim \mathcal{U}(0,1)$ is generated. The sample x is accepted if $u \leq \frac{c\,f(x)}{h(x)}$, otherwise it is rejected and the procedure is repeated until an acceptance occurs.

The efficiency of the algorithm depends on how tightly $h(x)$ bounds $c\,f(x)$: the closer the proposal envelope matches the target distribution, the higher the acceptance rate. Acceptance–rejection sampling is thus most effective when a proposal distribution can be chosen that both resembles the target distribution and allows efficient evaluation. For more details, see for instance [23, Chap 4] or [16, Section 8.1].

To apply the acceptance–rejection sampling method to the Klein-Nishina formula equation (10.2), we first rewrite $\frac{d\sigma}{d\Omega}(\theta,k)$ to $f(k,x)$ using $x = \cos\theta$ and ignore the constant factors which are not relevant for sampling:

$$f(k,x) = \frac{1}{(1+k(1-x))^2}\left(1 + x^2 + \frac{k^2(1-x)^2}{1+k(1-x)}\right) \tag{10.3}$$

Following [26], we start by choosing an envelope distribution $h(k,x)$ of the form

$$h(k,x) = \frac{a(k)}{b(k)-x}$$

with $a(k), b(k)$ to be determined such that the envelope function matches the target function at the boundaries $x = 1$ and $x = -1$. This leads to the conditions:

$$\begin{aligned} f(k,1) &= 2 = h(k,1) \\ f(k,-1) &= \zeta(k) = h(k,-1) \quad \text{with} \\ \zeta(k) &= \frac{2(2k^2+2k+1)}{(2k+1)^3} \end{aligned}$$

Imposing these endpoint conditions on $h(k,x) = \frac{a(k)}{b(k)-x}$, yields

$$\begin{aligned} \frac{a(k)}{b(k)-1} = 2 \quad &\Rightarrow \quad a(k) = 2(b(k)-1), \\ \frac{a(k)}{b(k)+1} = \zeta(k) \quad &\Rightarrow \quad \frac{2(b(k)-1)}{b(k)+1} = \zeta(k). \end{aligned}$$

Solving the second equation for $b(k)$ by multiplying out gives

$$b(k) = \frac{2+\zeta(k)}{2-\zeta(k)} = \frac{1+\frac{\zeta(k)}{2}}{1-\frac{\zeta(k)}{2}}$$

Thus, the envelope function is given with the values:

$$\begin{aligned} h(k,x) &= \frac{a(k)}{b(k)-x} \\ a(k) &= 2(b(k)-1) \\ b(k) &= \frac{1+\frac{\zeta(k)}{2}}{1-\frac{\zeta(k)}{2}} \end{aligned}$$

For the acceptance–rejection method to be applicable, we need to ensure that $c\,f(k,x) \leq h(k,x)$ for all $x \in [-1,1]$ and some constant $0 < c \leq 1$.

Therefore, consider the ratio $R_k(x) := h(k,x)/f(k,x)$ for $x \in [-1,1]$. Since $f(k,x) > 0$ for all $x \in [-1,1]$ (Klein–Nishina is strictly positive) and $h(k,x) > 0$ because $b(k) > 1$ which implies $b(k) - x > 0$ on $[-1,1]$, we have $R_k(x) > 0$ on $[-1,1]$. At the endpoints, $R_k(\pm 1) = 1$ by the matching conditions above, so $c_k \leq 1$. Therefore, $h(k,x) \geq c_k\,f(k,x)$ for all $x \in [-1,1]$, with a uniform constant $c_k \in (0,1]$.

Now the acceptance–rejection method can be applied. To sample the scattering angle θ, $x = \cos\theta$ is sampled from the distribution h by the inverse transform method:

The cumulative distribution function of h is

$$H(x) = \int_{-1}^{x} h(k,t)\,dt = \int_{-1}^{x} \frac{a}{b-t}\,dt = a\ln\left(\frac{b+1}{b-x}\right),$$

which, after normalization, becomes

$$\tilde{H}(x) = \frac{\ln\left(\frac{b+1}{b-x}\right)}{\ln\left(\frac{b+1}{b-1}\right)}.$$

Setting $\tilde{H}(x) = r$ for $r \sim \mathcal{U}(0,1)$ and solving for x yields

$$x = b - (b+1)\left(\frac{b-1}{b+1}\right)^{r}.$$

With the parametrization

$$b(k) = \frac{1+\frac{\zeta(k)}{2}}{1-\frac{\zeta(k)}{2}} \quad\Longrightarrow\quad \frac{b-1}{b+1} = \frac{\zeta(k)}{2},$$

this simplifies to the final sampling formula

$$x = b(k) - (b(k)+1)\left(\frac{\zeta(k)}{2}\right)^{r},$$

for a uniform random variable $r \sim \mathcal{U}(0,1)$.

This value is accepted if $\frac{f(k,x)}{h(k,x)} \geq u_1$, where u_1 denotes a uniform random variable. If the value is not accepted, a new value is sampled until acceptance occurs. The scattering angle θ is then obtained by $\theta = \arccos(x)$.

Once the scattering angle θ is sampled, the new energy of the scattered photon E' can be calculated using the Compton formula:

$$E' = \frac{E}{1 + \frac{E}{m_e c^2}(1 - \cos\theta)} \tag{10.4}$$

Given the scattering angle θ and an azimuthal angle $\phi = u_2 \cdot 2\pi$ for $u_2 \in [0,1]$ denoting a second random value, the new direction $\vec{\omega}'$ can be obtained by rotating the initial direction $\vec{\omega}$ accordingly. This is achieved by constructing a local orthonormal basis $\{\vec{e}_1, \vec{e}_2, \vec{\omega}\}$ and expressing the scattered direction as

$$\vec{\omega}' = \cos\theta\,\vec{\omega} + \sin\theta\Big(\cos\phi\,\vec{e}_1 + \sin\phi\,\vec{e}_2\Big).$$

10.3.3 Rayleigh Scattering

Rayleigh scattering is an elastic scattering process that occurs at low X-ray energies [20, Chapter 7]. The incident photon interacts coherently with bound electrons in the outer shells, driving a collective oscillation of the electron cloud. The atom then re-radiates a photon of the same energy (wavelength) but generally in a different direction. Because the interaction is elastic, no electrons are ejected and no ionization occurs. At diagnostic energies, the differential cross section is strongly forward-peaked, so most scattered photons are deflected by small angles.

Although Rayleigh scattering takes place in the X-ray tube, it can be excluded from the linear attenuation coefficient, as it does not contribute to the attenuation of the X-ray beam in the same way as Compton scattering or photoelectric absorption. As described in [29], the exclusion of Rayleigh scattering from the linear attenuation coefficient is based on the assumption that the characteristic radiation emitted by the target is isotropic, meaning it is emitted uniformly in all directions. As the X-ray is not attenuated by Rayleigh scattering, it is a common assumption to exclude Rayleigh scattering from the attenuation coefficient.

Therefore, it is justified to neglect Rayleigh scattering in the simulation of X-ray imaging, as it does not significantly affect the overall attenuation of the X-ray beam. This simplification allows for a more efficient simulation without compromising the accuracy of the results.

Chapter 11

The Simulation Algorithm

In this chapter, the algorithms developed for X-ray image simulation are presented. To enhance convergence, they integrate different sampling techniques and aspects of the *Forced Fixed Detection* approach.

In standard MC X-ray simulations a random variable determines whether a photon escapes the phantom or undergoes a scattering event. Thus, only a fraction of the simulated photons actually reaches the detector and contributes to the final image. To attain a satisfactory photon distribution at the detector, a very large number of photons with different directions, energies and scatter behaviors must be simulated. To reduce the variance and accelerate convergence, low-discrepancy sequences are used in this thesis to sample the initial photon properties and the stochastic processes during photon transport.

To compare the performance of conventional MC sampling with the proposed Sobol' and Halton based sampling, a simulation algorithm has been developed, which is based on the *Forced Fixed Detection* approach. Forced fixed detection is a variance reduction technique applied in [14, 17].

Therefore, a fixed number of photon scatter events is predefined. Then, at each scatter event, the escape probability is calculated. This represents the portion of the photon intensity which is not scattered but is directly forced to contribute to the detector signal. For the remaining intensity, a free path length is sampled from the exponential distribution according to Beer-Lambert's law and the photon is scattered. Again, the photon intensity is updated according to the probability of staying in the phantom and being scattered respectively. Thus, every simulated photon definetly contributes to the detector signal, which is not the case in conventional MC simulations.

11.1 Preliminaries

Before delving into the detailed description of the simulation algorithm, some preliminary assumptions are outlined.

First, as outlined in Section 10.3, the phantom is assumed to be surrounded by air. Furthermore, the attenuation of air is neglected. Thus, it is assumed that no attenuation occurs outside the phantom.

The phantom is assumed to be convex, such that a photon, which left the phantom, cannot re-enter it. This simplifies the implementation. However, the algorithm can be extended to handle non-convex phantoms as well. This can be handled either by

applying the ray traversal algorithm (Algorithm 6) multiple times to check whether the photon re-enters the phantom or by setting the attenuation of air to a very small value.

The positions of the photons are represented in spatial coordinates with a specific unit scale. The *voxel size* λ represents the size of a voxel in centimeters and is used for scaling. Thus, a photon with position $P = (x,y,z)$ denoted in centimeters corresponds to the voxel coordinates $P' = (\frac{x}{\lambda}, \frac{y}{\lambda}, \frac{z}{\lambda})$.

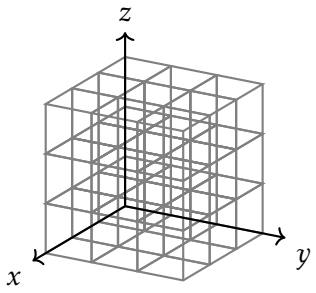
11.2 Algorithm overview

The algorithm begins by initializing a sampling technique (e.g. Monte Carlo, Sobol', Halton), a predefined number of photons, the beam geometry, the phantom geometry and a fixed number of scatter events N.

Then a total of $3(N+1)$ random values $(u_k)_{k=0}^{3(N+1)}$ per photon is sampled according to the sampling technique chosen.

With that in place, a set of photons, each assigned an initial energy E_0 sampled from the X-ray tube spectrum, an initial position S and an initial direction $\vec{\omega}_0$ sampled according to the beam geometry is chosen as described in Section 11.3.1. To sample the initial position, direction and energy of each photon, a total of three random values is required per photon: u_0, u_1, u_2.

The photons are placed within a voxel grid composed of cubic voxels that define the geometry of the scene. Each voxel encodes an integer identifying a specific material or tissue type and density.



Furthermore, each photon is initialized with an intensity of $W_0 = 1$, which is iteratively updated throughout the simulation to account for attenuation and scattering processes.

Next a ray traversal algorithm (Section 11.3.2) is applied to traverse the photon through air until it reaches the phantom. Hereby, the traversed voxels and according traveled distances d_j are captured. Beginning at the entry point of the phantom A_0, attenuation effects are happening and the adapted *Forced Fixed Detection algorithm* is applied.

Therefore, at every scatter point A_i with $i \in \{0,\dots,N-1\}$, the escape probability

$$p(A_i, \vec{\omega}_i, E_i) := \exp\left(- \int_{\overrightarrow{A_i C_i}} \mu(x, E_i) dx \right) \tag{11.1}$$

is computed according to the Beer-Lambert law (Eq. 9.1) depending on the current direction $\vec{\omega}_i$ and photon energy E_i. C_i denotes the exit point of the phantom along the photon direction. μ refers to the attenuation coefficient depending on the position x in the phantom and the photon energy E_i.

With the calculated escape probability, the free path length t_i is sampled attributing the random value u_{3i} while forcing the photon to stay within the phantom. The *forced* aspect of the forced fixed detection algorithm implies that the photon is guaranteed to scatter N times within the phantom before completely exiting the phantom. This is ensured by forcing the next scatter point to be in between the current position A_i and the exit point C_i. To use the uniform variate u_{3i}, a target value is computed according to the Beer-Lambert law (9.1):

$$\exp(-\int_0^{t_i} \mu(A_i + s\cdot\vec{\omega}_i, E_i)ds) = 1 - u_{3i} * (1 - p(A_i, \vec{\omega}_i, E_i)).$$

As the target value needs to be in the range $[p(A_i, \vec{\omega}_i, E_i), 1]$, the right-hand side is constructed as such. Consequently, the following equation is solved for t_i as in Algorithm 8 (line 13-14):

$$\int_0^{t_i} \mu(A_i + s\cdot\vec{\omega}_i, E_i)ds = -\ln(1 - u_{3i} * (1 - p(A_i, \vec{\omega}_i, E_i))). \tag{11.2}$$

Then, the next scatter point is computed as:

$$A_{i+1} = A_i + t_i \cdot \vec{\omega}_i$$

Further, the following two scenarios are considered at each forced scatter point $i \in \{0, \ldots, N\}$. Following the forced fixed detection approach, the photon intensity W_i is split into two parts:

- **Photon escapes the phantom without further scattering:**
  Proportional to the probability of escaping the phantom $p(A_i, \vec{\omega}_i, E_i)$, the intensity and photon energy contributes to the detector signal without further scattering with intensity

  $$W_{i+1}^{\text{escape}} = W_i \cdot p(A_i, \vec{\omega}_i, E_i).$$

  The accounting detector pixel is determined in Algorithm 8 (line 19). In case $i = 0$, this intensity is accounted as primary intensity.

- **Photon stays in the phantom and scatters:**
  The leftover intensity $W_i \cdot (1 - p(A_i, \vec{\omega}_i, E_i))$ splits into two: the absorbed intensity which will no longer be considered and the Compton scattered intensity. This ratio is determined by the ratio of the Compton scattering coefficient μ_{CS}

and the photoelectric coefficient μ_{PE}. Thus, the leftover intensity after Compton scattering is

$$W_{i+1} = W_i \cdot (1 - p(A_i, \vec{\omega}_i, E_i)) \cdot \frac{\mu_{\text{CS}}(A_{i+1}, E_i)}{\mu_{\text{CS}}(A_{i+1}, E_i) + \mu_{\text{PE}}(A_{i+1}, E_i)}.$$

Within Algorithm 8 (line 18), the new photon energy E_{i+1} and direction $\vec{\omega}_{i+1}$ after the Compton scatter event is sampled with two more random variables u_{3i+1} and u_{3i+2}.

This process is repeated according to the maximum scatter order N. The leftover intensity W_N after the last forced scatter event is sent to the detector without any further scattering.

An overview of the different rays and their paths is illustrated in Figure 11.1.

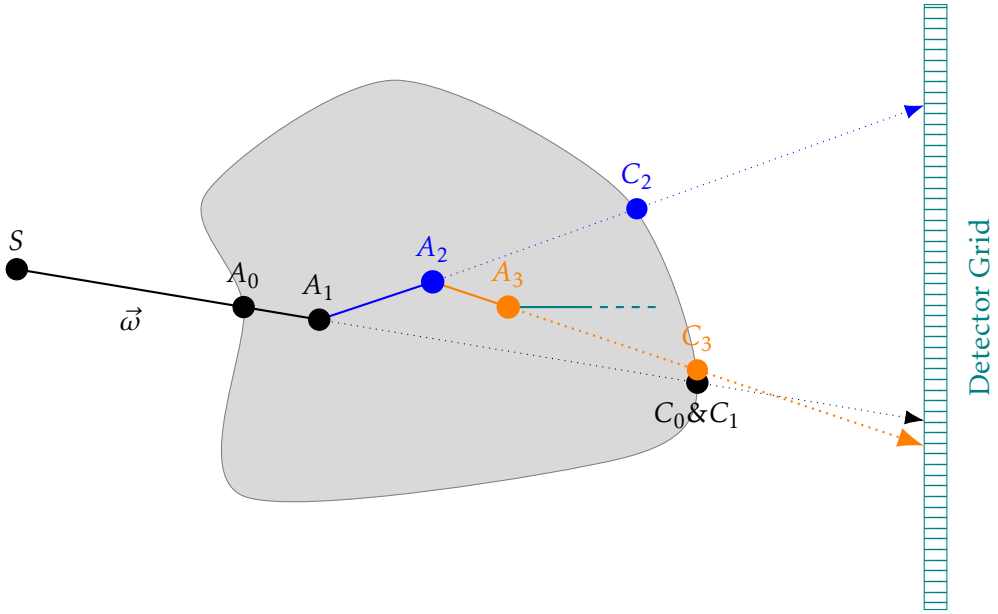


FIGURE 11.1: Illustration of the ray paths of a photon. The photon is emitted at the position S with direction $\vec{\omega}$ and scattered at A_1, A_2 and A_3.

11.3 Sub-Algorithms

To model the relevant physical processes in a structured manner, the main simulation is decomposed into several sub-algorithms. This section describes the purpose and implementation of each sub-algorithm in detail.

11.3.1 Photon Generation

To initialize a photon, the three properties — energy, position and direction — must be sampled. The photon energy is sampled independently from the position and direction as the spectral distribution accounts for the whole beam angle. For the generation, a total of three random variables $u_0, u_1, u_2 \in [0, 1]$ are applied per photon.

Photon Energy Sampling

To sample the photon energy, the inverse transform sampling method is applied as described in Section 10.2.1. To obtain the inverse function of the CDF, the following algorithm is adopted:

Algorithm 2 Inverse-Determination of the CDF

Require: Strictly increasing energy nodes $E[1..n]$
Require: Nonnegative fluence samples $F[1..n]$ (not all zero)
Ensure: Inverse CDF represented by nodes $E[0..n]$ and values $\mathrm{CDF}[0..n]$
1: $T \leftarrow \sum_{i=1}^{n} F[i]$ // Normalize
2: **for** $i = 1..n$ **do** $\mathrm{PDF}[i] \leftarrow F[i]/T$
3: $\mathrm{CDF}[1] \leftarrow \mathrm{PDF}[1]$
4: **for** $i = 2..n$ **do** $\mathrm{CDF}[i] \leftarrow \mathrm{CDF}[i-1] + \mathrm{PDF}[i]$
5: **(Guard)** set $\mathrm{CDF}[0] \leftarrow 0$; $E[0] \leftarrow E[1]$; enforce $\mathrm{CDF}[n] = 1$;
6: **return** $(E[0..n], \mathrm{CDF}[0..n])$

This algorithm constructs the CDF and returns two arrays: one with the energy nodes and one with the corresponding CDF values. To determine the photon energy $\mathcal{E}$ according to random variable u_0, a loop is applied to find the index j such that $\mathrm{CDF}[j-1] \leq u_0 < \mathrm{CDF}[j]$. The corresponding energy is then $\mathcal{E} = E[j]$.

Photon Position and Direction Sampling: Point Source Model

For the point source model, the initial position S of each photon is fixed at the source position $\mathcal{S}^*$. The direction $\vec{\omega}$ is sampled uniformly within a cone defined by the beam axis $\vec{\mathbf{a}}$ and opening angle Θ. This requires two independent random variables $u_1, u_2 \in [0,1]$.

The following algorithm is applying the method as described in Section 10.2.2 and samples the direction $\vec{\omega}$ for a beam with half-opening angle Θ in cone axis $(0,0,1)$ For general applicability, the algorithm is projecting the cone axis for a given direction $\vec{\mathbf{a}} \in \mathbb{R}^3$.

Algorithm 3 Photon Direction Sampling for a Point Source

Require: Unit cone axis $\hat{\mathbf{a}}$
Require: Half-opening angle Θ
Require: Random samples $u_1, u_2 \sim \mathcal{U}(0,1)$
Ensure: Emission direction $\vec{\omega}$
1: $\phi \leftarrow 2\pi u_2$
2: $\cos\theta \leftarrow 1 - u_1(1 - \cos\Theta)$
3: $\theta \leftarrow \arccos(\cos\theta)$
4: $\vec{w} \leftarrow (\sin\theta\cos\phi,\ \sin\theta\sin\phi,\ \cos\theta)$ // local sample on cap
5: Choose $\vec{t} = (0,0,1)$ if $|a_3| < 0.999$, else $\vec{t} = (1,0,0)$
6: $\hat{\mathbf{u}} \leftarrow \dfrac{\vec{t} \times \hat{\mathbf{a}}}{\|\vec{t} \times \hat{\mathbf{a}}\|}, \quad \hat{\mathbf{v}} \leftarrow \hat{\mathbf{a}} \times \hat{\mathbf{u}}$
7: $\vec{\omega} \leftarrow w_x\hat{\mathbf{u}} + w_y\hat{\mathbf{v}} + w_z\hat{\mathbf{a}}$
8: **return** $(\vec{\omega})$

Whereas the algorithm yields the direction $\vec{\omega}$, the start position is simply set to be the source position: $S = \mathcal{S}^*$

Photon Position Sampling for an Extended Source

For the extended source model, the four edge points of the rectangle source within a plane P must be given: $P_1, P_2, P_3, P_4 \in \mathbb{R}^3$. The initial position S of each photon is sampled by the simple calculation from section 10.2.2. Again two random values $u_1, u_2 \in [0,1]$ are required.

Algorithm 4 Photon Position Sampling for an Extended Source

Require: Rectangle corner points $P_1, P_2, P_3, P_4 \in \mathbb{R}^3$
Require: Random samples $u_1, u_2 \sim \mathcal{U}(0,1)$
Ensure: Emission position S
1: $\vec{v}_1 \leftarrow P_2 - P_1$
2: $\vec{v}_2 \leftarrow P_4 - P_1$
3: $S \leftarrow P_1 + u_1\vec{v}_1 + u_2\vec{v}_2$
4: **return** (S)

For the extended source modeling, the photon direction is set to be the direction normal to the plane P.

11.3.2 Ray Traversal

The ray traversal algorithm is used for traversing the photon through the voxel grid until either a certain material is reached or the photon exits the grid. The algorithm stops when a specified material is reached in the voxel grid. Firstly, if the photon starts in air the algorithm is used to project the photon until it reaches the phantom. Therefore, it consistently checks whether the next voxel is still air or not. Secondly, the algorithm is used to traverse the photon through the phantom to determine the exit point C of the phantom. Here, the algorithm checks whether the next voxel is still a material that is not air. When exiting the phantom, the algorithm is applied to traverse air until it exits the voxel grid and reaches the detector plane. Here, the algorithm is applied as long as the next voxel material is air. As the phantom is assumed to be convex, the photon must exit the grid after leaving the phantom and its projection through air.

The last two applications of the ray traversal algorithm are used within the *Forced Fixed Detection* algorithm to traverse the photon through the phantom and onto the detector. It is important to record the traversed voxel indices with the corresponding distances, to proportionally consider the attenuation effects.

To initialize the ray-traversal algorithm, the following inputs are required:

- The voxel grid $materialGrid \in \mathbb{Z}^{X\times Y\times Z}$ representing the materials of the scene including the phantom. X, Y, Z represent the shape of the grid.
- The voxel size $\lambda \in \mathbb{R}^+$.
- The initial position of the photon $A \in \mathbb{R}^3$.
- The (unit) direction of the photon $\vec{\omega} \in \mathbb{R}^3$.

Traversing the ray in the grid means traversing the ray through the cubic voxels. For the determination of the attenuation along the path, the precisely determined per-voxel distances, the voxel indices and their corresponding materials must be tracked. For quickly determining the sampled free path length, the starting points in each voxel are also captured.

This algorithm proceeds voxel by voxel. To carry out the ray traversal within a specific voxel, the component Algorithm 5 is applied at each voxel along the ray.

Algorithm 5 Ray Traversal Algorithm: Step

Require: Current position in voxel grid coordinates $\tilde{A}$
Require: Current unit direction $\vec{\omega} = (\omega_x, \omega_y, \omega_z)$
Ensure: Distance to next voxel boundary $maxDist$
Ensure: Updated position $\tilde{A}'$
1: **for** $i \in \{x, y, z\}$ **do**
2: **if** $\omega_i > 0$ **then**
3: $maxDist_i \leftarrow \frac{1 - (\tilde{A}_i - \lfloor \tilde{A}_i \rfloor)}{\omega_i}$
4: **else if** $\omega_i < 0$ and $\lfloor \tilde{A}_i \rfloor \neq \tilde{A}_i$ **then**
5: $maxDist_i \leftarrow -\frac{\tilde{A}_i - \lfloor \tilde{A}_i \rfloor}{\omega_i}$
6: **else if** $\omega_i < 0$ and $\lfloor \tilde{A}_i \rfloor = \tilde{A}_i$ **then**
7: $maxDist_i \leftarrow -\frac{1}{\omega_i}$
8: **end if**
9: **end for**
10: $distance \leftarrow \min(maxDist_x, maxDist_y, maxDist_z)$
11: $\tilde{A}' \leftarrow \tilde{A} + \vec{\omega} \cdot distance$
12: **return** $maxDist, \tilde{A}'$

Algorithm 5 computes for each component x, y, z the distance to the next voxel boundary depending on the direction of the ray. To justify the correct sign, it distinguishes between the following cases:

- When the direction component is positive (line 2), the offset to the next boundary is between the current position and the next voxel index, i.e. $1 - (\tilde{A}_i - \lfloor \tilde{A}_i \rfloor)$.
- When the direction component is negative and the current position is not at the voxel boundary (line 4), the offset to the next boundary is between the current position and the previous voxel index, i.e. $\tilde{A}_i - \lfloor \tilde{A}_i \rfloor$. As the direction is negative, the offset to the next boundary is negated.
- When the direction component is negative and the current position is at the voxel boundary (line 6), the offset to the next boundary is between the current position and the previous voxel index, i.e. for the grid 1. As the direction is negative, the offset to the next boundary is negated.

It then normalizeseach offset by the respective component of the direction vector. The minimum of these three distances in the direction of the ray is the distance to the next voxel boundary. The position is then updated accordingly.

The main ray traversal algorithm (Algorithm 6) applies Algorithm 5 iteratively until the stopping criterion is met. Whereas the stopping criterion in Algorithm 6 is solely running as long as the current material is not air, the algorithm can be easily adapted to stop at any other material and therefore be applied in the different scenarios described above.

Further, the algorithm converts the position A in world coordinates to voxel coordinates $\tilde{A}$ and vice versa. It captures the crossed voxel indices, their corresponding materials, the entry points in world coordinates and the per-voxel distances in centimeters in the arrays $crossedVoxels$, $crossedMaterials$, $entryPoints$ and $distances$ respectively.

The algorithm returns the following values:

- The array $crossedVoxels$ contains all voxel indices of voxels that the ray traversed.
- The array $crossedMaterials$ contains the corresponding materials of the crossed voxels.
- The array $entryPoints$ contains the starting points in the corresponding voxels in world coordinates.
- The array $distances$ contains the distances in centimeters such that their unit is consistent with the attenuation coefficients.
- The exit point C in world coordinates where the ray exits the phantom or the voxel grid.

Algorithm 6 Ray Traversal: Main Algorithm

Require: Material grid $materialGrid \in \mathbb{Z}^{X \times Y \times Z}$, voxel size $\lambda \in \mathbb{R}^+$
Require: Initial position $A \in \mathbb{R}^3$, unit direction $\vec{\omega} \in \mathbb{R}^3$
Ensure: Final position $C \in \mathbb{R}^3$
Ensure: Arrays $crossedVoxels, crossedMaterials, entryPoints, distances$
Ensure: Boolean $exitGrid$ indicating whether the photon exited the grid

Initialize empty arrays $crossedVoxels, crossedMaterials, entryPoints, distances$
$\tilde{A} \leftarrow A / \lambda$ // Convert to voxel grid coordinates
$currVoxelIdx \leftarrow \lfloor \tilde{A} \rfloor$
$exitGrid \leftarrow$ false
if any$(\tilde{A} < 0)$ or any$(\tilde{A} \geq \text{shape}(materialGrid))$ **then** // Check if outside grid
    $exitGrid \leftarrow$ true
    return $C = A$, $crossedVoxels$, $crossedMaterials$, $entryPoints$, $distances$, $exitGrid$
end if
$currMaterial \leftarrow materialGrid[\, currVoxelIdx \,]$
while $currMaterial$ is not air **do**
    Apply Algorithm 5 to get $maxDist$ and $\tilde{A}$
    $distance \leftarrow maxDist \cdot \lambda$ // Convert distance to world units in cm
    Update arrays $\langle crossedVoxels, crossedMaterials, entryPoints, distances \rangle$ with $(currVoxelIdx, currMaterial, A, distance)$
    $currVoxelIdx \leftarrow \lfloor \tilde{A} \rfloor$
    $A \leftarrow A + \vec{\omega} \cdot distance$ // Update position in world coordinates
    if any$(\tilde{A} < 0)$ or any$(\tilde{A} \geq \text{shape}(materialGrid))$ **then**
        $exitGrid \leftarrow$ true
        return $C = A$, $crossedVoxels$, $crossedMaterials$, $entryPoints$, $distances$, $exitGrid$
    end if
    $currMaterial \leftarrow materialGrid[\, currVoxelIdx \,]$
end while
return $C = A$, $crossedVoxels$, $crossedMaterials$, $entryPoints$, $distances$, $exitGrid$

11.3.3 Compton Scattering

The Compton scattering algorithm strictly follows the description in Section 10.3.2. The algorithm implements the Klein–Nishina formula (Eq. 10.2) to sample the scattering angle θ with the acceptance–rejection method. The energy update is applied with the Compton formula (Eq. 10.1). Finally, the new direction $\vec{\omega}'$ is computed with the sampled angles θ and azimuthal angle ϕ.

The algorithm requires the following inputs:

- The initial photon energy $E \in \mathbb{R}$.
- The initial (unit) direction of the photon $\vec{\omega} \in \mathbb{R}^3$.
- Two random variables $u_1, u_2 \in [0, 1]$.

As described in Section 10.3.2, the algorithm samples the scattering angle θ with the acceptance–rejection method.

Algorithm 7 Compton Scattering (Klein–Nishina + Acceptance–Rejection)

Require: Initial photon energy E, unit direction $\vec{\omega}$
Require: Random inputs $u_1, u_2 \in [0,1]$ (reused until acceptance)
Ensure: Scattered energy E' and unit direction $\vec{\omega}'$

$E_{\text{rest}} \leftarrow 0.511\,\text{MeV}$ // Initialize electron rest energy
$k \leftarrow E / E_{\text{rest}}$
Define $f(k,x) \leftarrow \frac{1}{(1+k(1-x))^2}\left(1 + x^2 + \frac{k^2(1-x)^2}{1+k(1-x)}\right)$ // Klein–Nishina formula (Eq. 10.2)
$\zeta(k) \leftarrow \frac{2(2k^2+2k+1)}{(2k+1)^3}$
$b(k) \leftarrow \frac{1+\zeta/2}{1-\zeta/2}, \quad a(k) \leftarrow 2\,(b-1)$
Define $h(k,x) \leftarrow \frac{a}{b-x}$ // Envelope on $x \in [-1,1]$
repeat // Acceptance–Rejection Sampling loop
    Draw $r \sim \mathcal{U}(0,1)$
    $x \leftarrow b - (b+1)\left(\frac{\zeta}{2}\right)^r$ // Sample $x = \cos\theta$ from $h(k,x)$
    $\text{acc} \leftarrow \frac{f(k,x)}{h(k,x)}$
until $u_1 \le \text{acc}$
$\theta \leftarrow \arccos(x)$
$E' \leftarrow \frac{E}{1+k(1-\cos\theta)}$ // Energy update (Compton formula (10.4))
$\phi \leftarrow 2\pi u_2$ // Azimuthal angle
if $|\omega_z| < 0.999$ **then**
    $\vec{v} \leftarrow (0,0,1)$
else
    $\vec{v} \leftarrow (1,0,0)$
end if
$\vec{e}_1 \leftarrow \frac{\vec{v}\times\vec{\omega}}{\|\vec{v}\times\vec{\omega}\|}, \quad \vec{e}_2 \leftarrow \vec{\omega}\times\vec{e}_1$ // Orthonormal basis
$\vec{\omega}' \leftarrow \cos\theta\,\vec{\omega} + \sin\theta\left(\cos\phi\,\vec{e}_1 + \sin\phi\,\vec{e}_2\right)$
return E', $\vec{\omega}'$

11.3.4 Forced Fixed Detection

The forced fixed detection algorithm is applied at each forced scatter point. Here, the photon intensity distributes into multiple paths:

- **Primary intensity:** The part of the photon intensity that is directly forwarded to the boundary of the grid without further scattering and eventually contributes to the detector signal.

$$W \cdot \int \mu(x,E)$$

- **Absorption:** The part of the photon intensity that undergoes a photoelectric absorption at the forced scatter point and does not contribute to the detector

signal.

$$W \cdot \left(1 - \int \mu(x,E)\right) \cdot \frac{\mu_{\text{PE}}}{\mu_{\text{PE}} + \mu_{\text{CS}}}$$

- **Compton scattering:** The part of the photon intensity that undergoes Compton scattering at the forced scatter point and continues its path with updated energy and direction.

$$W \cdot \left(1 - \int \mu(x,E)\right) \cdot \frac{\mu_{\text{CS}}}{\mu_{\text{PE}} + \mu_{\text{CS}}}$$

The implementation of the algorithm requires the following inputs:

- The voxel grid $materialGrid \in \mathbb{Z}^{X \times Y \times Z}$ representing the materials of the scene including the phantom. X, Y, Z represent the shape of the grid.
- The voxel size $\lambda \in \mathbb{R}^{+}$.
- The initial position $A_0 \in \mathbb{R}^3$, (unit) direction $\vec{\omega}_0 \in \mathbb{R}^3$ and energy $E_0 \in \mathbb{R}$ of the photon

For explanatory purposes, the algorithm is split into two parts: the step algorithm (Algorithm 8) where one forced scatter event is processed and the main algorithm (Algorithm 9) where the step algorithm is applied iteratively for N forced scatter events.

Algorithm 8 Forced Fixed Detection: Step

Require: Material grid $materialGrid \in \mathbb{Z}^{X\times Y\times Z}$, voxel size $\lambda \in \mathbb{R}^+$
Require: Initial position $A \in \mathbb{R}^3$, unit direction $\vec{\omega} \in \mathbb{R}^3$, energy E and intensity W of the photon
Require: Random values $u_1, u_2, u_3 \in [0,1]$
Require: Attenuation functions $\mu(x,E), \mu_{\text{CS}}(x,E), \mu_{\text{PE}}(x,E)$ for all materials
Ensure: New position A', direction $\vec{\omega}'$, energy E' and intensity W' of the scattered photon
Ensure: Escaped position $\hat{C}$ and intensity $escapeIntensity$ of the escaped proportion

1: Apply Algorithm 6 to traverse the ray from point A in direction $\vec{\omega}$ through the phantom and obtain exit position C and arrays $materials$, $distances$
2: Initialize empty array $atts$
3: $totalAtt \leftarrow 0$
4: **for** $j \leftarrow 0$ to $len(materials) - 1$ **do**
5: $\quad material \leftarrow materials[j]$
6: $\quad distance \leftarrow distances[j]$
7: $\quad voxelAtt \leftarrow \mu(\cdot,E) \cdot distances[j]$ // $\mu(\cdot,E)$ according to material in voxel
8: $\quad atts[j] \leftarrow voxelAtt$
9: $\quad totalAtt \leftarrow totalAtt + voxelAtt$
10: **end for**
11: $escapeProb \leftarrow \exp(-totalAtt)$
12: $attGoal \leftarrow -\ln(1 - u_1 \cdot (1 - escapeProb))$
13: iteratively find J such that $\sum_{j=0}^{J} atts[j] \leq attGoal < \sum_{j=0}^{J+1} atts[j]$
14: $goalDist \leftarrow \sum_{j=0}^{J} distances[j] + \frac{attGoal - \sum_{j=0}^{J} atts[j]}{\mu(\cdot,E)}$ // distance to next scatter point
// $\mu(\cdot,E)$ according to material in voxel $J+1$
15: $A' \leftarrow A + goalDist \cdot \vec{\omega}$
16: $escapeIntensity \leftarrow W \cdot escapeProb \cdot E$
17: $W' \leftarrow W \cdot (1 - escapeProb) \cdot \frac{\mu_{\text{CS}}}{\mu_{\text{CS}} + \mu_{\text{PE}}}$
18: Apply Algorithm 7 for Compton Scattering with direction $\vec{\omega}$, energy E and random variables u_2, u_3 to obtain new direction $\vec{\omega}'$ and energy E'
19: Apply Algorithm 6 to traverse the ray from exit point C of the phantom through air until it exits the grid and obtain the detector position $\hat{C}$
20: **return** $A', W', E', \vec{\omega}'$ for the scattered photon, $escapeIntensity, \hat{C}$ for the escaped photon

The algorithm applies the Beer-Lambert law (Eq. 9.1) as described in Section 11.2 to compute the escape probability of the photon. Further, the probability of Compton scattering is implemented with an update on the intensity of the scattered photon. This yields a physically accurate simulation by proportionally considering the attenuation effects and the ratio of Compton scattering and photoelectric absorption.

For sampling the free path length until the next forced scatter point, equation (11.2) is applied. The integral

$$\int_0^t \mu(A_i + s\vec{\omega}_i, E_i) ds$$

is given by the cumulative sum of the attenuation coefficients $\mu(\cdot, E_i)$ along the ray multiplied by the corresponding distances within each voxel. Here, the algorithm iteratively finds the index J such that the cumulative sum of the attenuation coefficients up to index J is less than or equal to the target attenuation $attGoal$ as described in line 13 of Algorithm 8.

Algorithm 9 Forced Fixed Detection: Main Algorithm

Require: Material grid $materialGrid \in \mathbb{Z}^{X\times Y\times Z}$, voxel size $\lambda \in \mathbb{R}^+$
Require: Initial position $A_0 \in \mathbb{R}^3$, unit direction $\vec{\omega}_0 \in \mathbb{R}^3$, energy E_0 of the photon
Require: Number of forced scatter events N
Require: $3N$ Random values $u_0, \dots u_{3N-1} \in [0,1]$
Require: Attenuation functions $\mu(x,E), \mu_{\text{CS}}(x,E), \mu_{\text{PE}}(x,E)$ for all materials
Ensure: Arrays $exitPoss^*, exitInts^*$ for primary intensities
Ensure: Arrays $exitPoss, exitInts$ for scattered intensities
1: Initialize empty arrays $exitPoss^*, exitInts^*$ // for primary intensities
2: Initialize empty arrays $exitPoss, exitInts$
3: $W_0 \leftarrow 1$ // Initial intensity of the photon
4: **for** $i \leftarrow 0$ to $N-1$ **do**
5: Apply Algorithm 8 with inputs $A_i, \vec{\omega}_i, E_i, W_i, u_{3i}, u_{3i+1}, u_{3i+2}$ to obtain $A_{i+1}, W_{i+1}, E_{i+1}, \vec{\omega}_{i+1}$ for the scattered photon and $W_{i+1}^{\text{escape}}, C_i$ for the escaped photon
6: **if** $i = 0$ **then**
7: Append $C_i, W_{i+1}^{\text{escape}}$ to $exitPoss^*, exitInts^*$ // primary intensity
8: **else**
9: Append $C_i, W_{i+1}^{\text{escape}}$ to $exitPoss, exitInts$ // scattered intensity
10: **end if**
11: **end for**
12: Apply Algorithm 6 to traverse the photon from A_N with direction $\vec{\omega}_N$ through the phantom and obtain exit position C_N and arrays: $materials$, $distances$
13: Initialize array $atts$ with the attenuation coefficients $\mu(\cdot, E_N)$ according to material in $materials$
14: $lastInt \leftarrow W_N \cdot E_N \cdot \exp(-\langle distances, atts\rangle)$ // final escaping intensity
15: Append $C_N, lastInt$ to $exitPoss, exitInts$
16: **return** $exitPoss^*, exitInts^*$ for primary intensities, $exitPoss, exitInts$ for scattered intensities

The forced fixed detection algorithm then yields the exit positions and according intensities for both the primary and scattered photons.

11.4 Algorithm Composition

For the implementation of the Photon Simulation Algorithm, the sub-algorithms from Section 11.3 are composed.

The algorithm is initialized with the following parameters:

- The voxel grid $materialGrid \in \mathbb{Z}^{X\times Y\times Z}$ representing the materials of the scene including the phantom. X, Y, Z represent the shape of the grid.
- The voxel size $\lambda \in \mathbb{R}^+$.

- The sampling strategy: Monte Carlo / Halton / Sobol'.
- The source model: Point source / Extended source.
- The number of forced scatter events N.

The algorithm begins with initializing the random values according to the chosen sampling strategy. Then, the photon is initialized according to the source model.

As described in Section 11.3.1, the photon initialization is applied by sampling the photon energy with Algorithm 2 and evaluating the resulting function with the first random variable u_0. Then, depending on the source model either Algorithm 3 or Algorithm 4 is applied to sample the initial direction or position of the photon respectively and set the other parameter accordingly. Here, the random variables u_1 and u_2 are applied.

Then the photon is traversed through air to the phantom with Algorithm 6 to obtain the initial position A_0 in the phantom. Finally, the forced fixed detection algorithm (Algorithm 9) is applied with N scatter events, the photon parameters and the remaining random values to obtain the arrays for primary intensities $exitPoss^*, exitInts^*$ and for scattered intensities $exitPoss, exitInts$.

Algorithm 10 Photon Simulation Algorithm

Require: Material grid $materialGrid \in \mathbb{Z}^{X\times Y\times Z}$, voxel size $\lambda \in \mathbb{R}^+$
Require: Sampling strategy: Monte Carlo / Halton / Sobol'
Require: Number of scatter events N
1: Initialize random numbers $(u_i)_{i=0}^{3N+2} \in [0,1]^{3(N+1)}$ according to the sampling strategy
2: Initialize photon with position S, energy E_0, direction $\vec{\omega}_0$ according to the source model and the corresponding Algorithm 2-4 using three random values u_0, u_1, u_2 as described in Section 11.3.1
3: Traverse photon through air to phantom with Algorithm 6 to obtain initial position A_0 in phantom
4: Apply Algorithm 9 given N scatter events, the photon parameters and random values $(u_i)_{i=3}^{3N+2}$ to obtain the arrays $exitPoss^*, exitInts^*$ for primary intensities and $exitPoss, exitInts$ for scattered intensities
5: **return** $exitPoss^*, exitInts^*$ for primary intensities, $exitPoss, exitInts$ for scattered intensities

Algorithm 10 simulates the scattering of a single photon. In practice, the algorithm is applied for a large number of photons to obtain statistically significant results that are relevant for real-world applications such as CT imaging. In the implementation for this thesis, the algorithm is further vectorized to simulate the scattering of multiple photons in parallel and speed up the computation.

Chapter 12

Evaluation of the QMC X-ray Simulation Framework

This final chapter provides an evaluation of the simulation framework developed in Chapter 10 and Chapter 11. The focus is on assessing the impact of different sampling strategies — classical Monte Carlo and the Quasi-Monte Carlo methods based on Halton and Sobol' low-discrepancy sequences — on the quality of simulated X-ray images. The chapter first outlines the experimental setup, then introduces the evaluation metrics and finally, presents the evaluation of both primary and scattered intensities. A concluding section summarizes the findings and highlights promising directions for future research.

12.1 Experimental Setup

All evaluations are carried out using the simulation framework introduced in Chapter 10. The experiments are configured via YAML-based setup files for reproducibility. The YAML-based setup is included so readers can reproduce the experiments. Due to the stochastic nature of the MC method, results may vary between runs.

For the evaluation, two main scene setups are considered: To determine how evenly distributed the simulated photons are, a homogeneous phantom is used. To evaluate the accuracy of scatter estimation, a phantom with an embedded high-density object is employed. The following sections detail the configuration of these setups.

12.1.1 Homogeneous Water–Wall Phantom

The first evaluation setup consists of a homogeneous water–wall phantom, which is used to assess the uniformity of primary and scattered intensities across the detector. The configuration setup is described in the following.

Phantom

The following configuration sets up the water–wall phantom:

```
PHANTOM:
    object: WaterWall
    shape: [200, 200, 1200]
    wall_thickness: 7.0
    voxel_size: 0.1
    detector_distance: 1.0
```

The phantom object is a `WaterWall`. The `shape: [200, 200, 1200]` specifies the *number of voxels* along (x, y, z). With cubic voxels of size `voxel_size` $= 0.1\,\text{cm} = 1\,\text{mm}$, the physical dimensions of the voxel grid are $200\,\text{mm} \times 200\,\text{mm} \times 1200\,\text{mm}$ (i.e., $20\,\text{cm} \times 20\,\text{cm} \times 120\,\text{cm}$). The wall thickness is set to `wall_thickness` $= 7.0\,\text{cm}$, such that the relevant beam–phantom interaction corresponds to a water layer of 7 cm. The detector is placed directly behind the phantom, with `detector_distance` $= 1.0$ corresponding to a 1 cm gap between the water–wall and the detector plane.

A visualization of the scene setup is shown in Figure 12.1.

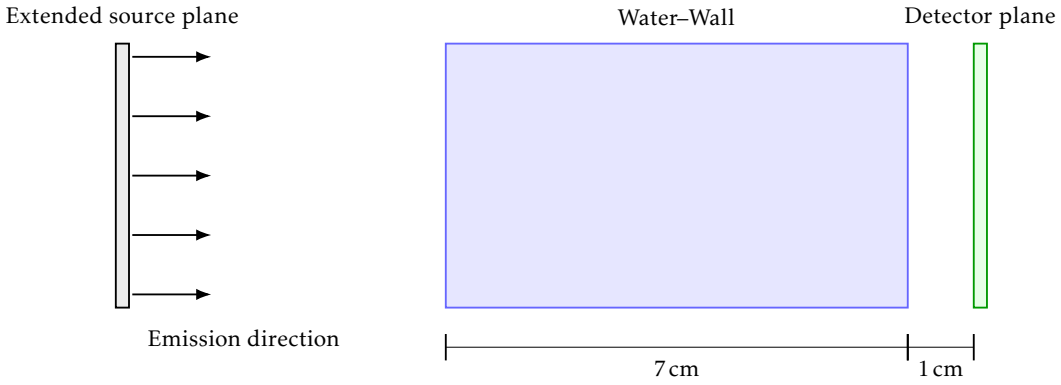


Figure 12.1: Schematic setup of the water–wall scene.

Photon Sampling

The photon generation and transport are configured as follows:

```
SAMPLER:
    sampling_strategy: "Halton"
    num_photons: 3e6
    num_scatters: 20
    batch_size: 1e5
```

Here, `sampling_strategy` determines the type of sequence used to generate random numbers. The options considered are `"MC"` (pseudo-random Monte Carlo (MC)), `"Halton"` (Halton low-discrepancy sequence) and `"Sobol"` (Sobol' low-discrepancy sequence). The total number of simulated photons `num_photons` is varied between 0.5×10^6 and 5×10^6 in steps of 0.5×10^6.

The parameter `num_scatters` defines the maximum number of scattering events per photon for the forced fixed detection algorithm (see Chapter 11). The `batch_size` controls the number of photons processed per iteration in the vectorized setup of the algorithms. The setup of this value depends on the hardware used (here set to 10^5). The photons are emitted from an extended quadratic source that spans across the cross-section of the $200 \times 200 \times 1200$ scene. The emission direction is set to be perpendicular to the plane. This corresponds to the *extended source model* as described in Section 10.2.2.

This setup allows a systematic comparison of the three sampling strategies under increasing photon counts, while keeping the geometry and transport parameters fixed.

12.1.2 Bone-in-Water Phantom

The second evaluation setup consists of a water–wall phantom with embedded bone structures. This configuration is used to assess the robustness of scatter estimation in the presence of high-density inclusions, which introduce stronger attenuation and more complex scattering behavior compared to the homogeneous case. The configuration is as follows:

Phantom

The following setup configuration is used to set up the bone-in-water phantom:

```
PHANTOM:
    object: BoneWaterWall
    shape: [200, 200, 1200]
    wall_thickness: 7.0
    voxel_size: 0.1
    detector_distance: 1.0

    n_bones: 3
    wall_material_id: 1
    bone_material_id: 2
```

The phantom object is a `BoneWaterWall`. The entry `shape: [200, 200, 1200]` specifies the *number of voxels* along (x,y,z). With cubic voxels of size `voxel_size` $= 0.1\,\text{cm} = 1\,\text{mm}$, the physical dimensions of the voxel grid are $200\,\text{mm} \times 200\,\text{mm} \times 1200\,\text{mm}$ (i.e., $20\,\text{cm} \times 20\,\text{cm} \times 120\,\text{cm}$). The wall thickness is again set to `wall_thickness` $= 7.0\,\text{cm}$, ensuring comparability with the homogeneous water–wall setup. The detector is placed directly behind the phantom, with `detector_distance` $= 1.0$ corresponding to a 1 cm gap between phantom and detector plane.

In contrast to the homogeneous water–wall, this phantom includes `n_bones` $= 3$ bone structures embedded within the water–wall. The bones cylinders lie perpendicular to the detector grid as depicted in Figure 12.2. The parameter `wall_material_id` $= 1$ specifies water as the background material, while `bone_material_id` $= 2$ assigns bone as the material for the inclusions. The material ID not only references a material, but also its typical density, which is used to compute the attenuation coefficients. This setup introduces regions of high attenuation and anisotropic scattering, thereby providing a more challenging and realistic test for the evaluation of QMC methods in scatter estimation.

Photon Sampling

The photon generation and transport are configured mostly identically to the homogeneous water–wall setup. The only difference is that instead of the extended source model, the point source model is used here, where the source point is set to be at the center of the opposite side of the detector. This corresponds to a typical cone-beam CT setup.

Figure 12.2 illustrates the scene setup with the point source emitting cone-beam rays through the water–wall containing bone cylinders.

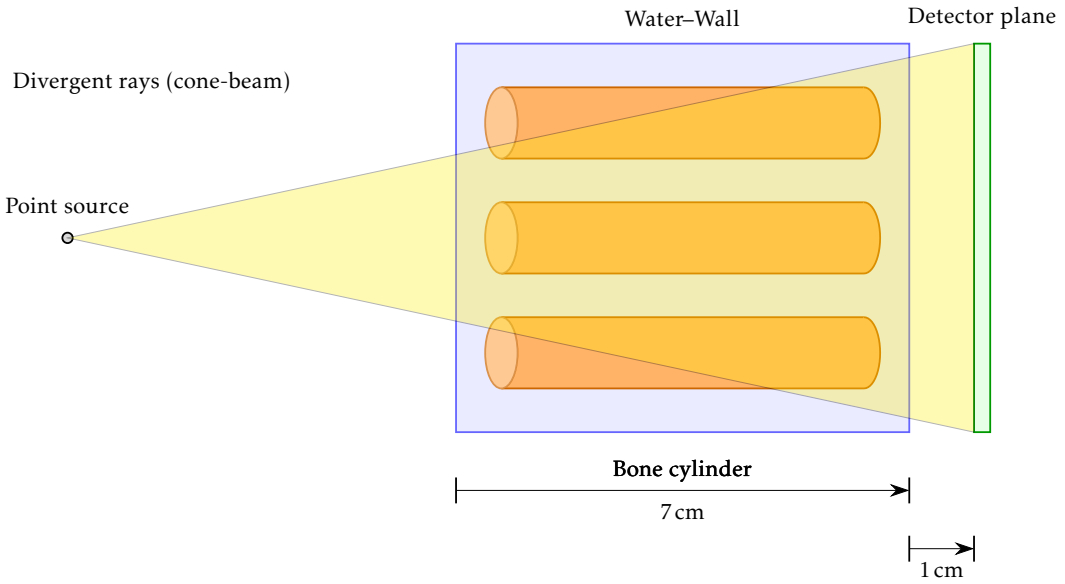


FIGURE 12.2: Point source emitting cone-beam rays through a water–wall containing bone cylinders.

12.2 Evaluation Metrics

In Monte Carlo simulations, noise is not merely a by-product but rather an inherent feature and, in the case of scatter estimation, the very reason for carrying out the simulation. The stochastic nature of photon transport allows one to approximate the distribution of scattered photons on the detector plane with increasing accuracy as the number of simulated photons grows. Since no analytical reference solution is available and within the scope of this work it was not feasible to generate sufficiently resolved reference simulations for the same phantoms, a direct error estimate by comparison to a ground truth could not be obtained.

Instead, alternative metrics were employed to assess the quality of the simulations. A particularly relevant metric is the *variance*, which is illustrated by means of the *water–wall* phantom (see Chapter 12.1.1). Due to the homogeneity of this phantom, one expects the intensity of scattered photons to be uniformly distributed across all detector pixels. Deviations from this expectation manifest themselves as increased variance in the pixel intensities. Variance therefore serves as a suitable measure in this context, since it admits a well-defined reference value and provides a quantitative assessment of departures from the ideal uniform distribution:

Definition 12.2.1 (Variance in discrete detector images)**.**
Let $I \in \mathbb{R}^{m \times n}$ denote a discrete detector image with pixel intensities $I_{i,j}$. The *variance* of the image is defined as

$$\mathrm{Var}(I) = \frac{1}{mn} \sum_{i=1}^{m} \sum_{j=1}^{n} \left(I_{i,j} - \bar{I}\right)^2,$$

where $\bar{I}$ denotes the mean intensity over all pixels.

In addition to the global variance, another important measure is obtained from analyzing one-dimensional intensity profiles across the detector. In the case of the water–wall phantom, the expected profile is smooth and uniform. To quantify deviations from this behavior, the *total variation* of the profile is computed and used as an additional evaluation metric. While variance captures global deviations from uniformity, total variation measures local oscillations and irregularities along the profile.

Definition 12.2.2 (Total variation of a discrete profile)**.**
Let $x = (x_1, \ldots, x_n) \in \mathbb{R}^n$ denote a one-dimensional intensity profile extracted from the detector. The *total variation* of x is defined as

$$\mathrm{TV}(x) = \sum_{i=1}^{n-1} |x_{i+1} - x_i|.$$

Low values of $\mathrm{TV}(x)$ correspond to smooth and stable profiles, whereas large values indicate frequent or strong oscillations.

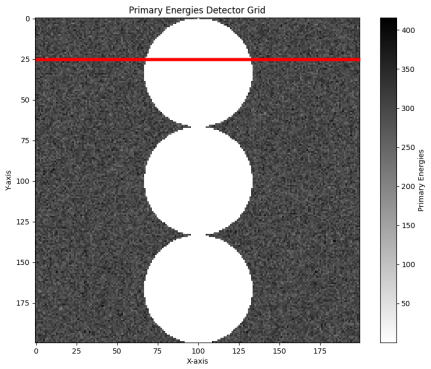
12.3 Evaluation Results

The evaluation results are based on the application of the simulation framework developed throughout this work (GitHub[1]). This framework allows for simulating X-ray photon transport through voxelized phantoms using different sampling strategies. Because the pseudo-random MC sampling strategy is non-deterministic, the simulations for the MC method were repeated five times per configuration and the results were averaged. To avoid biasing the plots, a representative plot is shown for the MC method in the following.

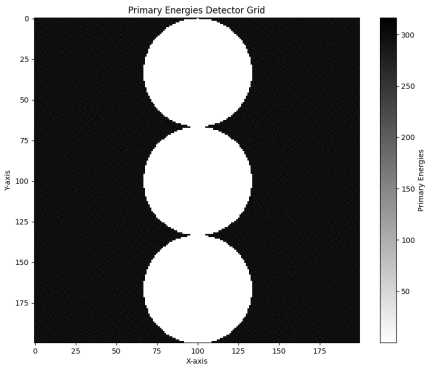
12.3.1 Primary Intensities

Although primary intensities can be computed directly by tracing only the straight rays from the source to each detector pixel and computing their attenuation along the way, in physically correct photon simulations they arise naturally whether desired or not. Primary intensities arise from photons with various directions and energies that reach the detector without any interaction. This leads to a certain amount of noise in the primary intensities, which can be reduced by increasing the number of simulated photons. As the following experiments will exhibit, the sampling strategy has a significant impact on the noise level of the primary intensities.

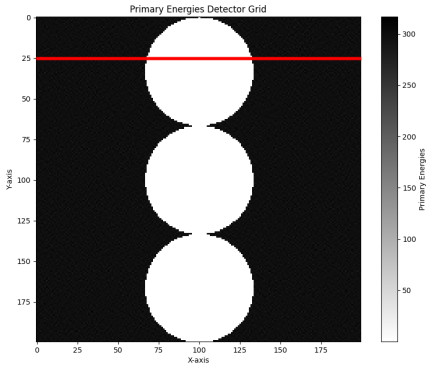
[1] https://github.com/janikvhrubant/photon-imaging-sim



(A) Monte Carlo sampling



(B) Halton sampling



(C) Sobol' sampling

FIGURE 12.3: Simulated primary intensities for an X-ray of the bone-in-water phantom using Monte Carlo, Halton and Sobol' sampling.

Figure 12.3 shows the simulated primary intensities for the bone-in-water phantom using different sampling strategies for the initial direction and photon energy. Whereas the MC result exhibits noticeable noise, the Halton and Sobol' results are significantly smoother with almost no visible noise. In the simulation shown, 5×10^6 photons are used for each sampling strategy.

The noise can be further observed in the intensity profile along the horizontal red line. In Figure 12.4, the intensity profiles for the sampling strategies are compared. They all show the expected intensity drop in the regions where the bone cylinders are located. However, the MC profile is significantly noisier than the Halton and Sobol' profiles.

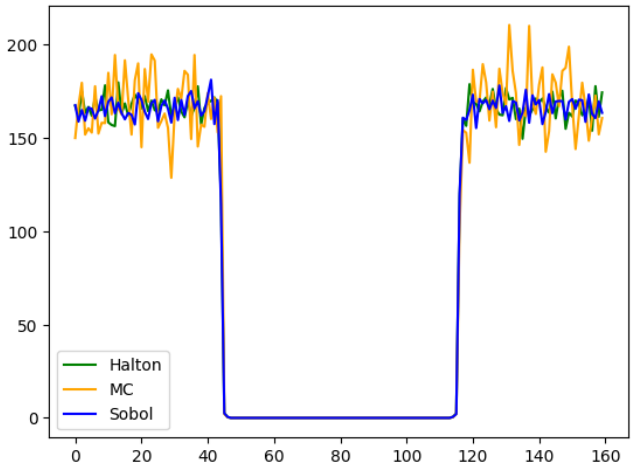


FIGURE 12.4: Intensity profile along the horizontal red line in Figure 12.3.

To confirm these results quantitatively, the noise according to Definition 12.2.1 is computed for the primary intensities in the water–wall phantom for each sampling strategy and various numbers of simulated photons. Figure 12.5 summarizes the results in a plot. Before calculating the noise, the intensities on the detector are normalized by the number of simulated photons to ensure comparability for different numbers of photons.

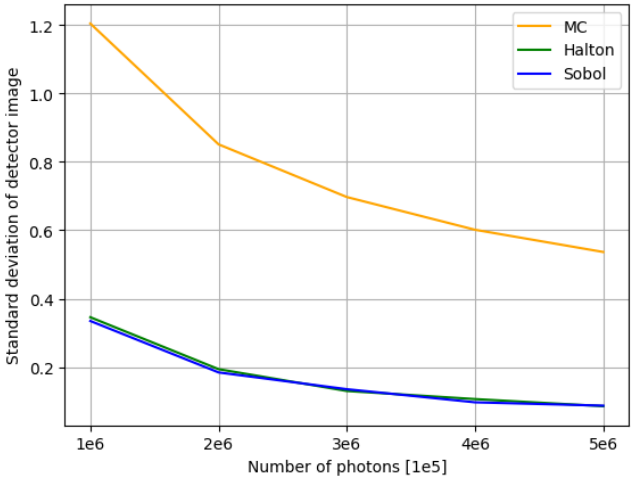


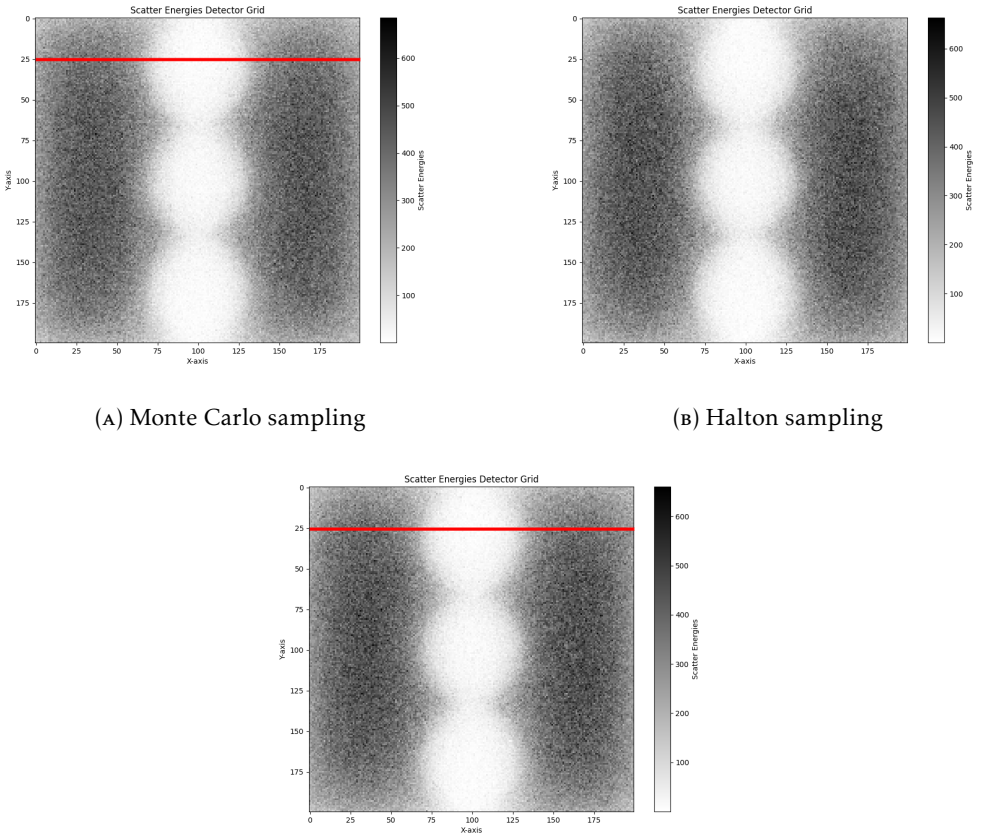
FIGURE 12.5: Plot of noise levels for primary intensities in water–wall Phantom for varying numbers of simulated photons.

12.3.2 Scattered Intensities

The chapter now turns to the evaluation of scattered intensities. Unlike primary intensities, which can be computed analytically and exhibit sharp edges that accurately represent the phantom's geometry, scattered intensities must be obtained through simulation according to the underlying physical interaction laws. As a consequence, scattered intensities appear more uniform and display smooth transitions rather than delineating precise structural boundaries. This effect arises from their inherently stochastic nature and from the fact that, depending on the attenuation

properties of the phantom, only a small fraction of scattered photons reach the detector. Scatter estimation is therefore particularly challenging and requires a large number of simulated photons to achieve sufficiently low noise levels.

In an experimental setup, the scattering is simulated for the bone-in-water phantom introduced in Section 12.1.2, applying the three different sampling strategies MC, Halton and Sobol'. The resulting scattered intensities are shown in Figure 12.6. Within these experiments 5×10^6 photons are simulated for each sampling strategy and the number of scattering events is set to 10.



(A) Monte Carlo sampling

(B) Halton sampling

(C) Sobol' sampling

FIGURE 12.6: Simulated scattered intensities for an X-ray of the bone-in-water phantom using Monte Carlo, Halton and Sobol' sampling.

Along the horizontal red line, the intensity profiles are extracted for the three sampling strategies, as shown in Figure 12.7. As in the full images, scattered intensities are significantly noisier than primary intensities.

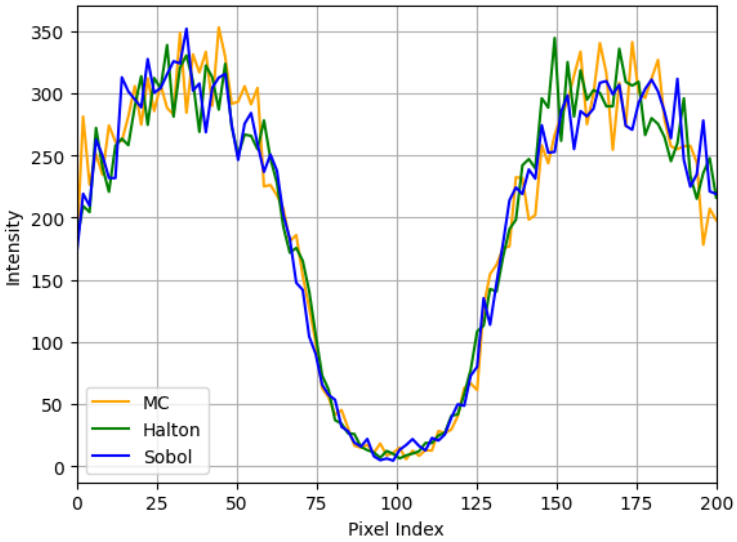


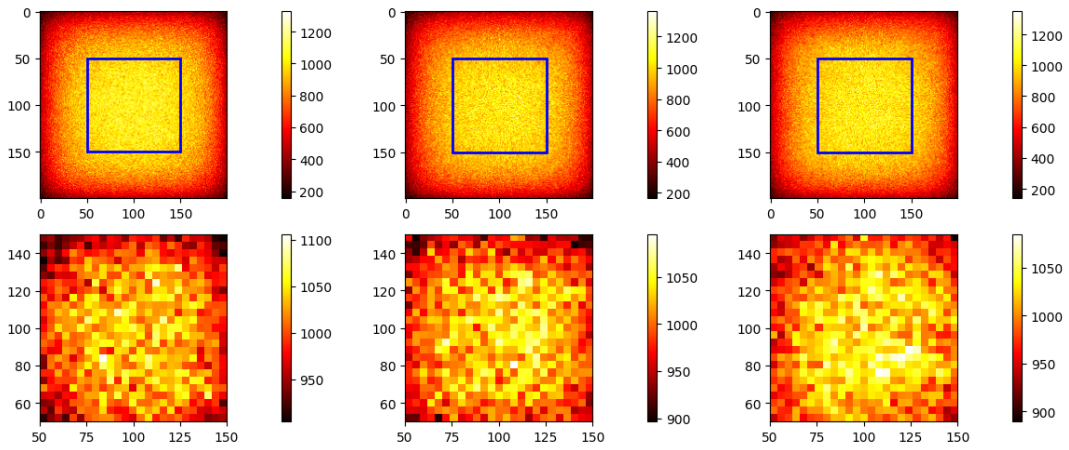
FIGURE 12.7: Intensity profile along the horizontal red line in Figure 12.6.

As it is difficult to visually assess the noise levels in the scattered intensity images, a quantitative evaluation is performed by computing the total variation along the profiles. In Table 12.1, the advantages of the Halton and Sobol' sampling strategies are summarized. For the horizontal red line, the total variation with Halton sampling is 8% lower than MC; the Sobol variation is 18% lower than the MC variation.

Sampling Strategy	Total variation on red lines	Total variation on all lines (averaged)
Halton	2095	2483
Sobol'	1866	2356
Monte Carlo (averaged)	2291	2517

TABLE 12.1: Total variation of the intensity of scattered photons for the bone-in-water phantom.

For a more detailed evaluation of the noise levels, the water–wall phantom is used again. This phantom is homogeneous and therefore the scattered intensities are expected to be spatially uniform across the entire detector. This allows a meaningful computation of the variance as a second metric for the evaluation of the noise levels.



(A) Monte Carlo (B) Halton (C) Sobol'

FIGURE 12.8: Comparison of intensity grids of scattered photons for the water–wall phantom using different sampling strategies.

As shown in Figure 12.8, the scattered intensities diminish towards the edges of the detector. This is due to the bounded scene and the detection geometry which captures photon intensities up to the edges of the scene. To avoid biasing the variance computation, the variance is computed only on the central 100×100 pixels of the detector and is bounded by the blue lines.

Tracking the variance within this region of interest for different numbers of simulated photons, the results are summarized in Figure 12.9. Here, the intensities are again normalized by the number of simulated photons to ensure comparability. The photon counts range from 0.5×10^6 to 5×10^6 in steps of 0.5×10^6.

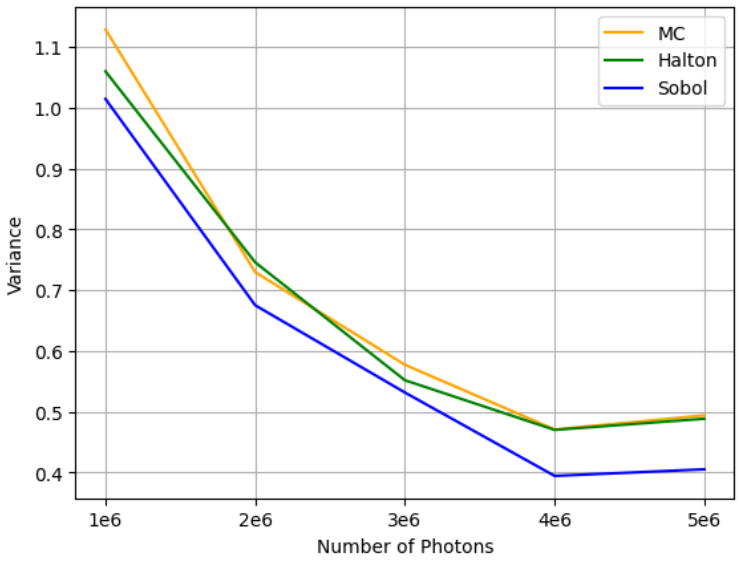


FIGURE 12.9: Noise of scattered photons on the water–wall for different photon quantities.

This experiment confirms the advantages of QMC sampling for scatter estimation.

Sobol' sampling consistently achieves the lowest noise levels. For the highest number of simulated photons (5×10^6), the Sobol' sampling strategy achieves a variance that is 16% lower than the MC variance. The Halton sampling strategy performs similarly to the MC sampling in this experiment.

The different results for the two experiments follow from the initialization type of the photons. Furthermore, the different phantoms have a significantly different attenuation behavior due to the bone structures.

12.4 Conclusion

The evaluation of QMC methods for X-ray scatter estimation demonstrated their potential to reduce noise levels in simulated images. Using both a homogeneous water–wall phantom and a more complex bone-in-water phantom, the experiments showed that low-discrepancy sequences such as Halton and Sobol' can outperform traditional MC sampling in terms of noise reduction.

Even at the primary stage, the distribution of primary intensities benefits significantly from the use of QMC methods. Both Halton and Sobol' sequences produced noticeably smoother primary intensity images compared to standard MC sampling. Therefore, only the first three random values per photon are needed.

Scattered intensities require more extensive sampling due to the complex interactions involved. For every scattered photon, three further random values are needed for each scattering event (scatter angle with energy, azimuthal angle, free path length). The number of scattering events is set to 10 in the experiments. This results in a total of $3 + 3 \times 10 = 33$ random values per photon. Here, the Sobol' sequence consistently achieved the lowest noise levels, with reductions of up to 18% in the bone-in-water phantom and 16% in the water–wall phantom compared to MC sampling.

When sampling scattered photons, the photon intensity (for the forced fixed detection simulation) and the photon energy decrease at every scattering event. Therefore, the first scattering events contribute the most to the overall intensity at the detector grid. Since six random values are needed for the photon generation and the first scattering event, low-discrepancy sequences as the Halton sequence suffer from the curse of dimensionality yielding little advantage over MC simulations. As the experiments suggest, the Sobol' sequence overcomes this problem. This can be explained by the fact that the Sobol' sequence maintains low-discrepancy in higher dimensions better than the Halton sequence. In [22], it is shown that the Halton sequence starts to suffer from the sixth dimension.

12.5 Future Work

The evaluation presented in this chapter provides a solid foundation for understanding the benefits of QMC methods in X-ray scatter estimation. However, several avenues for future research remain open.

As shown in the experiments, the curse of dimensionality affects the performance of low-discrepancy sequences. This suggests a hybrid approach: use low-discrepancy sequences for the first few random values and switch to pseudo-random sampling for later values. Therefore, future work could explore such hybrid sampling strategies and evaluate their effectiveness in reducing noise levels.

It would be interesting to extend the evaluation to an optimization problem, where the goal is to minimize the noise level for a certain phantom and determine the optimal number of QMC dimensions to use.

Further enhancements to the simulation framework could include integrating this hybrid approach and enhancing the framework's robustness for a wider range of phantoms and imaging scenarios.

Bibliography

[1] Takuya Akiba et al. "Optuna: A next-generation hyperparameter optimization framework". In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, pp. 2623–2631.

[2] M. J. Berger et al. *XCOM: Photon Cross Sections Database*. *NIST Standard Reference Database 8 (XGAM)*. Last Update to Data Content: November 2010. NBSIR 87-3597. 2010. DOI: 10.18434/T48G6X. URL: https://www.nist.gov/pml/xcom-photon-cross-sections-database.

[3] Vladimir I Bogachev. *Measure theory*. Springer, 2007.

[4] Thorsten Buzug. *Computed tomography: From photon statistics to modern cone-beam CT*. English. Springer Berlin Heidelberg, Jan. 2014. ISBN: 978-3-540-39407-5. DOI: 10.1007/978-3-540-39408-2.

[5] Russel E Caflisch. "Monte carlo and quasi-monte carlo methods". In: *Acta numerica* 7 (1998), pp. 1–49.

[6] Geant4 Collaboration. *Geant4: A Simulation Toolkit*. Version 11.3.2. 2025. URL: https://geant4.web.cern.ch/.

[7] George Cybenko. "Approximation by superpositions of a sigmoidal function". In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.

[8] Bastien Doignies. "Echantillonnage différentiable et simulations Monte Carlo". PhD thesis. Université Claude Bernard-Lyon I, 2024.

[9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[10] John H Halton. "On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals". In: *Numerische Mathematik* 2.1 (1960), pp. 84–90.

[11] Michael Hardy. *Combinatorics of Partial Derivatives*. 2006. arXiv: math/0601149 [math.CO]. URL: https://arxiv.org/abs/math/0601149.

[12] J Hubbell. "Photon cross sections, attenuation coefficients and energy absorption coefficients". In: *National Bureau of Standards Report NSRDS-NBS29, Washington DC* (1969).

[13] "Interview with Bastien Doignies on Quasi-Monte Carlo Methods for X-Ray Simulation". Conducted on May 26, 2025. 2025.

[14] H.W.A.M. de Jong, E.T.P. Slijpen, and F.J. Beekman. "Acceleration of Monte Carlo SPECT simulation using convolution-based forced detection". In: *IEEE Transactions on Nuclear Science* 48.1 (2001), pp. 58–64. DOI: 10.1109/23.910833.

[15] Oskar Klein and Yoshio Nishina. "The scattering of light by free electrons according to Dirac's new relativistic dynamics". In: *Nature* 122.3072 (1928), pp. 398–399.

[16] Gunther Leobacher and Friedrich Pillichshammer. *Introduction to Quasi-Monte Carlo integration and applications*. Springer, 2014.

[17] Guiyuan Lin, Shiwo Deng, and Xiaoqun Wang. "An efficient quasi-Monte Carlo method with forced fixed detection for photon scatter simulation in CT". In: *PLOS ONE* 18 (Aug. 2023), e0290266. DOI: 10.1371/journal.pone.0290266.

[18] Guiyuan Lin et al. "Scatter correction based on quasi-Monte Carlo for CT reconstruction". In: *arXiv preprint arXiv:2501.05039* (2025).

[19] Kjetil O Lye, Siddhartha Mishra, and Deep Ray. "Deep learning observables in computational fluid dynamics". In: *Journal of Computational Physics* 410 (2020), p. 109339.

[20] *Medical Imaging Systems : An Introductory Guide*. eng. Image Processing, Computer Vision, Pattern Recognition, and Graphics; 11111. Cham, 2018. Chap. 7,8. ISBN: 9783319965208.

[21] Siddhartha Mishra and T Konstantin Rusch. "Enhancing accuracy of deep learning algorithms by training with low-discrepancy sequences". In: *SIAM Journal on Numerical Analysis* 59.3 (2021), pp. 1811–1834.

[22] William J Morokoff and Russel E Caflisch. "Quasi-monte carlo integration". In: *Journal of computational physics* 122.2 (1995), pp. 218–230.

[23] Thomas Müller-Gronbach, Erich Novak, and Klaus Ritter. *Monte Carlo-Algorithmen*. Springer-Verlag, 2012.

[24] Andreas Neuenkirch and Peter Parczewski. *Monte Carlo Methods: Lecture Notes*. FSS 2025++, revised version, June 24, 2025. University of Mannheim, 2025.

[25] Art B Owen. "Multidimensional variation for quasi-Monte Carlo". In: *International Conference on Statistics in honour of Professor Kai-Tai Fang's 65th birthday*. World Scientific. 2005, pp. 49–74.

[26] Emin N Özmutlu. "Sampling of angular distribution in Compton scattering". In: *International journal of radiation applications and instrumentation. Part A. Applied radiation and isotopes* 43.6 (1992), pp. 713–715.

[27] Adam Paszke et al. "Pytorch: An imperative style, high-performance deep learning library". In: *Advances in neural information processing systems* 32 (2019).

[28] Gavin Poludniowski. *SpekPy: Python package for simulating X-ray spectra*. Version 2.0.13. 8.08.2024. URL: https://bitbucket.org/caxtus/book/src/master/.

[29] Gavin Poludniowski, Artur Omar, and Pedro Andreo. *Calculating X-ray Tube Spectra: Analytical and Monte Carlo Approaches*. CRC Press, 2022.

[30] Gavin Poludniowski et al. "SpekPy v2. 0—a software toolkit for modeling x-ray tube spectra". In: *Medical Physics* 48.7 (2021), pp. 3630–3637.

[31] SciPy Developers. *Quasi-Monte Carlo Sampling Tools: `scipy.stats.qmc`*. https://docs.scipy.org/doc/scipy/reference/stats.qmc.html. Accessed: 2025-08-18. 2025.

[32] A. Sisniega et al. "Automatic Monte-Carlo Based Scatter Correction For X-ray cone-beam CT using general purpose graphic processing units (GP-GPU): A feasibility study". In: *2011 IEEE Nuclear Science Symposium Conference Record*. 2011, pp. 3705–3709. DOI: 10.1109/NSSMIC.2011.6153699.

[33] Jörg Steidel et al. "Dose reduction potential in diagnostic single energy CT through patient-specific prefilters and a wider range of tube voltages". In: *Medical physics* 49.1 (2022), pp. 93–106.

[34] Pauli Virtanen et al. "SciPy 1.0: fundamental algorithms for scientific computing in Python". In: *Nature methods* 17.3 (2020), pp. 261–272.

[35] Markus Zahrnhofer. *Einführung in Quasi-Monte Carlo Verfahren*. 2007.